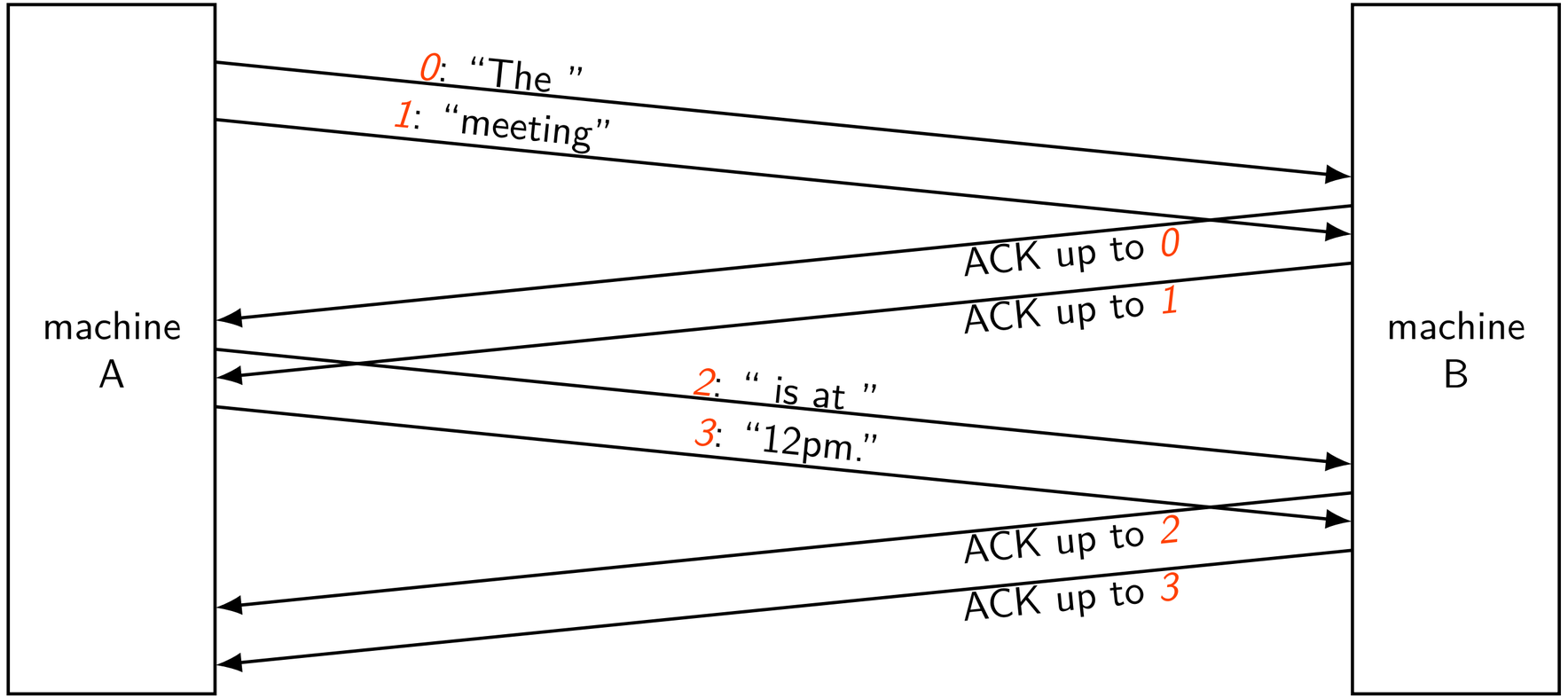
sending two at a time



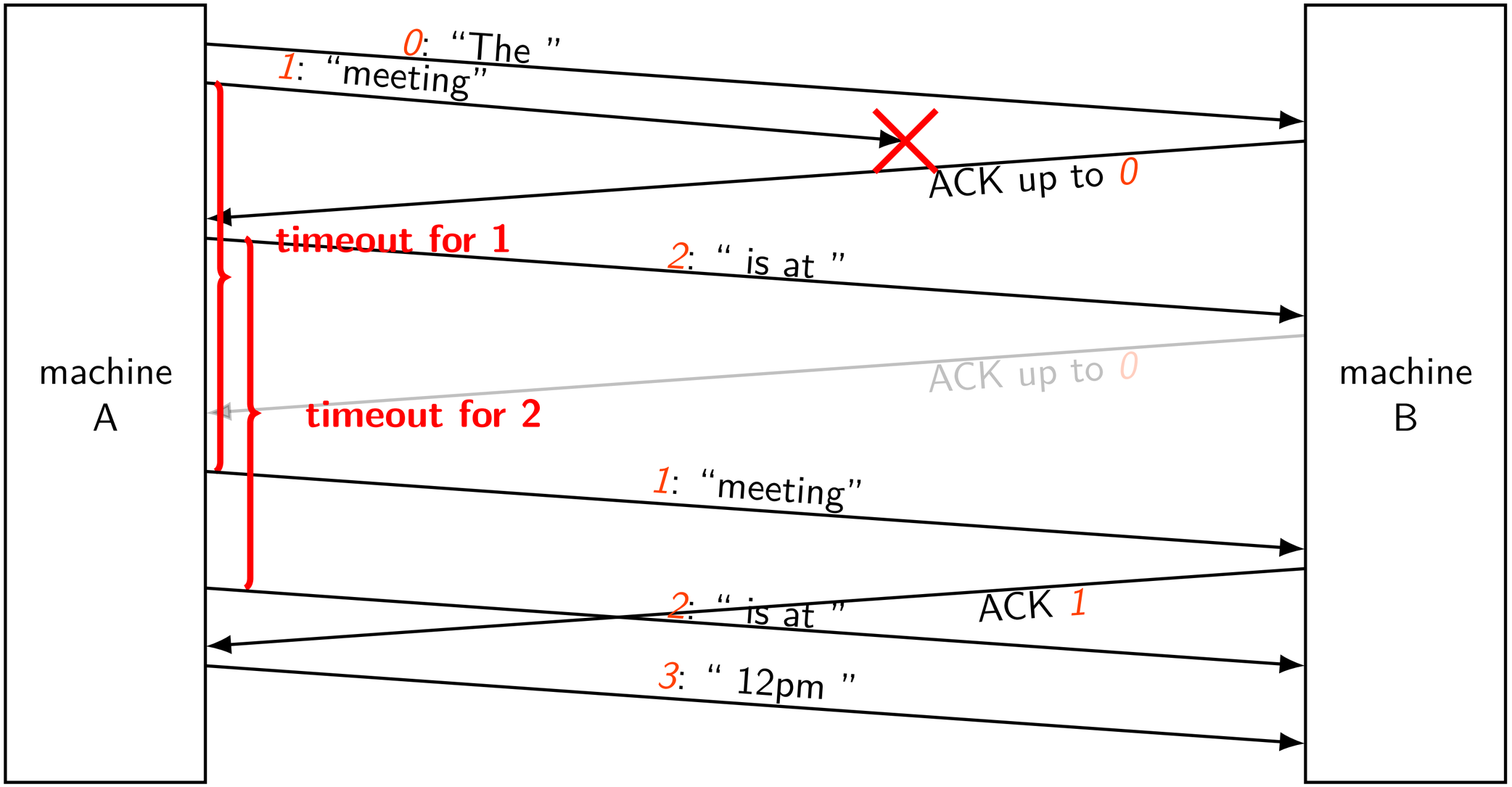
(ACK up to X = ACK X and everything before it)

key idea: always have two in flight

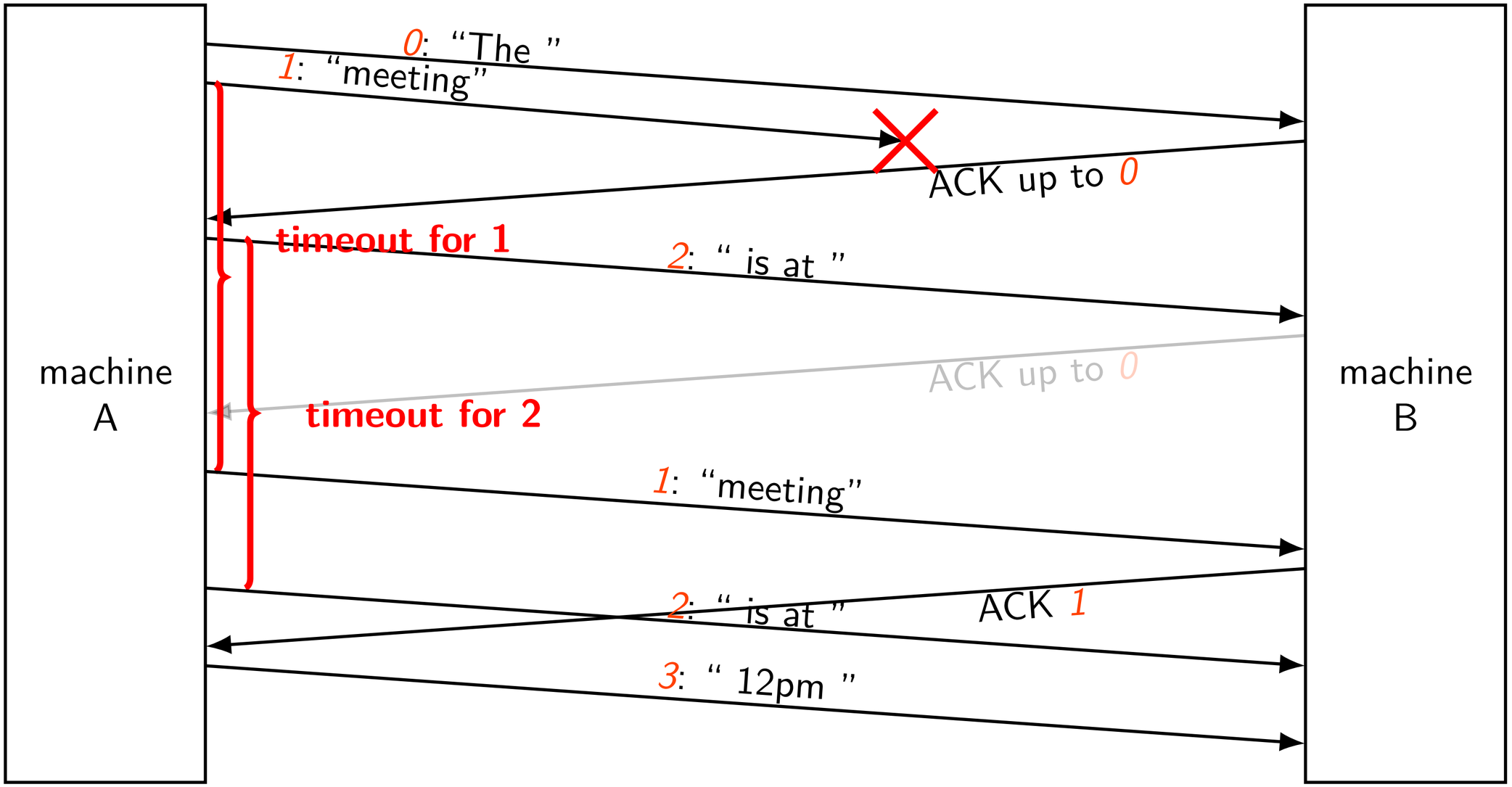
send next when previous ack'd

timeouts per message

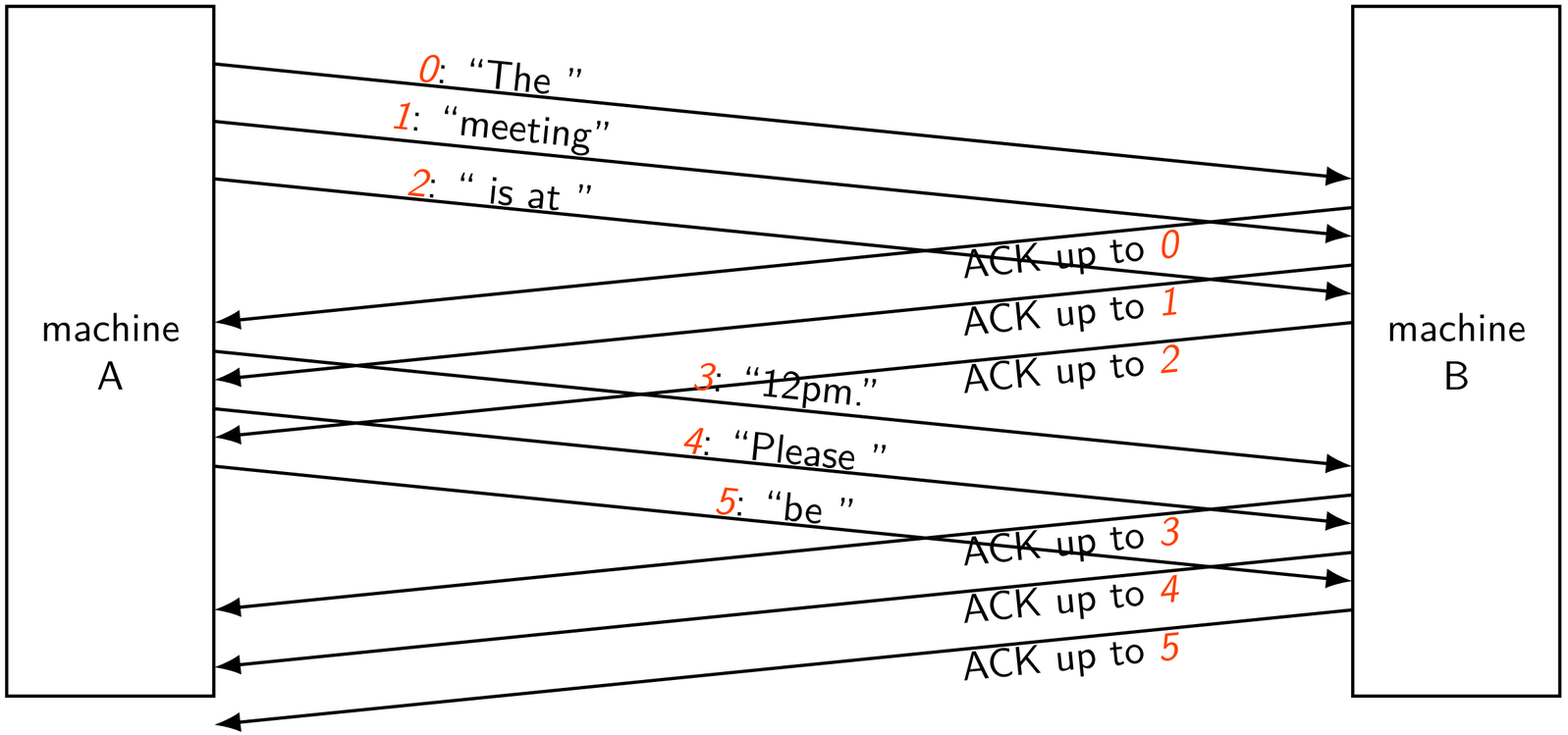
timeouts per message



timeouts per message



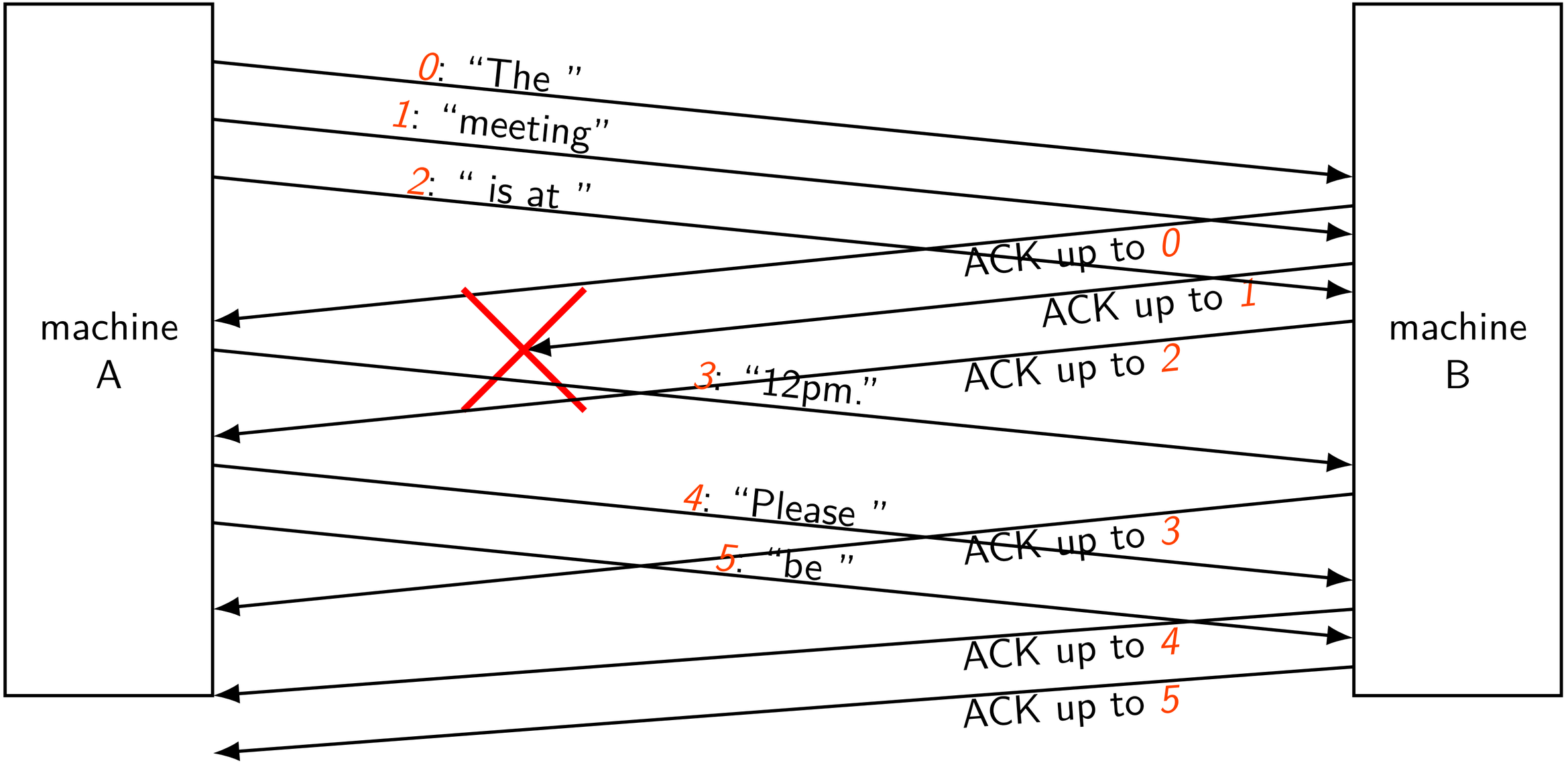
sending three at a time



choose "window size" to have in flight

send when previous acknowledged

lost ACKs?


machine A
machine B
0: "The "
1: "meeting"
2: " is at "
ACK up to 0
ACK up to 1
3: "12pm."
ACK up to 2
4: "Please "
5: "be "
ACK up to 3
ACK up to 4
ACK up to 5

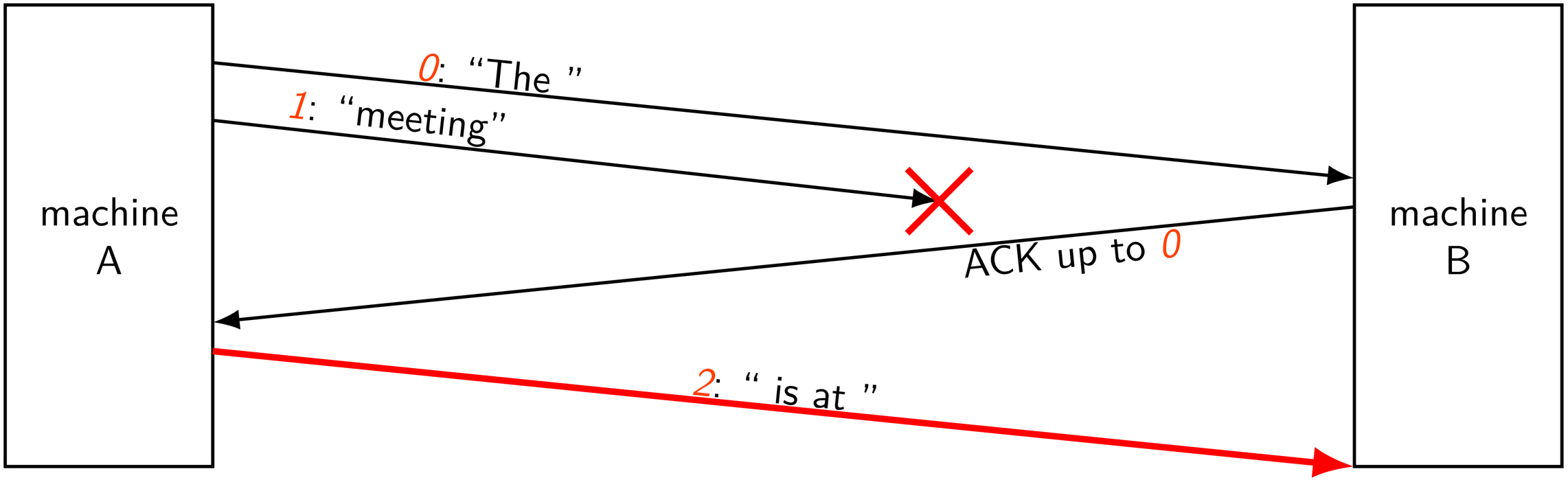
missing messages?

question: what should receiver do with sequence number 2?

one idea: ignore it?

better idea: send something back to sender

missing messages?

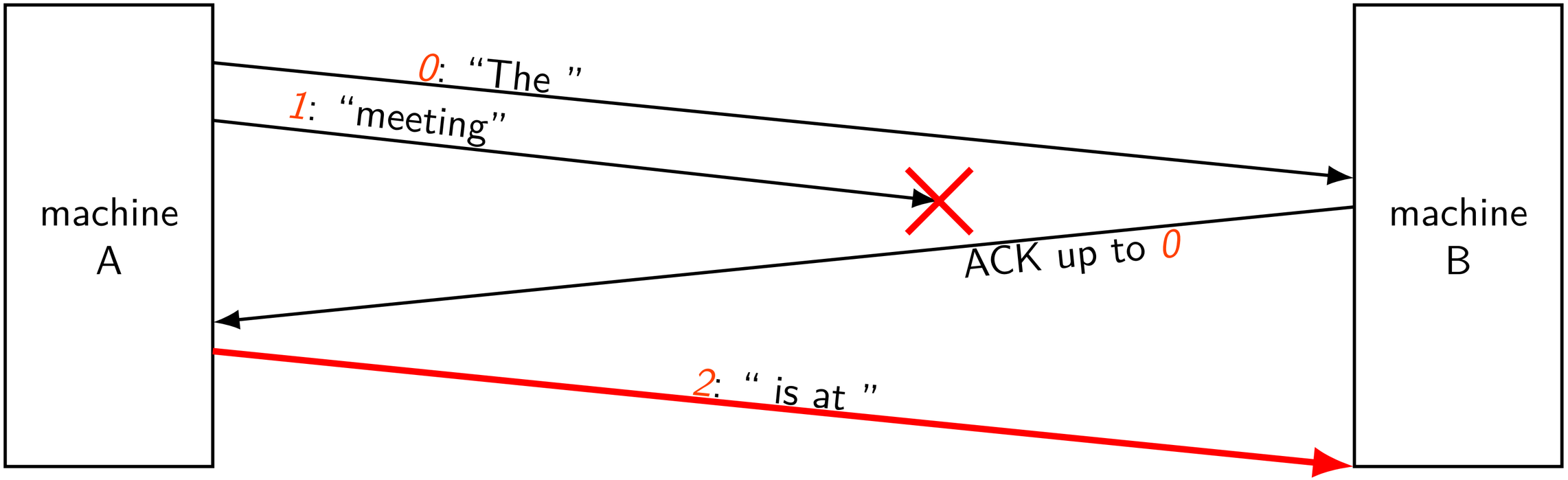


question: what should receiver do with sequence number 2?

one idea: ignore it?

better idea: send something back to sender

missing messages?



question: what should receiver do with sequence number 2?

one idea: ignore it?

better idea: send something back to sender

missing messages?

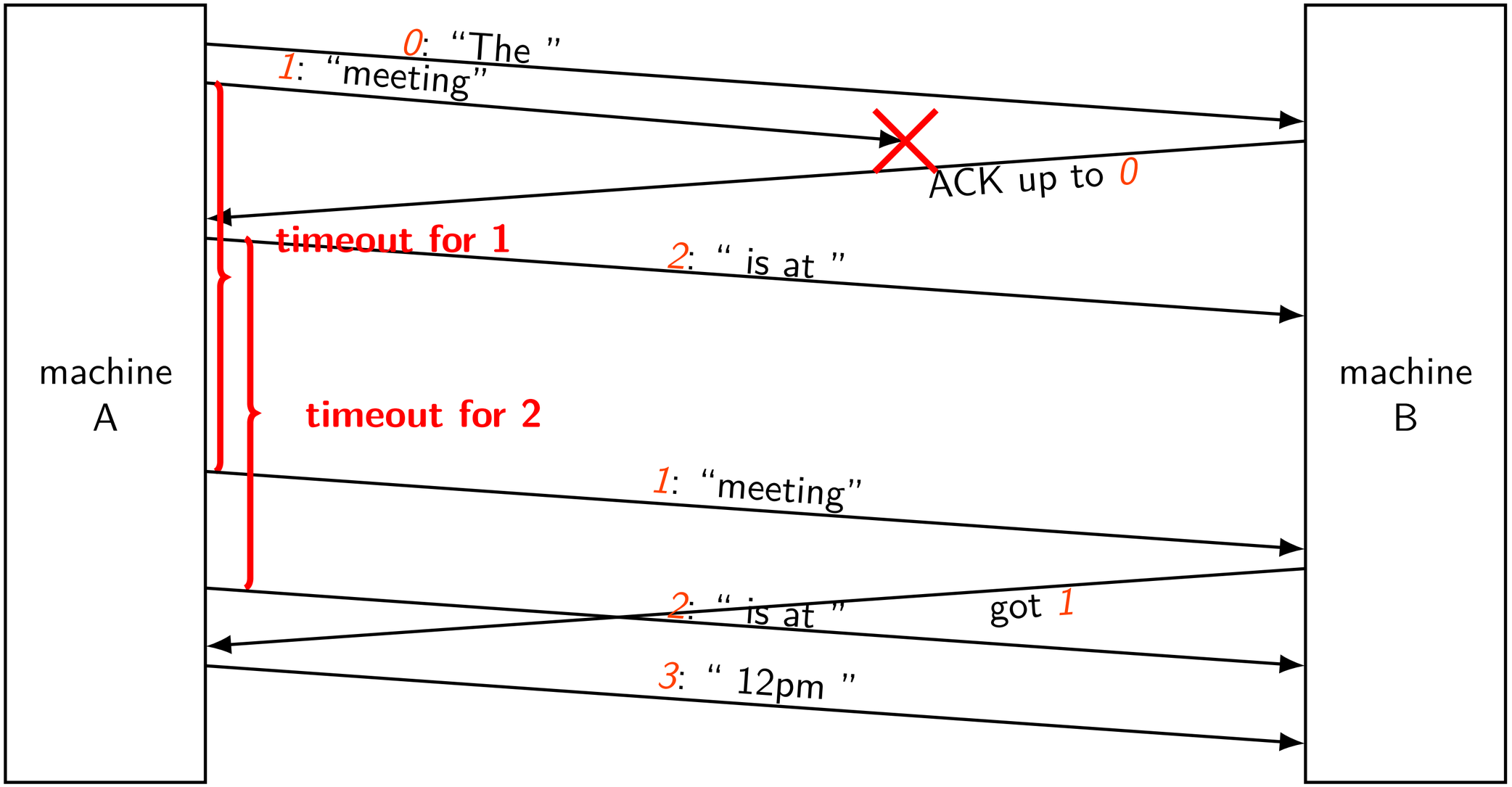
question: what should receiver do with sequence number 2?

one idea: ignore it?

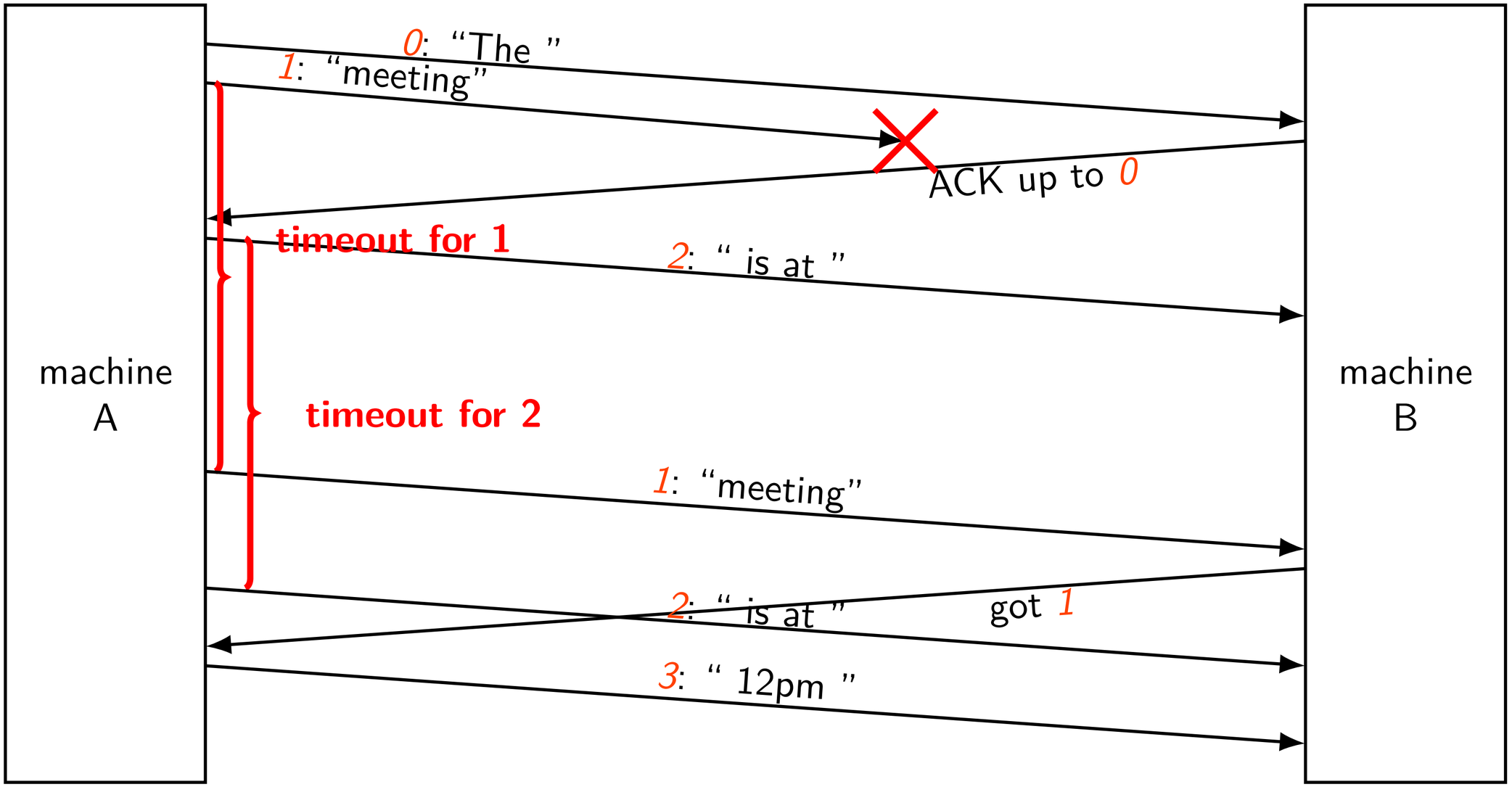
better idea: send something back to sender

not great: ignore it (1)

not great: ignore it (1)



not great: ignore it (1)



not great: only ACK if next (2)

works because we still have timeouts

but we'd like to give more feedback to receiver

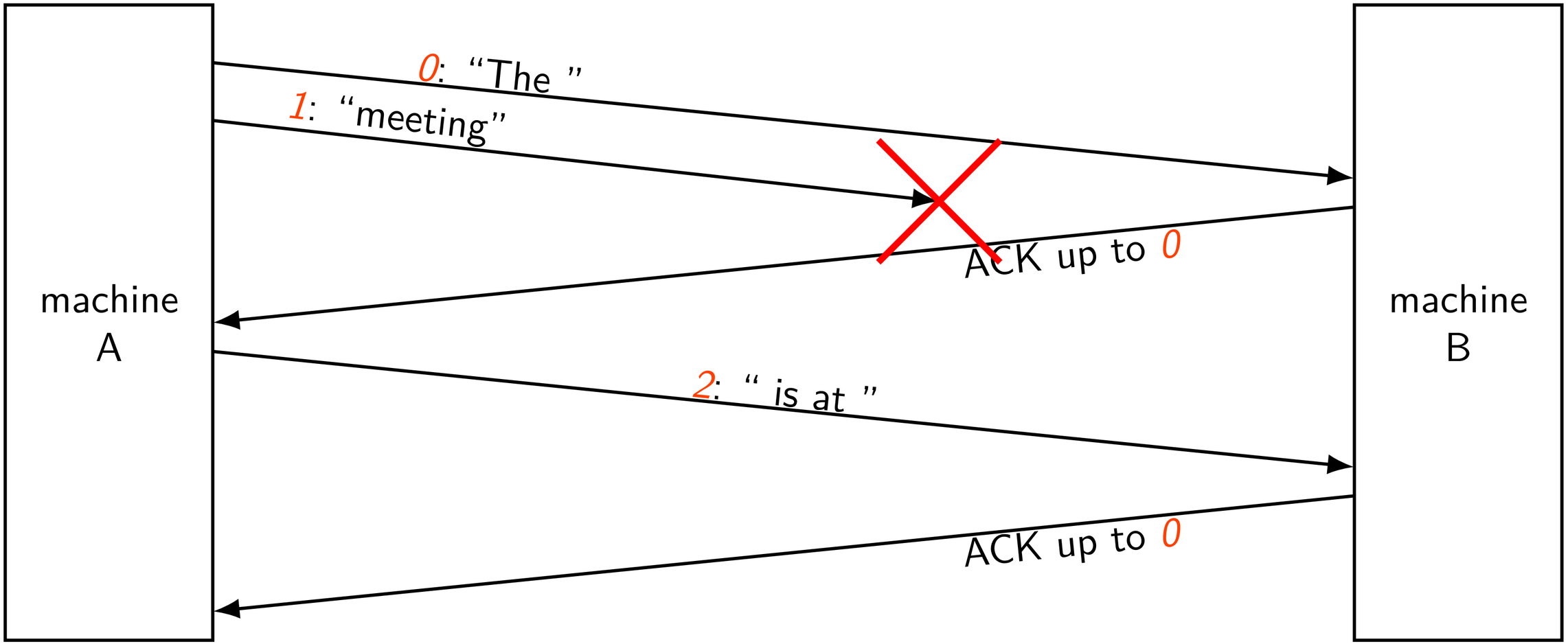
againframe(twoAndTimeouts)

better idea: always ACK

only ACK x if *everything up to and including* x received

intuition: ACK tells sender *where to start sending more*

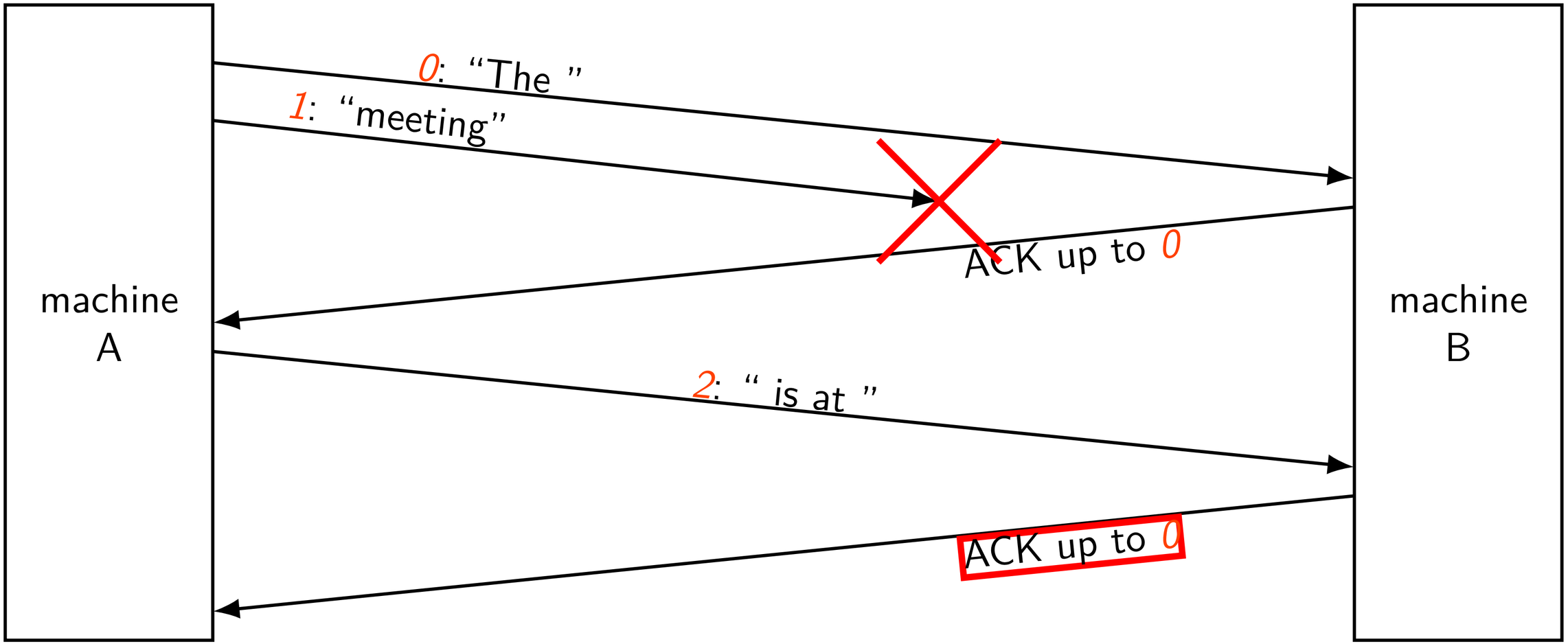
better idea: always ACK



only ACK x if *everything up to and including* x received

intuition: ACK tells sender *where to start sending more*

better idea: always ACK



only ACK x if *everything up to and including* x received

intuition: ACK tells sender *where to start sending more*

fast retransmit

if large window + data packet 2 is lost, then sender will see

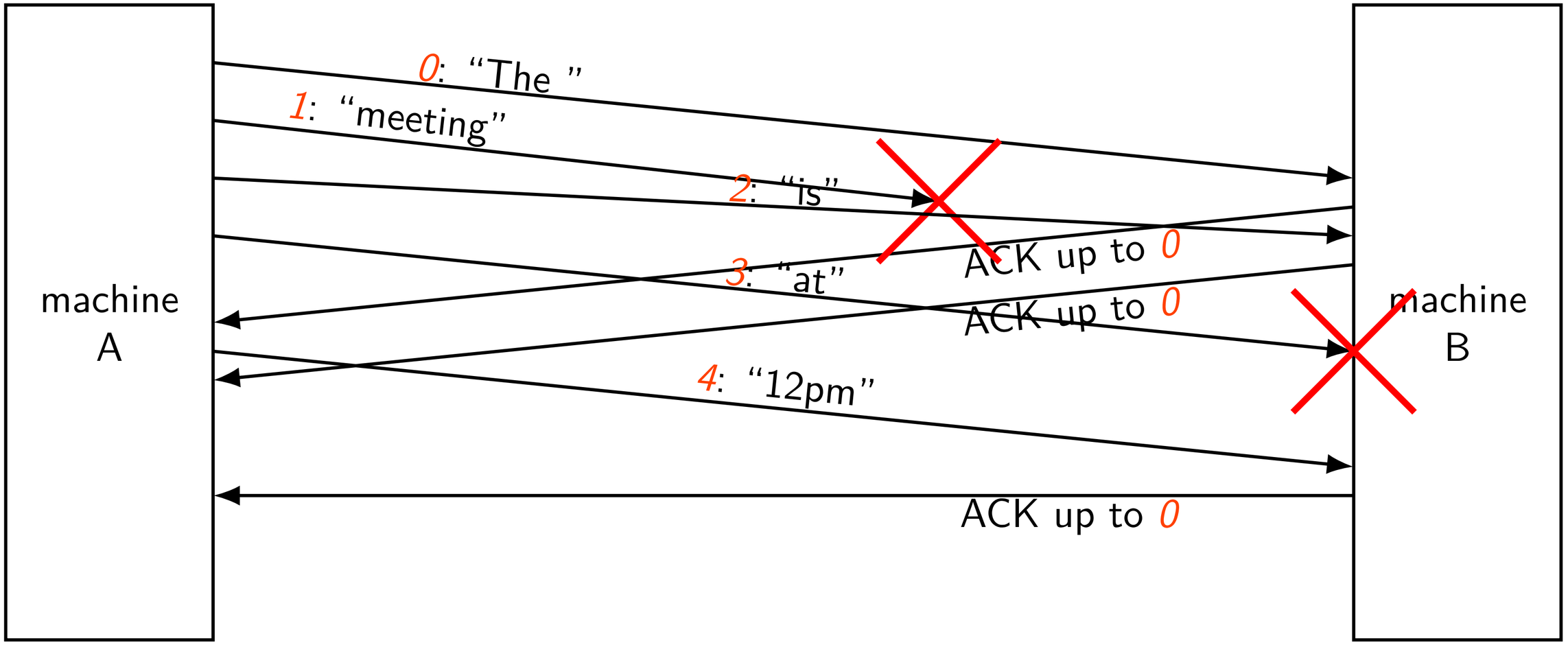
ACK 0, *ACK 1, ACK 1, ACK 1, ACK 1, ACK 1*

duplicate ACKs indicate missing packet 2

shouldn't wait for timeout

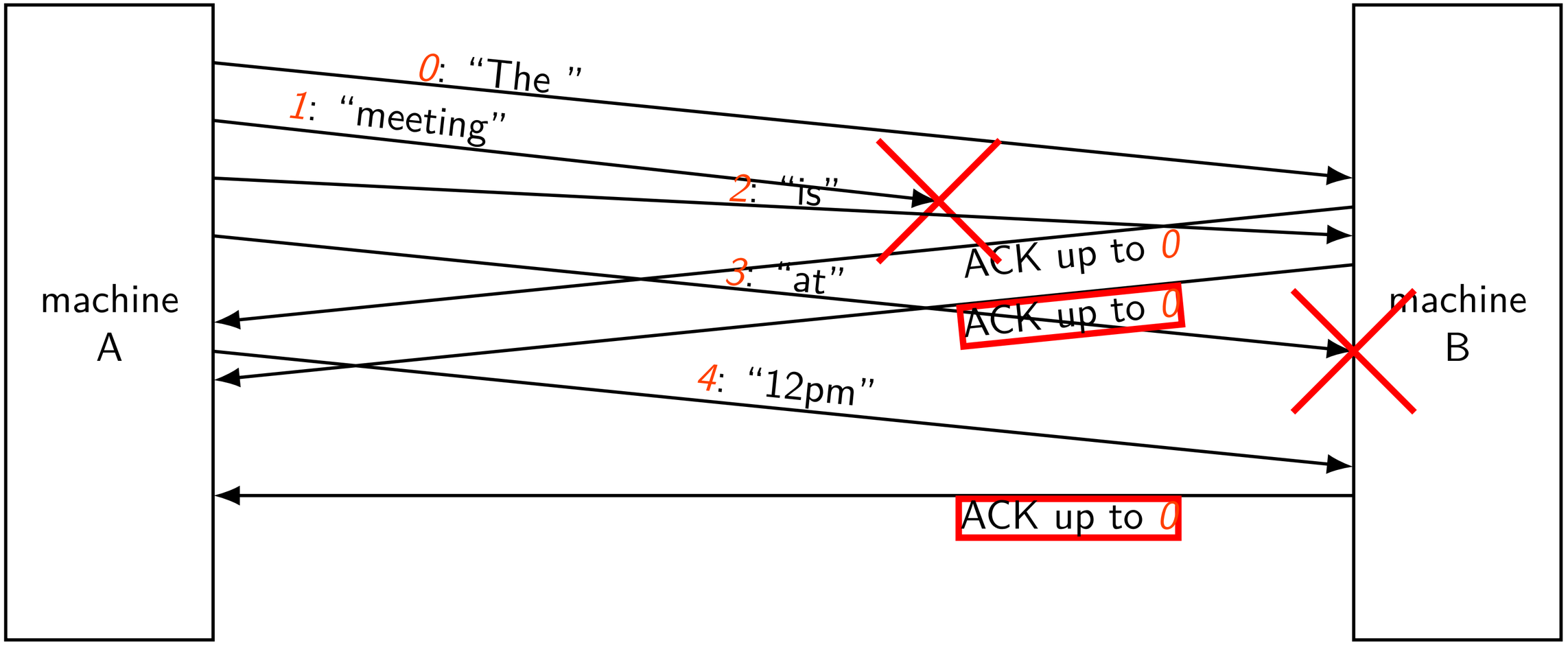
→ TCP heuristic: retransmit immediately after $$3 duplicate ACKs
not 1 duplicate ACK to tolerate some reordering
also some other details (we'll talk later)

multiple missing



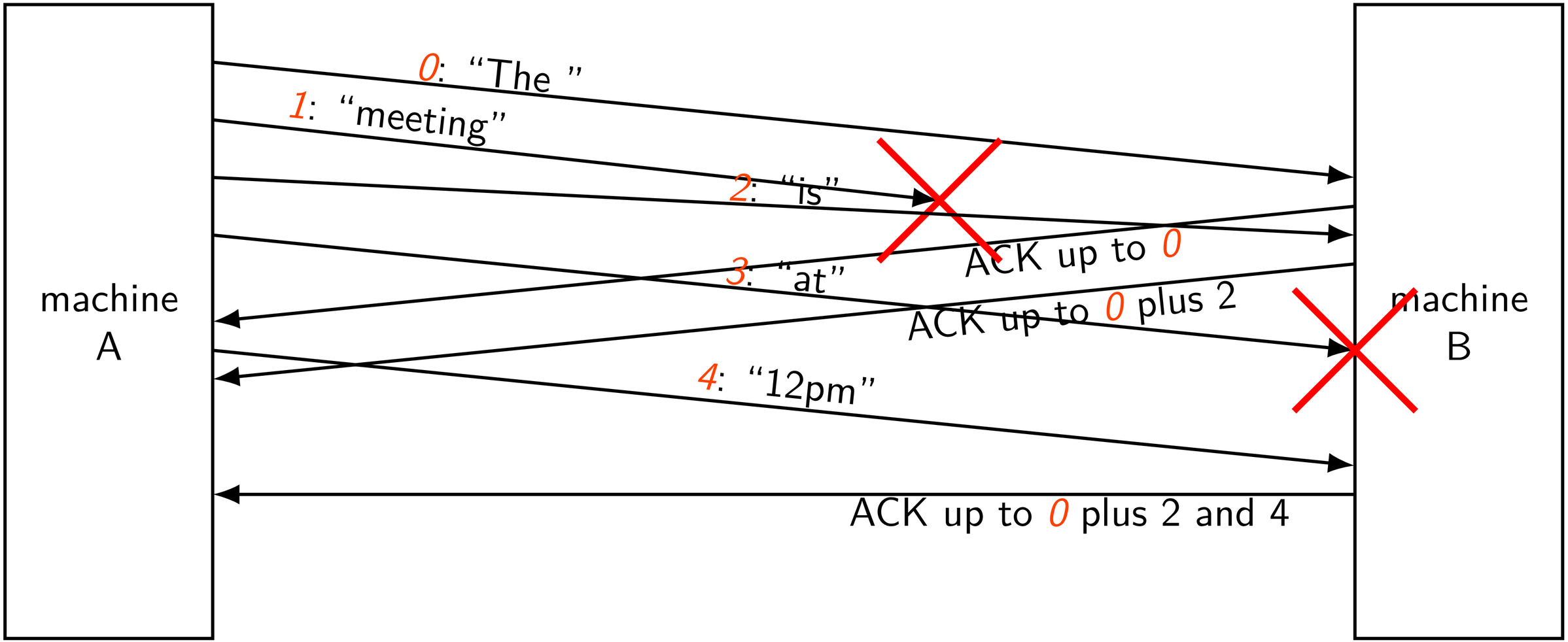
duplicate ACK heuristic will quickly resend 1, but not 3

multiple missing

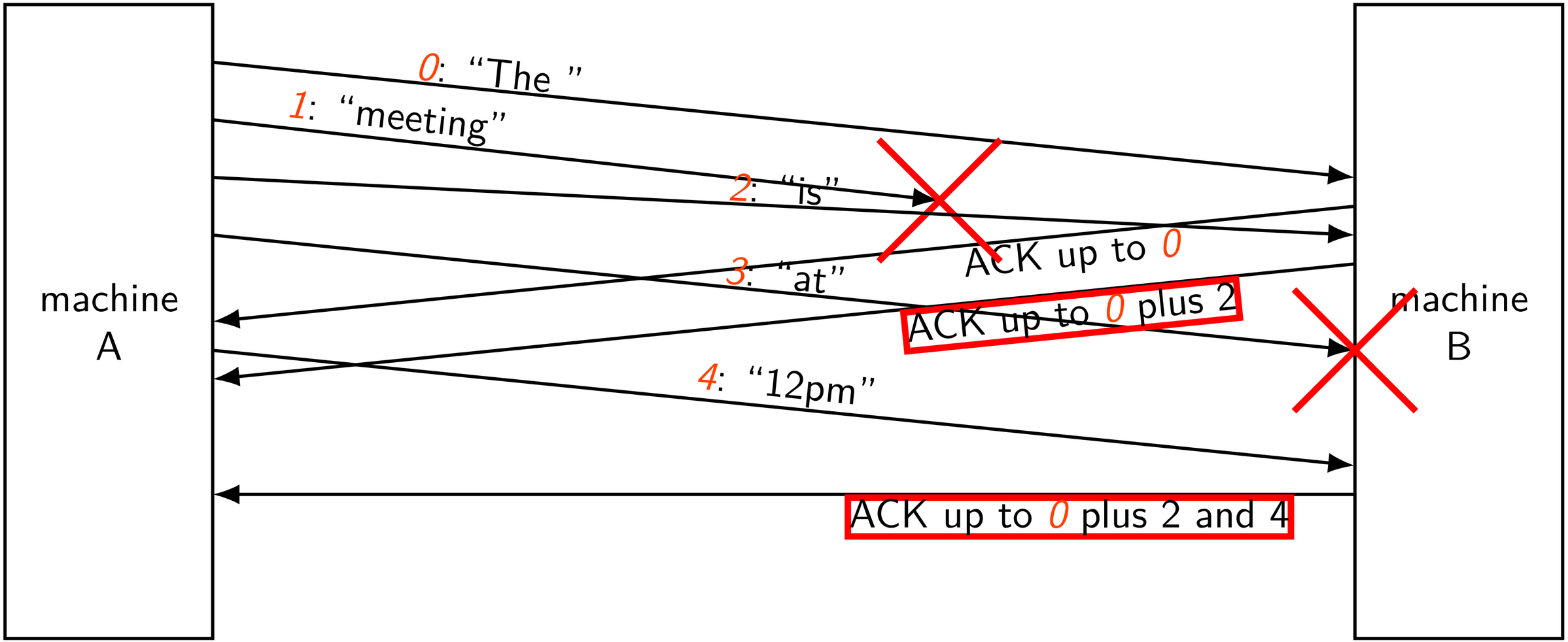


duplicate ACK heuristic will quickly resend 1, but not 3

selective acknowledgments



selective acknowledgments



selective acknowledgments in TCP

optional feature (“extension”) described in RFC 2018

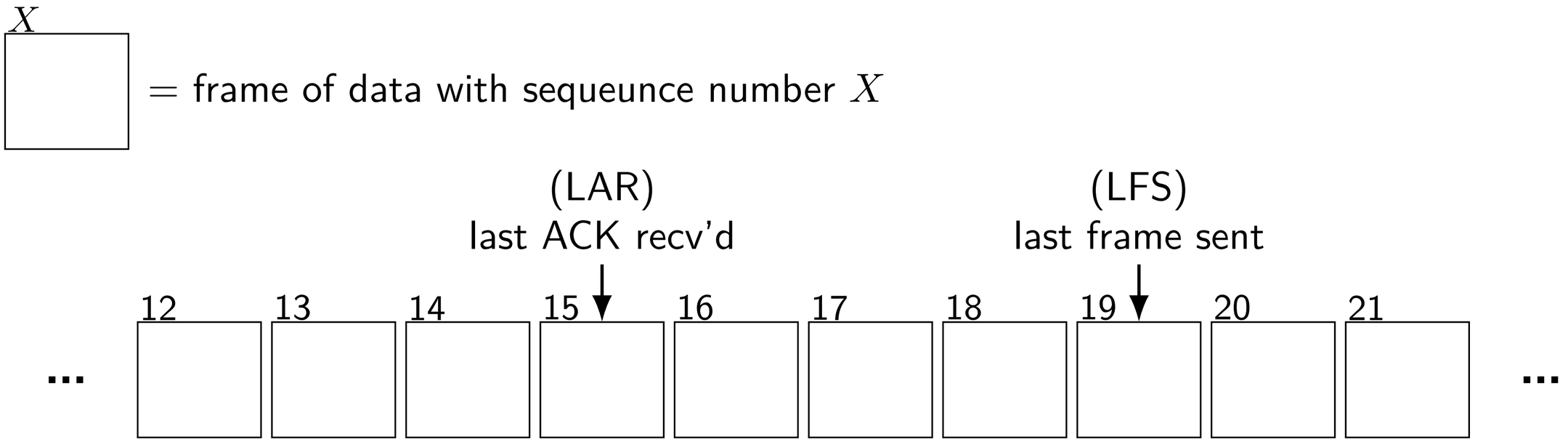
send list of ranges received

typically room for 3 ranges

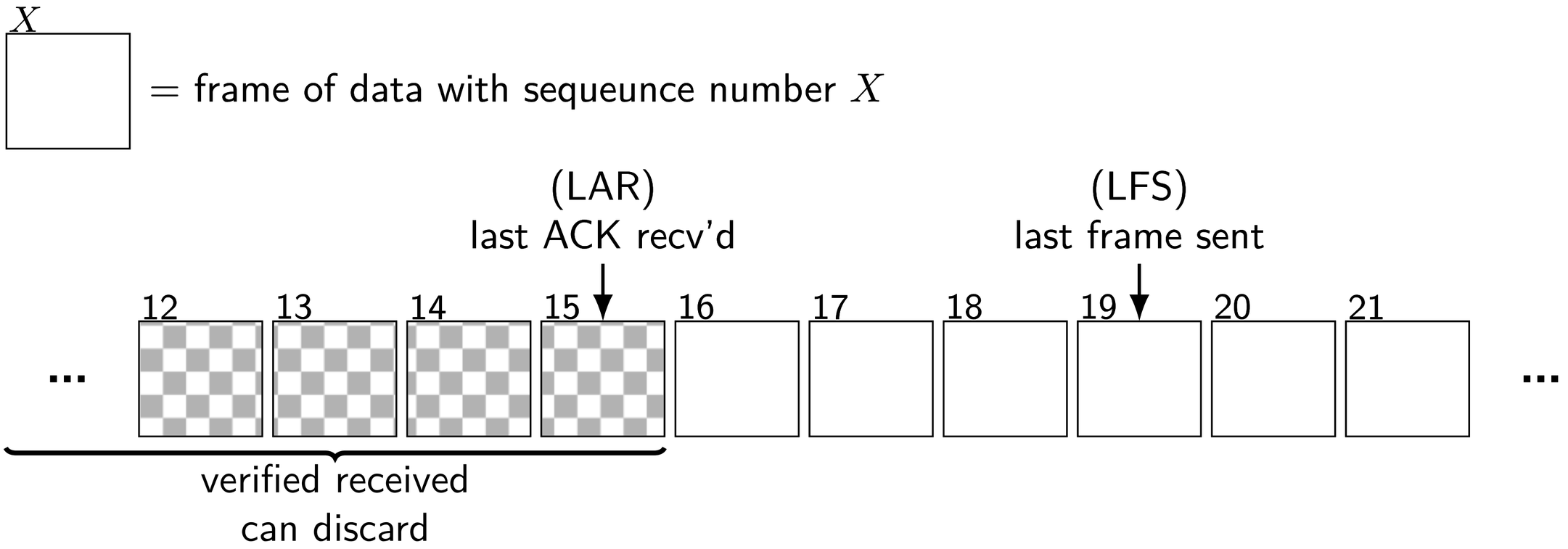
if more than 3 ranges to report, then:

- include range with most recently received frame
- include other ranges until sent three times

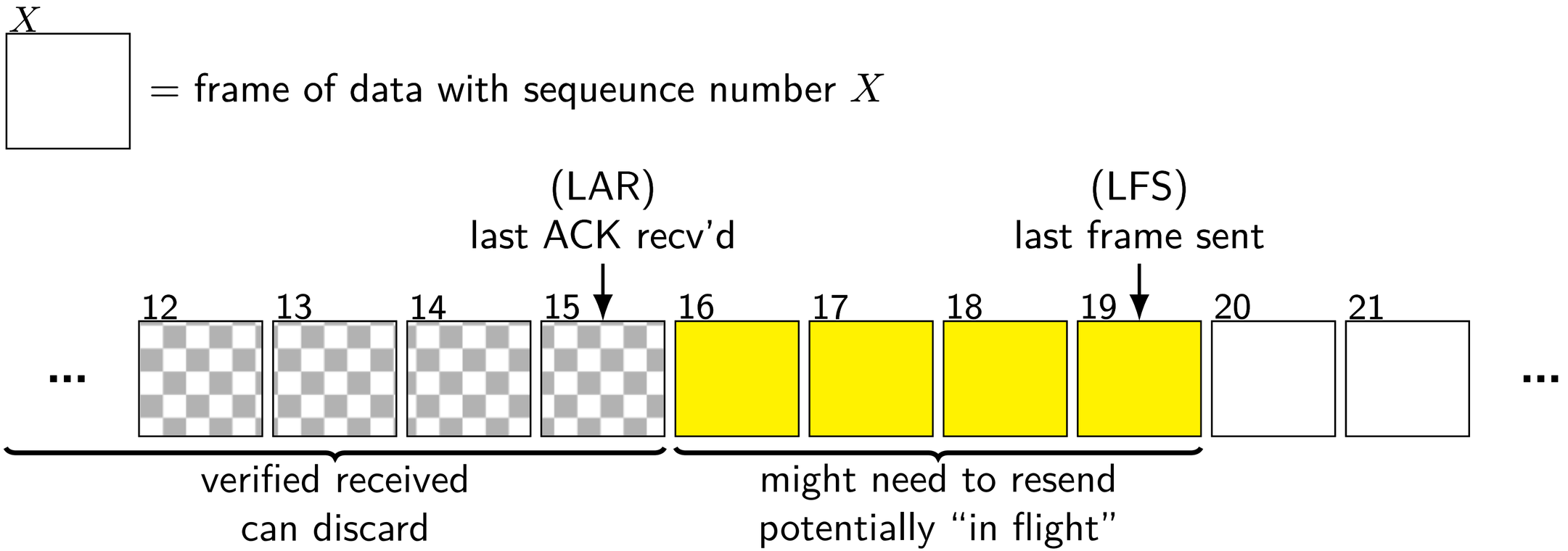
sender window tracking



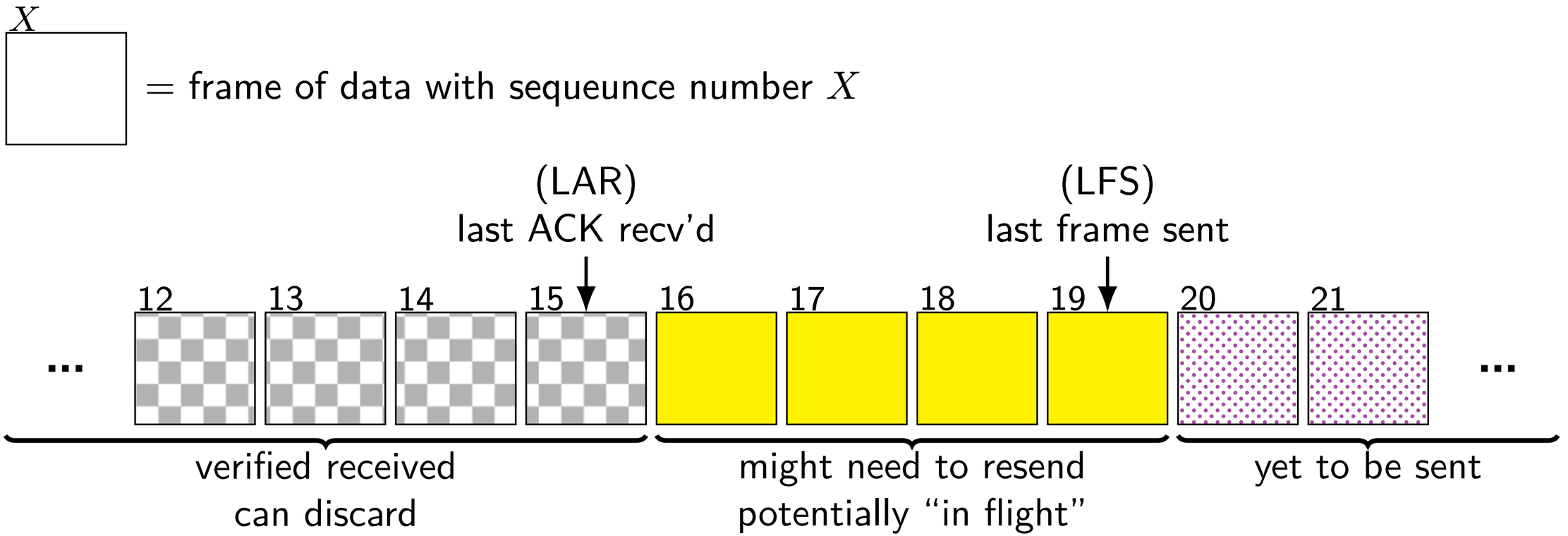
sender window tracking



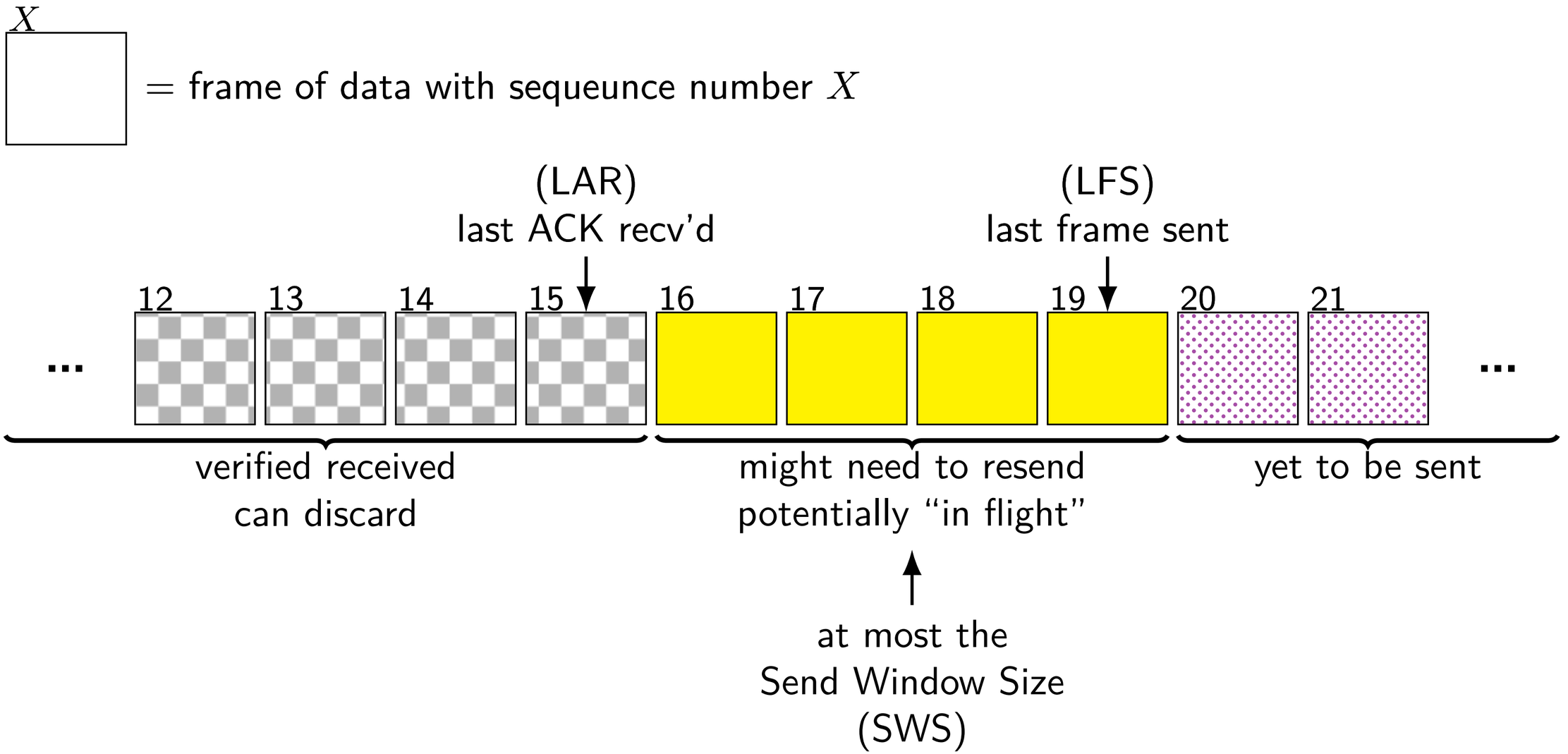
sender window tracking



sender window tracking



sender window tracking



sender window tracking

exercise 1: out-of-bounds ACK

last ACK recv'd (LAR)	10
last frame sent (LFS)	15
send window size (SWS)	5

what probably happened if we receive an ACK for…

9? 10? 13? 16?

A. if network reorders frames
B. lost ACK for frame ≤ 10
C. lost ACK for frame > 10
D. lost frame 11
E. resent frame from timeout

exercise 2: sender logic

last ACK recv'd (LAR)	10
last frame sent (LFS)	15
send window size (SWS)	5

In this case, there's a timeout that will trigger frame 13 to be resent. If still active, this timeout should be cancelled upon …

A. receiving ACK 12 B. receiving ACK 13

C. receiving ACK 14 D. sending frame 16

exercise 3a: new data

last ACK recv'd (LAR)	4
last frame sent (LFS)	6
send window size (SWS)	5

if we compute a new frame of data with sequence number 7 to eventually send, we should

A. send it now, advancing LFS
B. wait until we get an ACK for 5 or 6 to send it
C. wait until we get an ACK for 6 to send it
D. wait until the frame with sequence number 6 is resent to send it
D. something else

exercise 3b: new data

last ACK recv'd (LAR)	4
last frame sent (LFS)	8
send window size (SWS)	4

if we compute a new frame of data with sequence number 9 to eventually send, we should

A. send it now, advancing LFS
B. wait until we get an ACK for 5 or 6 or 7 or 8 to send it
C. wait until we get an ACK for 6 or 7 or 8 to send it
D. decline to accept the data because we will never be able to send it
E. something else

sender logic summarized

track variables:

- LFS (last frame sent)
- LAR (last ACK recv'd)
- SWS (send window size)

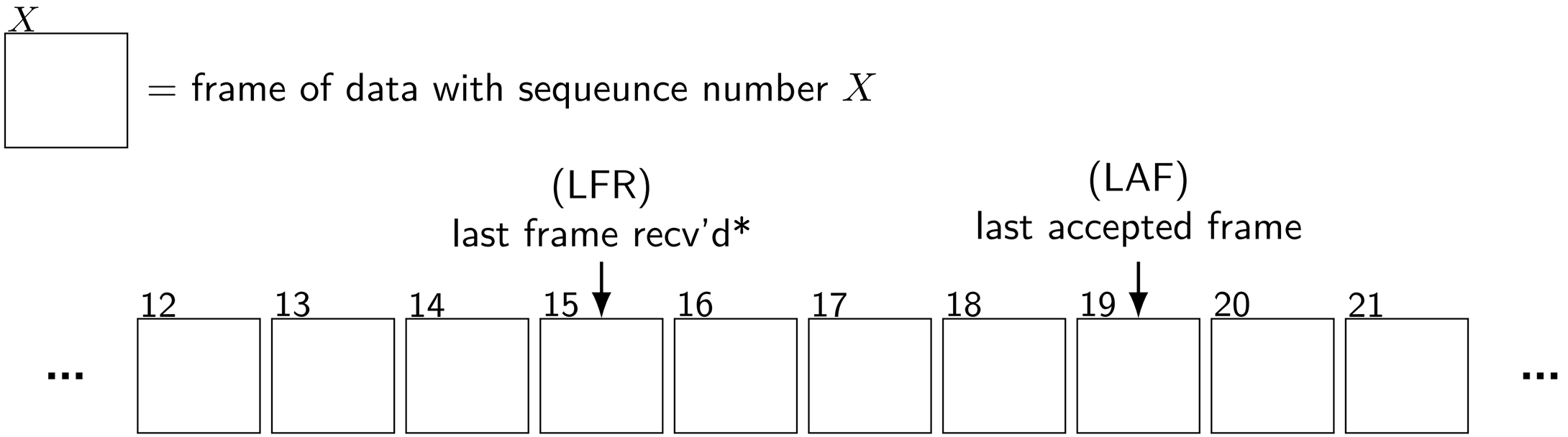
when receiving ACK $LAR < X \leq LFS$:

- LAR $\leftarrow X$
- clear any timers to resend frames $\leq X$

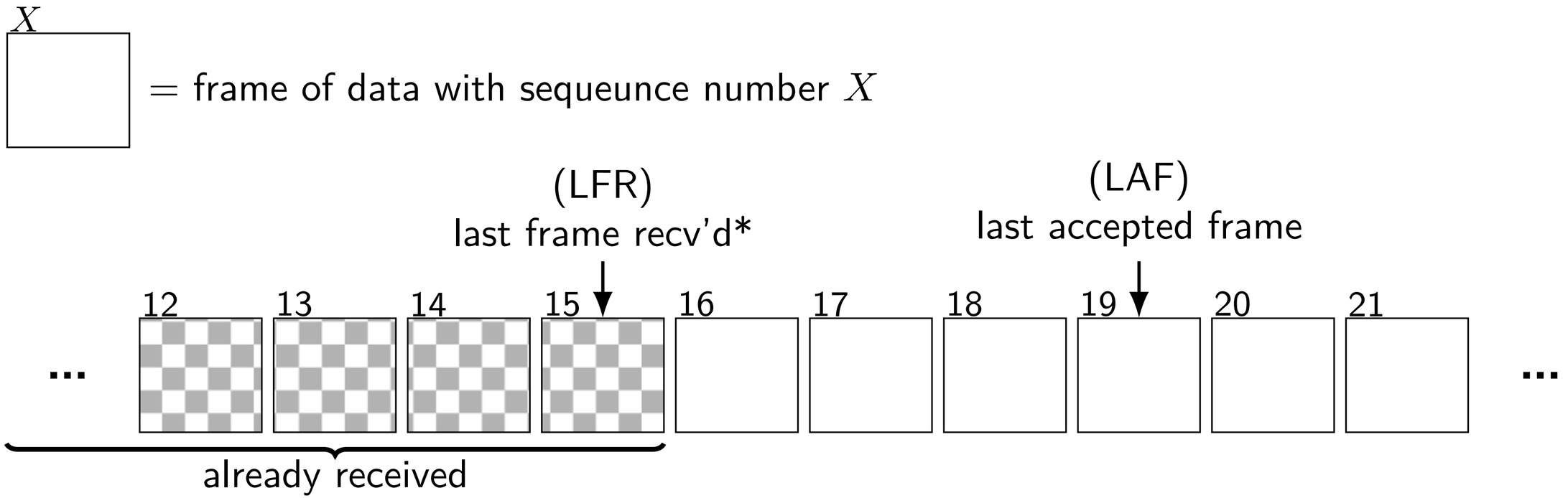
whenever SWS [send window size] > LFS - LAR *and*
data for frame LFS + 1 is available:

- send frame LFS + 1
- set timer to resend frame LFS + 1
- LFS $\leftarrow$ LFS + 1

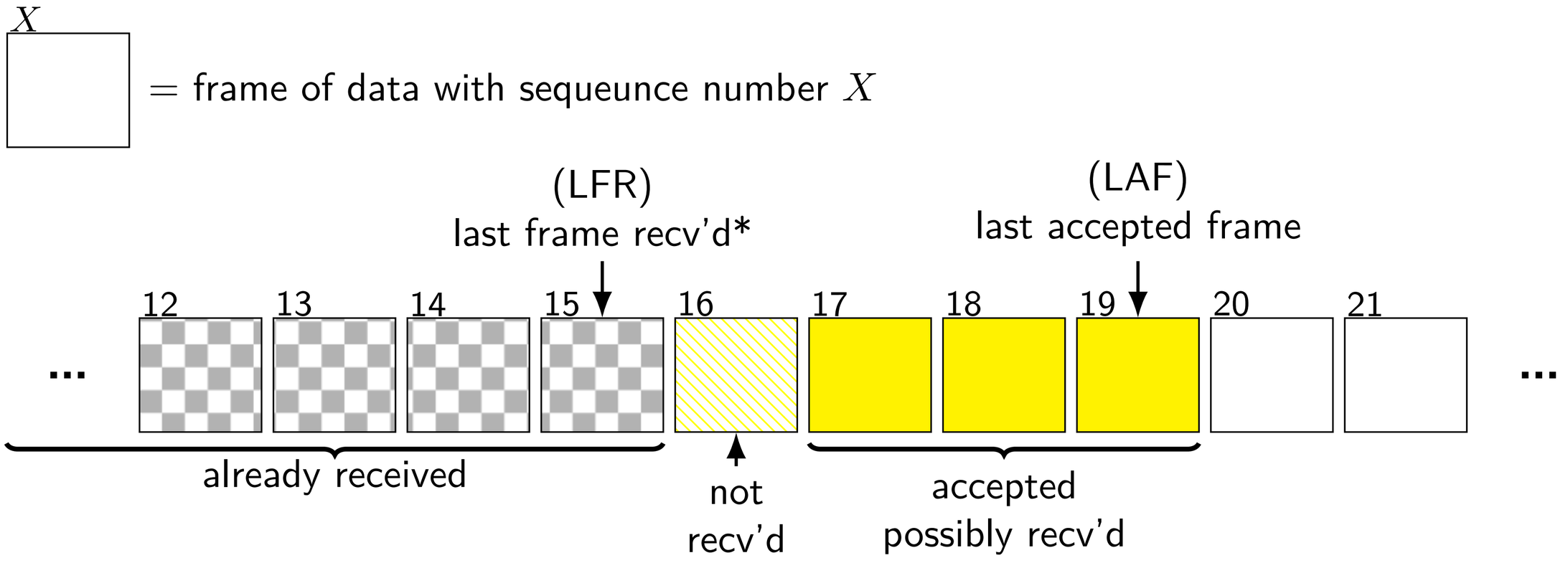
receiver window tracking



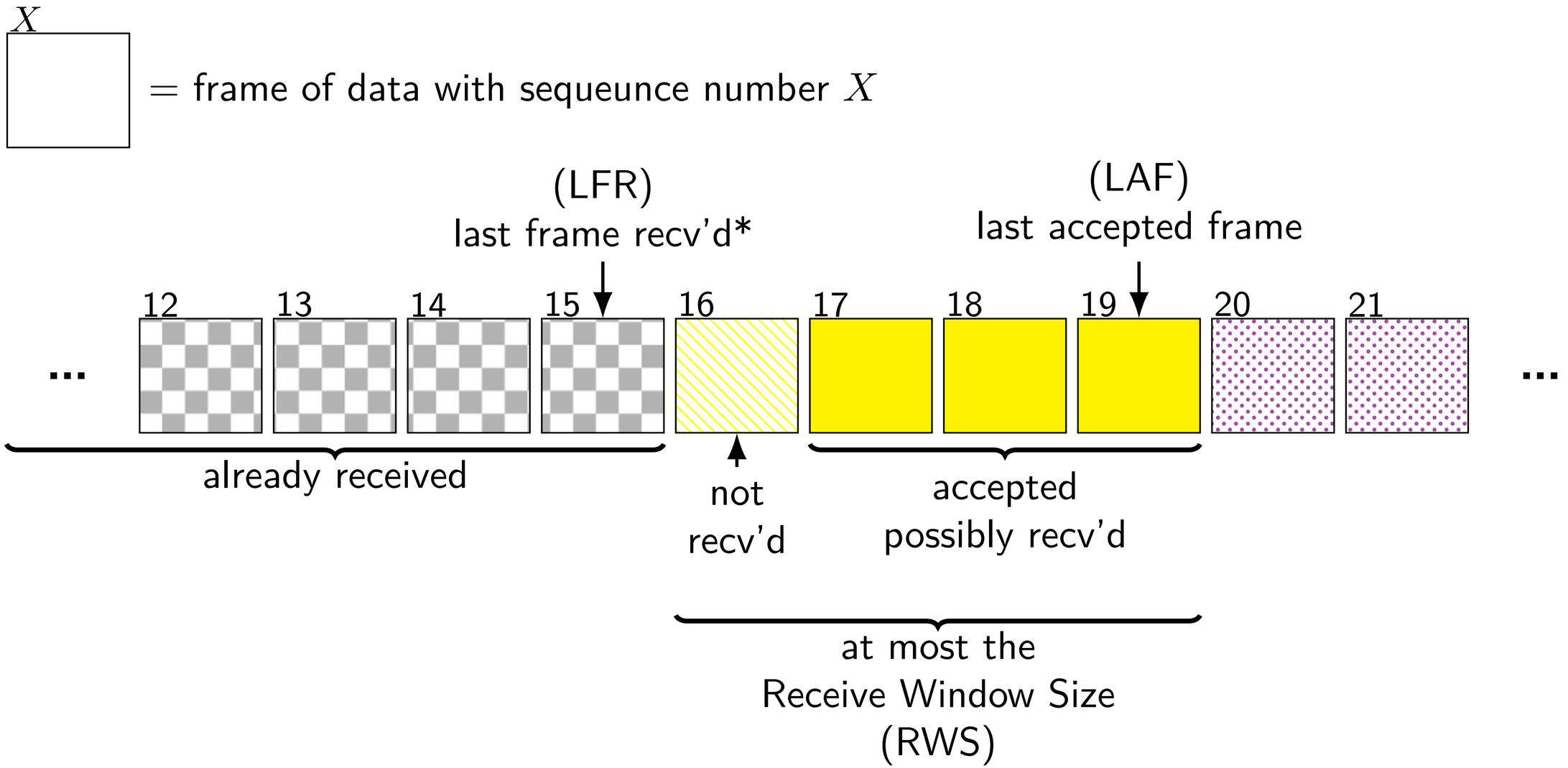
receiver window tracking



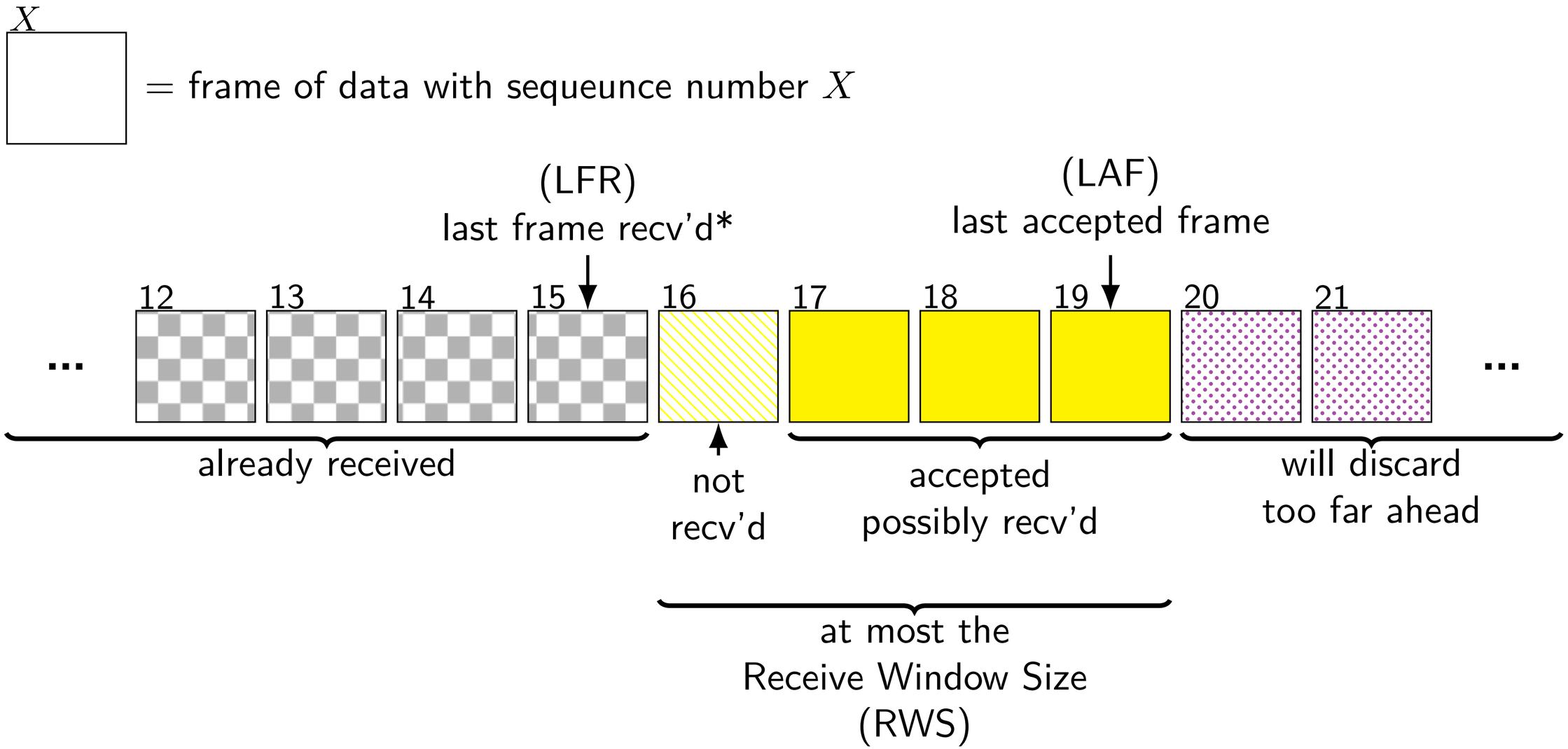
receiver window tracking



receiver window tracking



receiver window tracking



receiver logic summarized

track variables:

- LFR (last frame recv'd) — excludes frames after a missing frame
- LAF (last accepted frame)
- RWS (receive window size)

when receiving frame $LFR < X \leq LAF$:

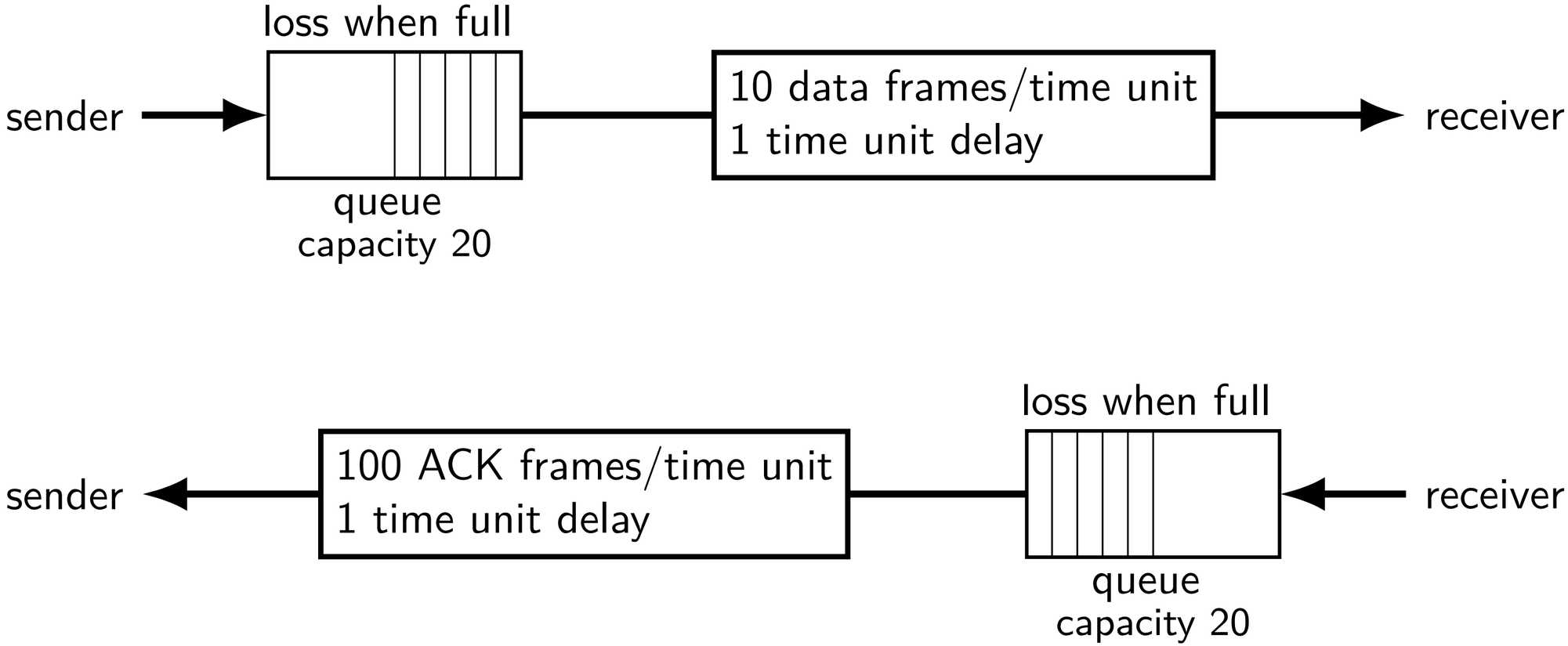
LFR $\leftarrow$ (first missing frame after LFR) - 1

- only advances if X $=$ LFR
- could advance by more than one if frames previously out of order

LAS $\leftarrow$ LFR + RWS

- only advances if X $=$ LFR

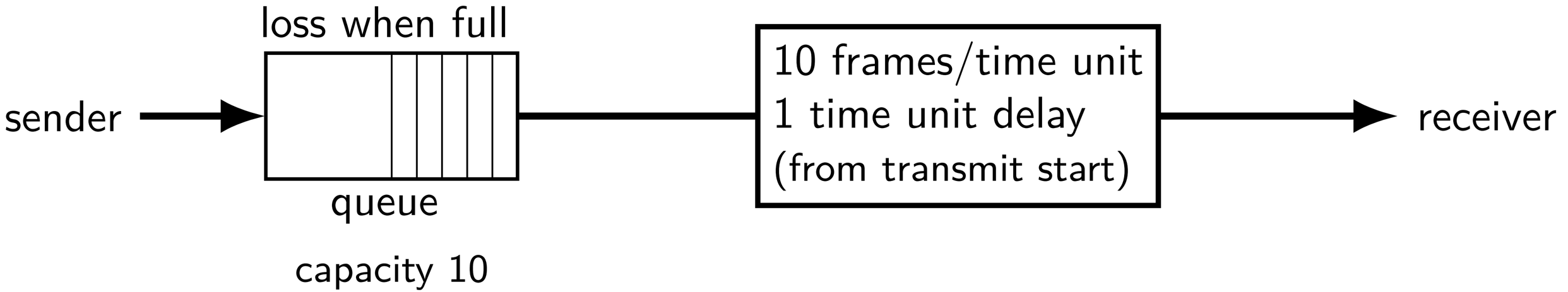
simple network model



simulator from upcoming assignment

command line `-delay 1 -bandwidth-forward 10 -bandwidth-backward 100 -buffer 30`

exercise: forward latency

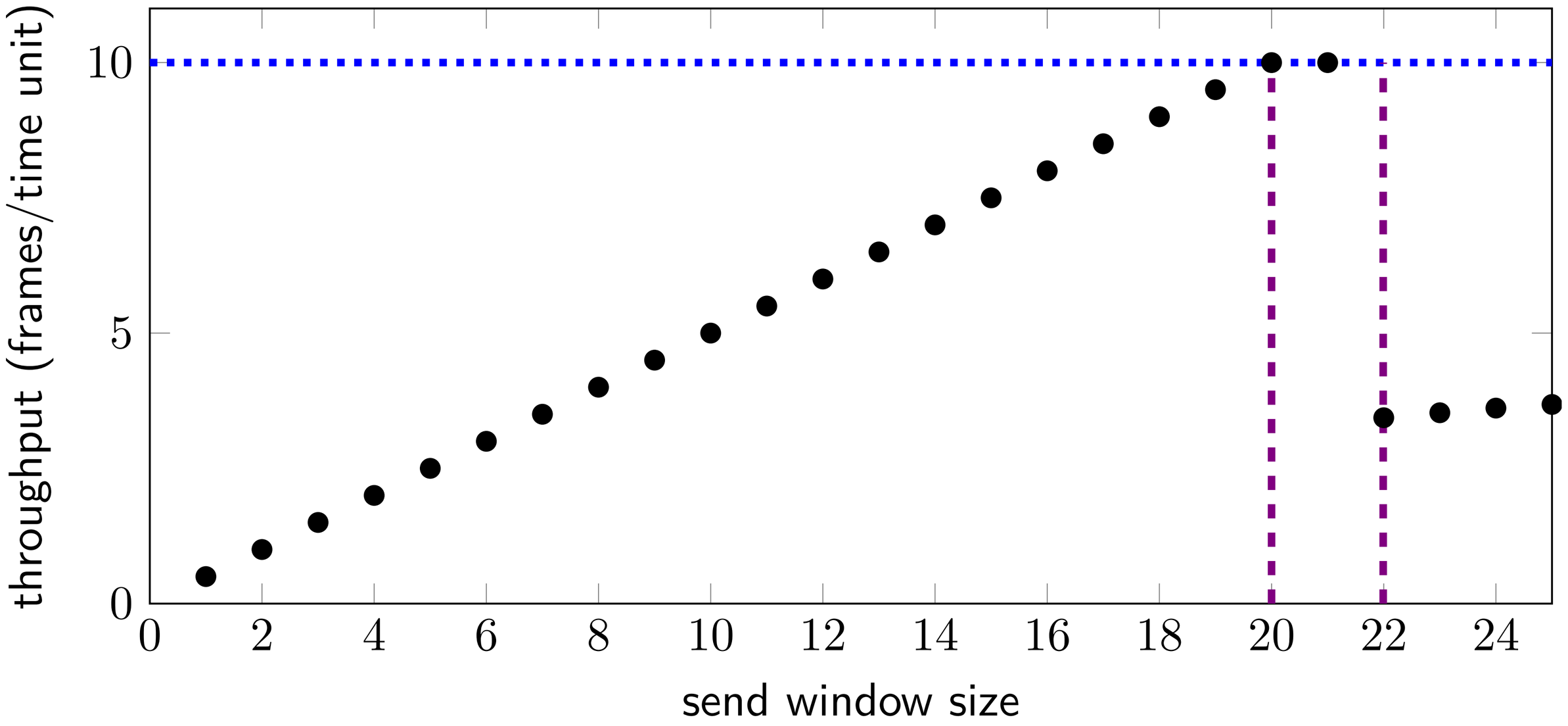


minimum latency = 1 time unit

exercise: maximum latency?

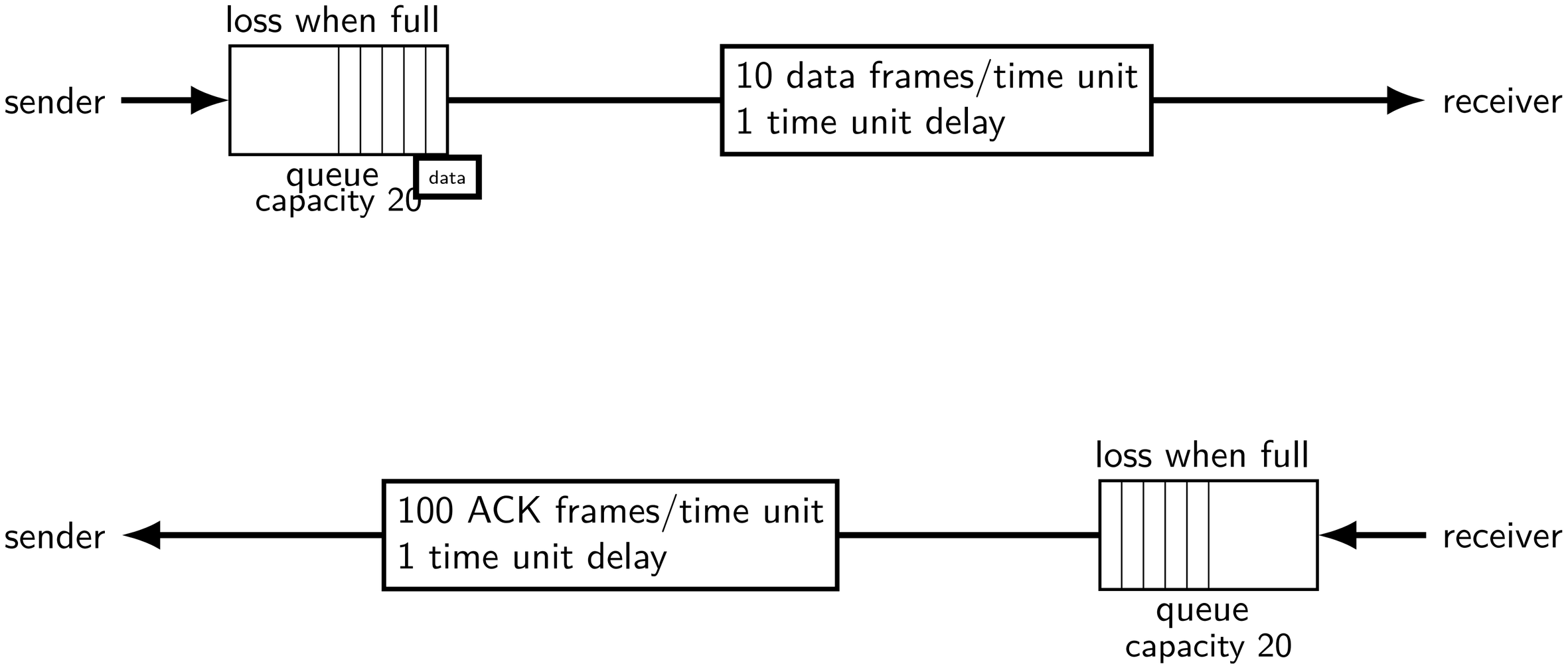
A. 1 time unit
B. 1.1 time unit
C. 1.2 time unit
C. 1.4 time unit
D. 1.9 time unit
E. 2.0 time unit
F. 2.1 time unit
G. something else

throughput and window size

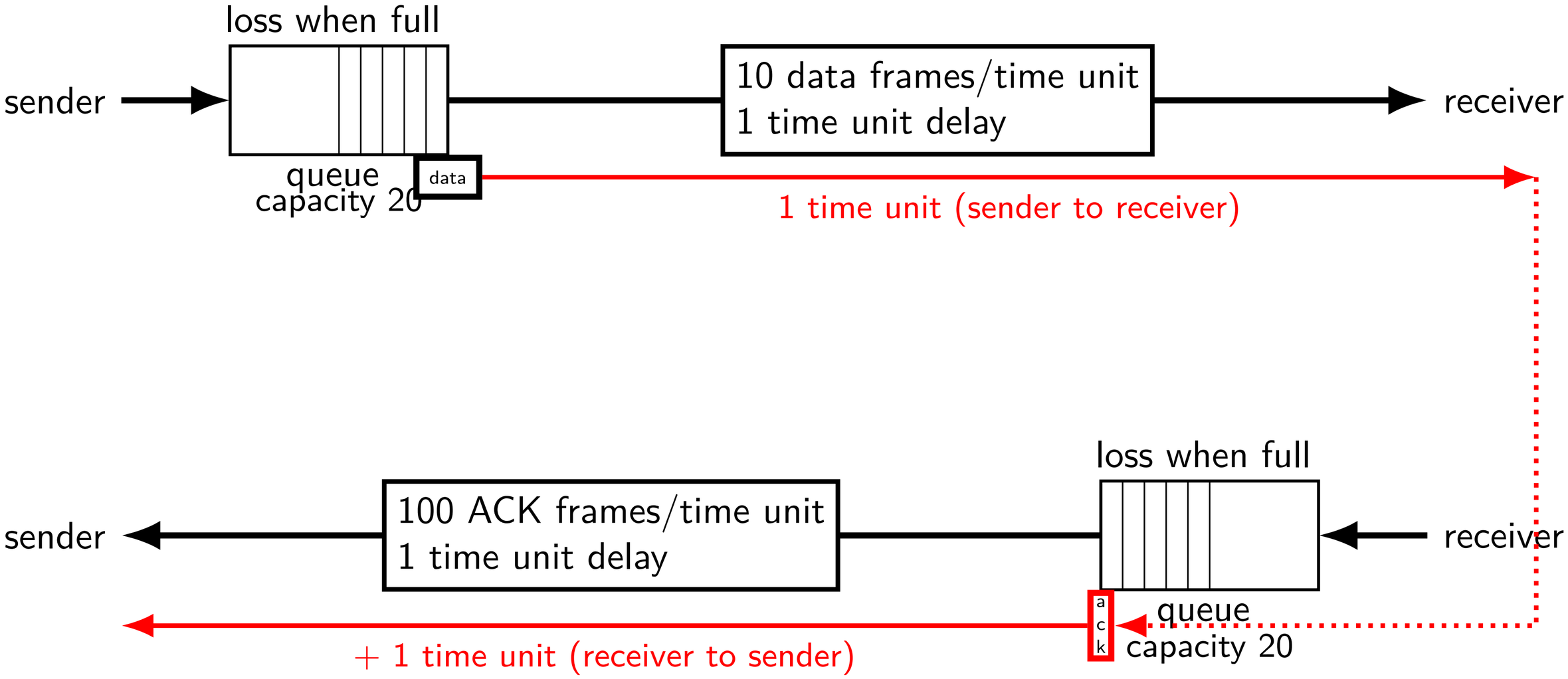


packet transit time

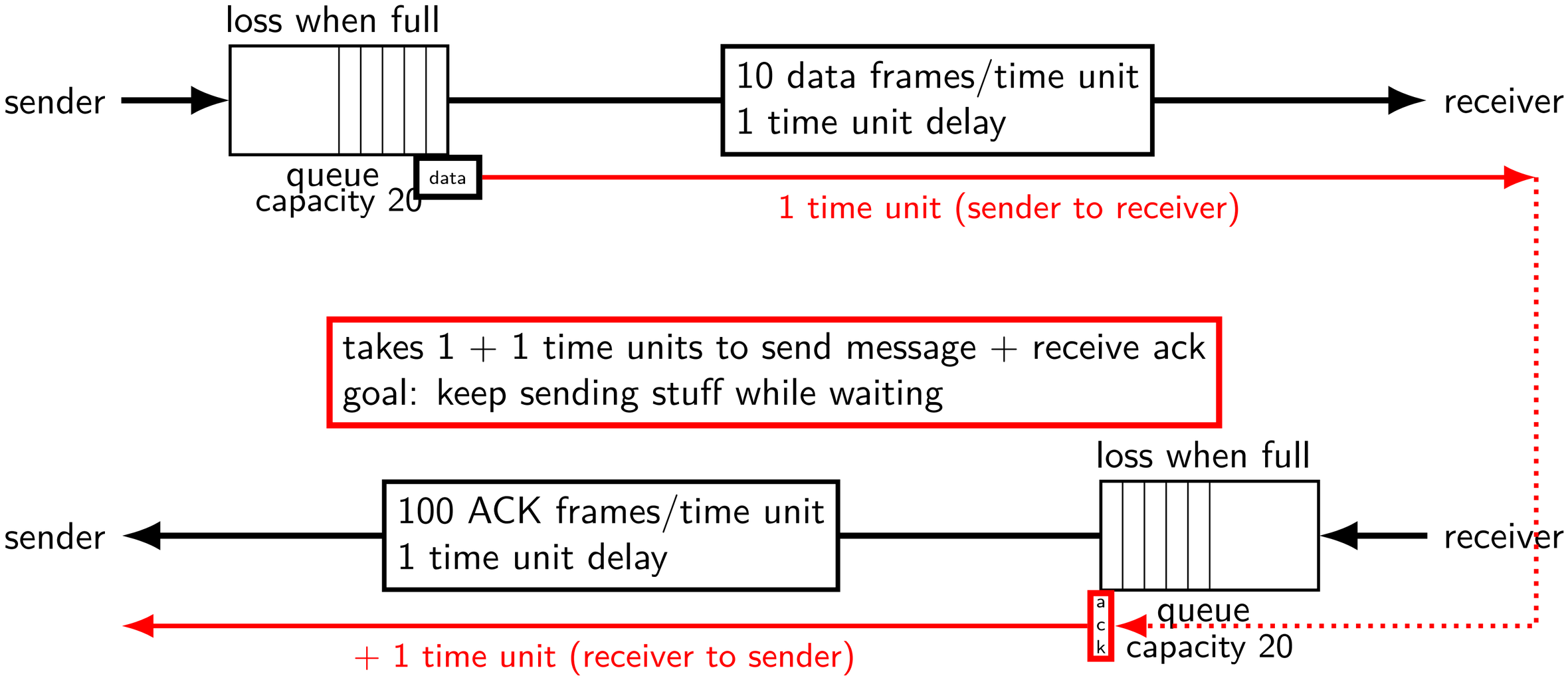
packet transit time



packet transit time



packet transit time



filling the pipe

round-trip time of 2 time units

from send data to receive ACK (assuming no queuing delay)

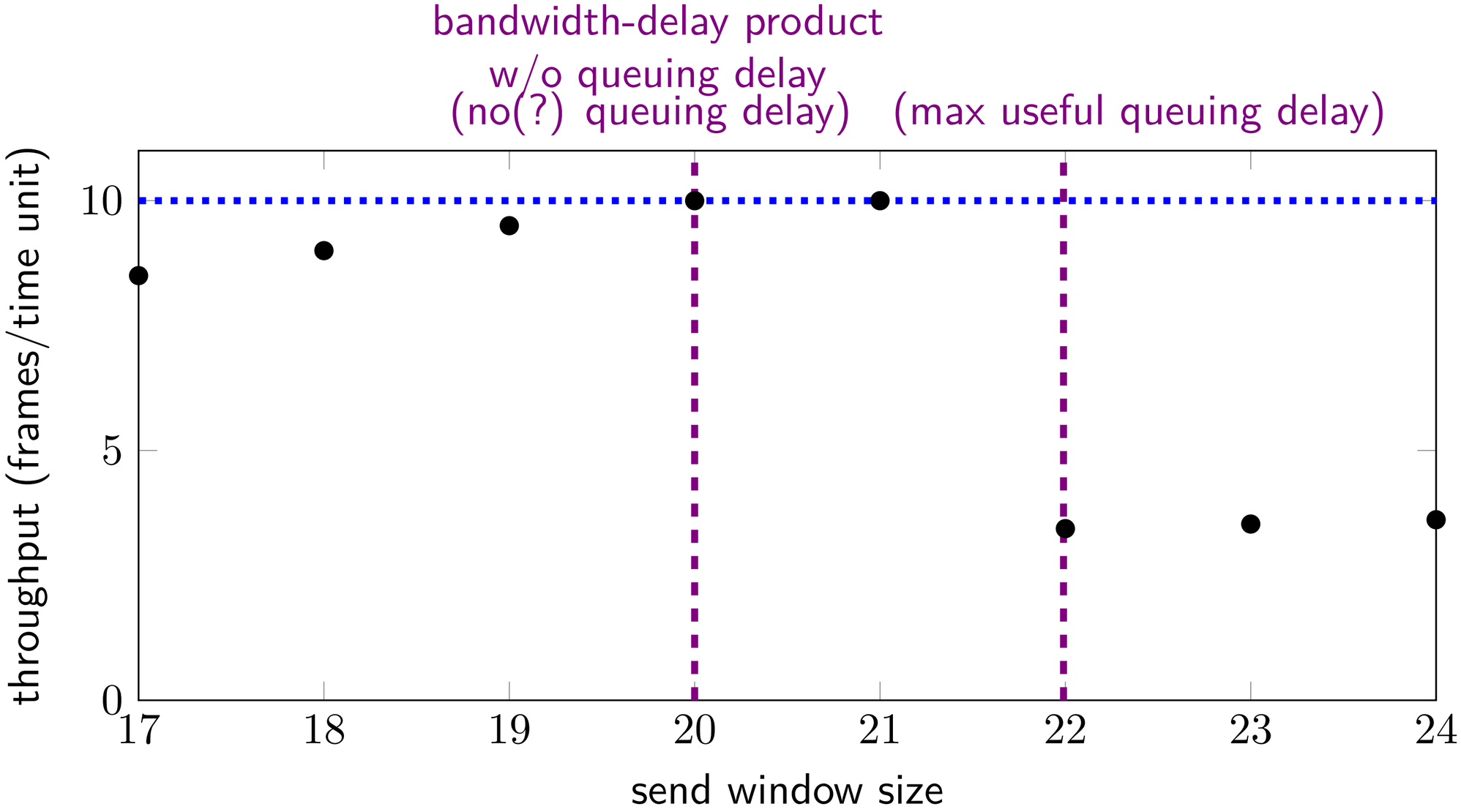
can send 10 data frames per time unit

= can send 20 data frames while waiting for ACK

“bandwidth-delay product”

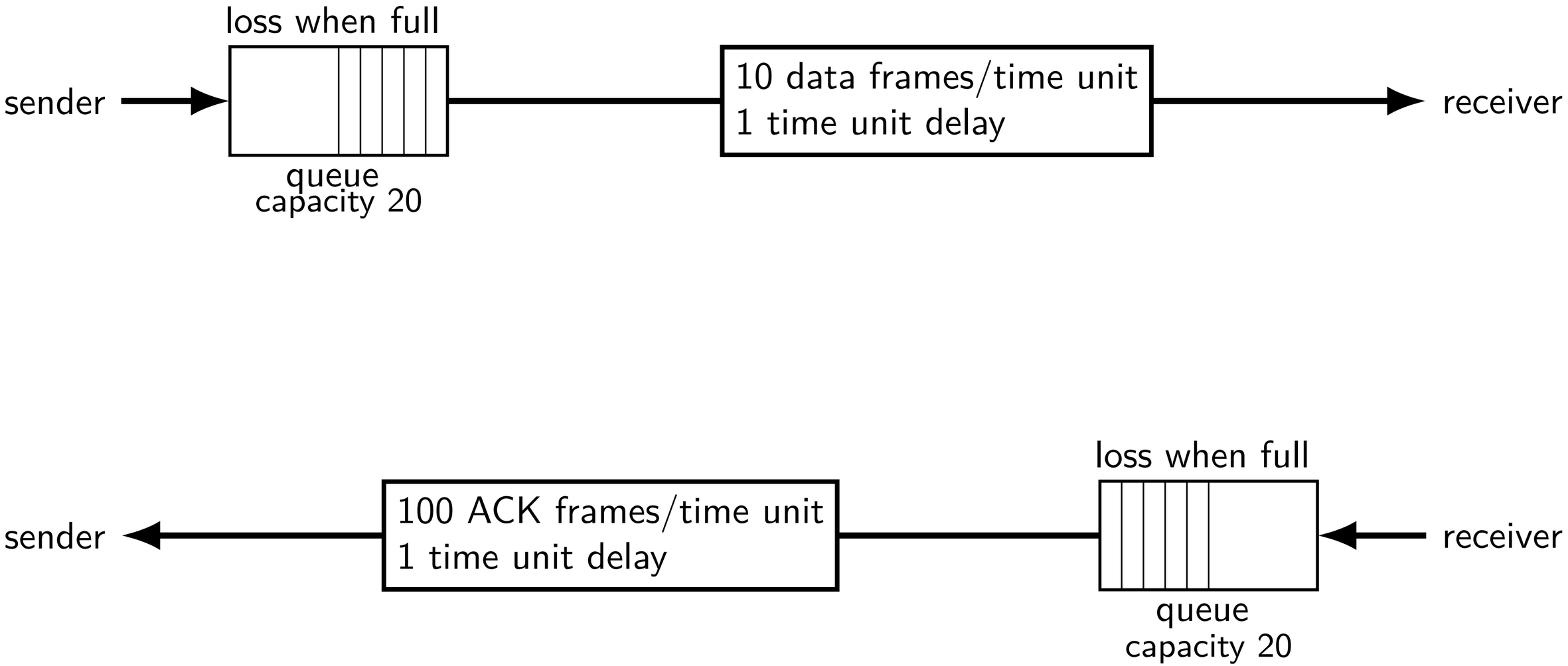
10/time unit (banwidth) times 2 time unit (RTT = delay)

throughput and window size (detail)

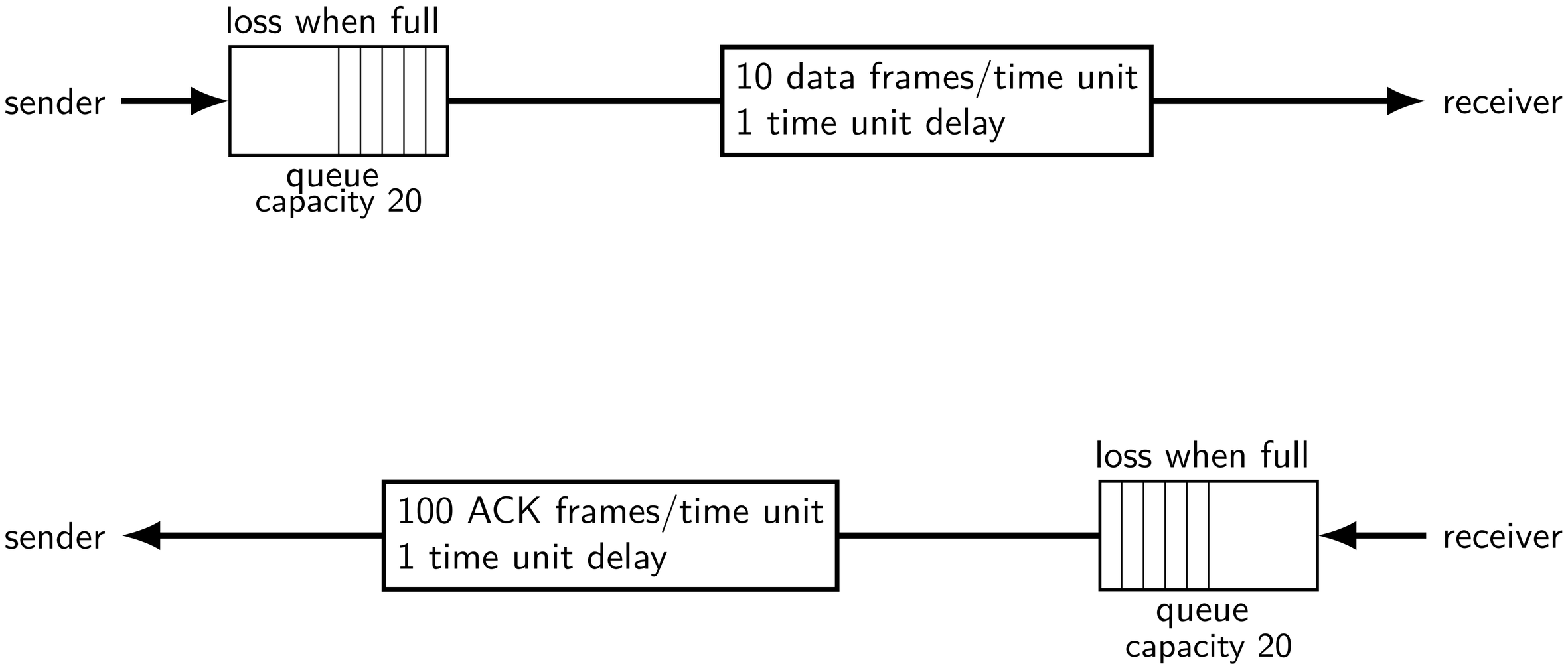


filling the pipe

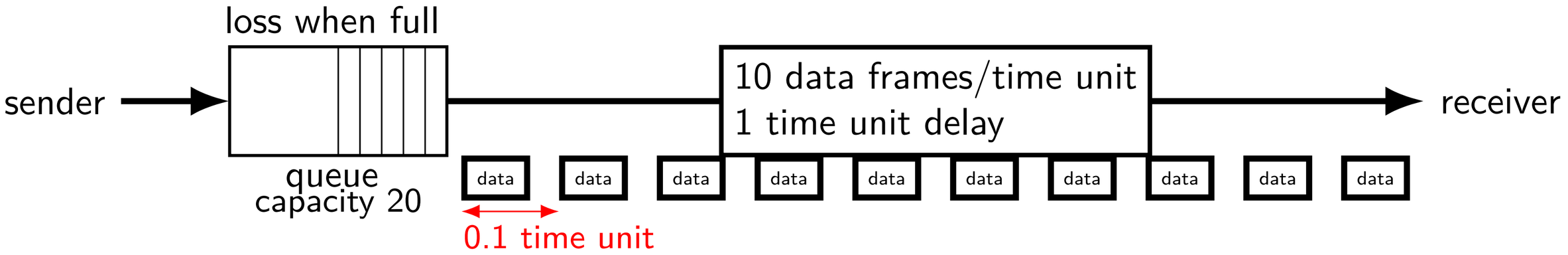
filling the pipe

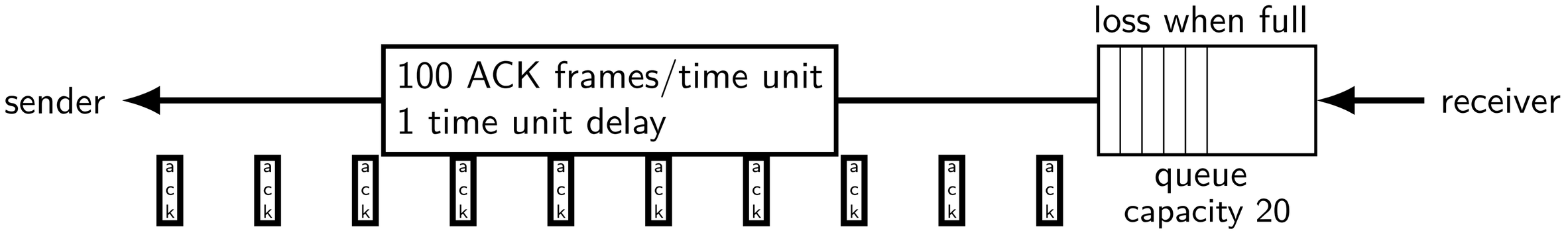


filling the pipe



filling the pipe





on bursts

max possible queuing delay suggests window size of 30

- approx. 3 time units times 10

problem: “bursts” temporarily exceed queue size

achievable average queue size not that high

sender could moderate by “pacing” packets

sliding windows used to solve

flow control

keep sender from getting too far ahead of receiver

… by having *window sizes set correctly*

how? receiver tells sender what window size is okay

congestion control

keep network from being overloaded

(while making good use of available bandwidth)

… by having *window sizes set correctly*

how? it's complicated — big topic later

sequence number wraparound

protocol so far requires arbitrarily large sequence numbers

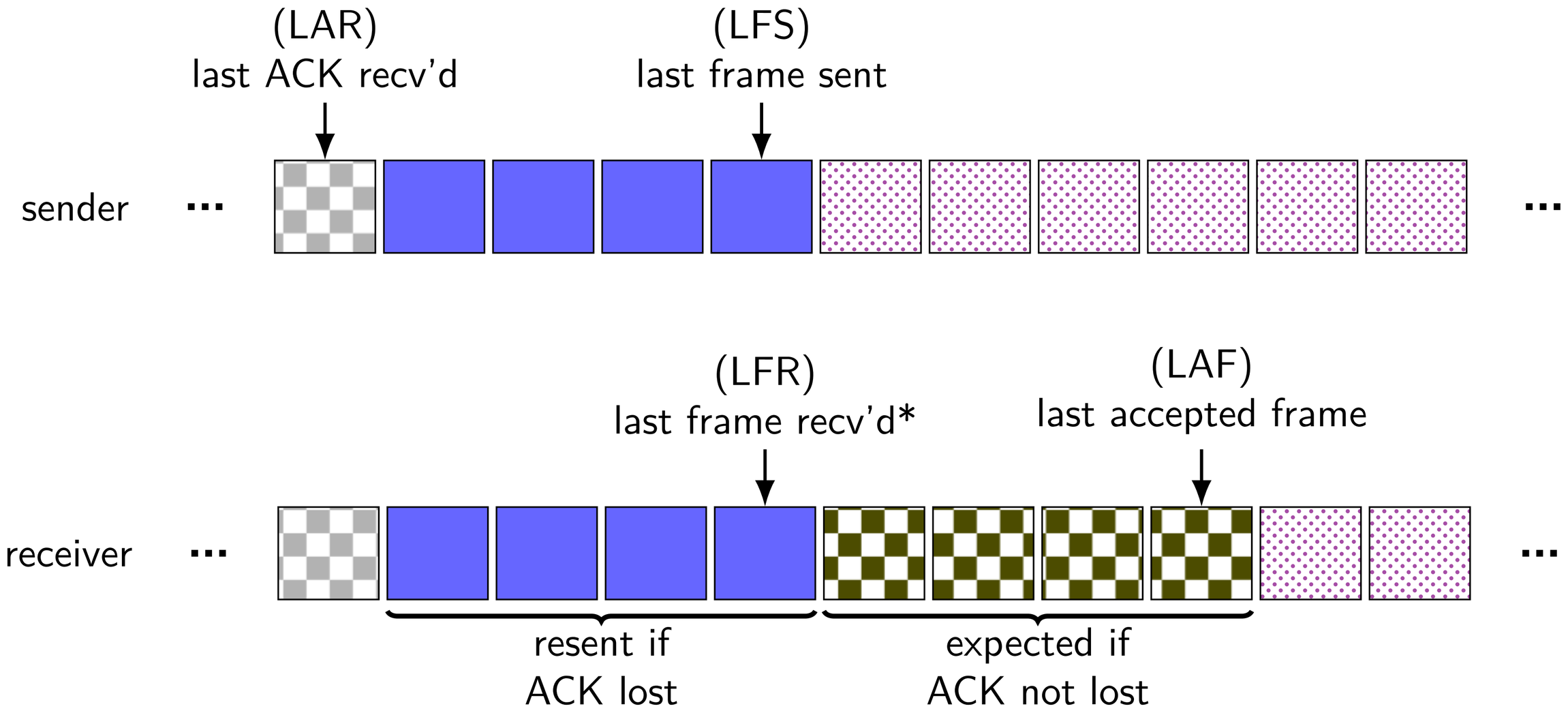
- doing $<$ and $>$ checks on sequence number, so they need to increase

would like to use smaller sequence numbers

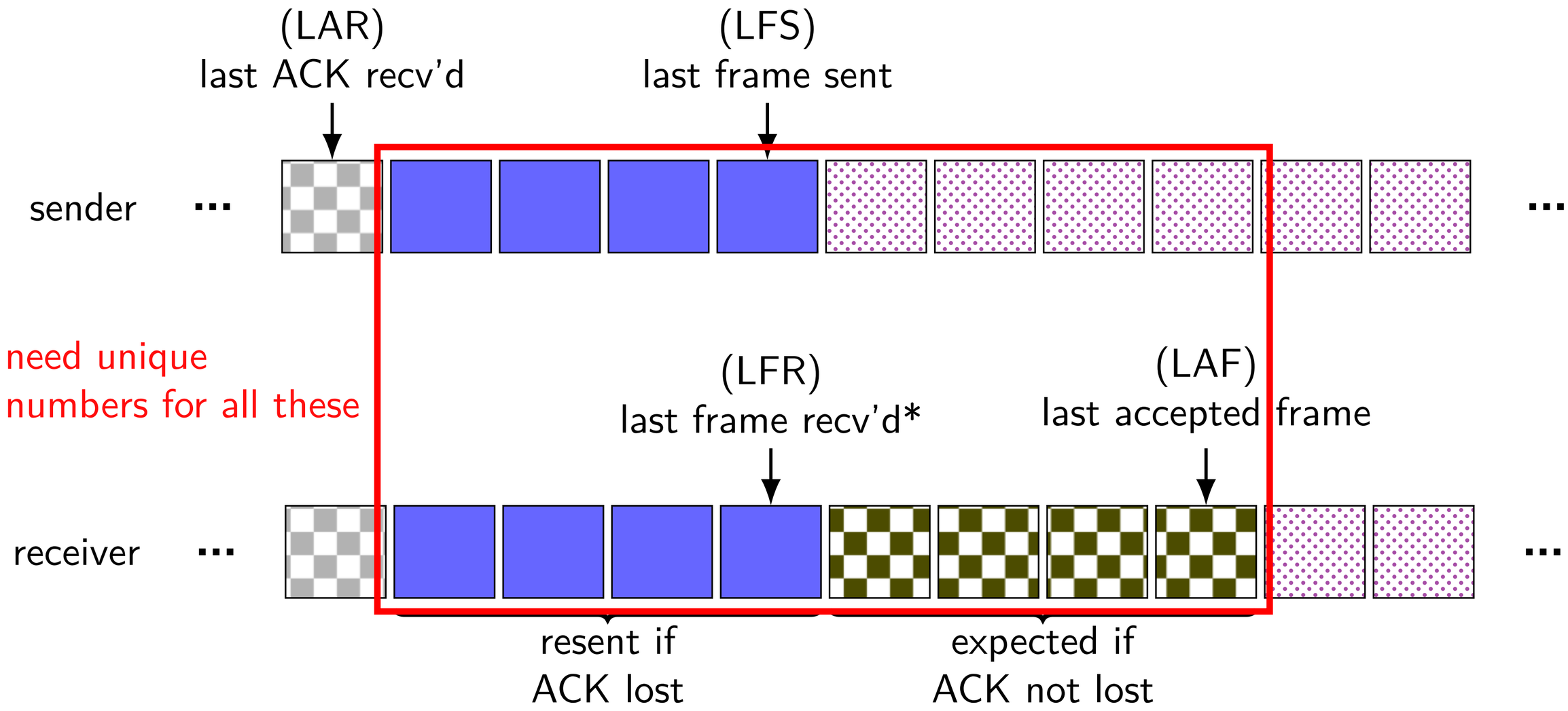
- think: transferring multi-gigabyte file

question: what goes wrong when we reuse sequeunce numbers?

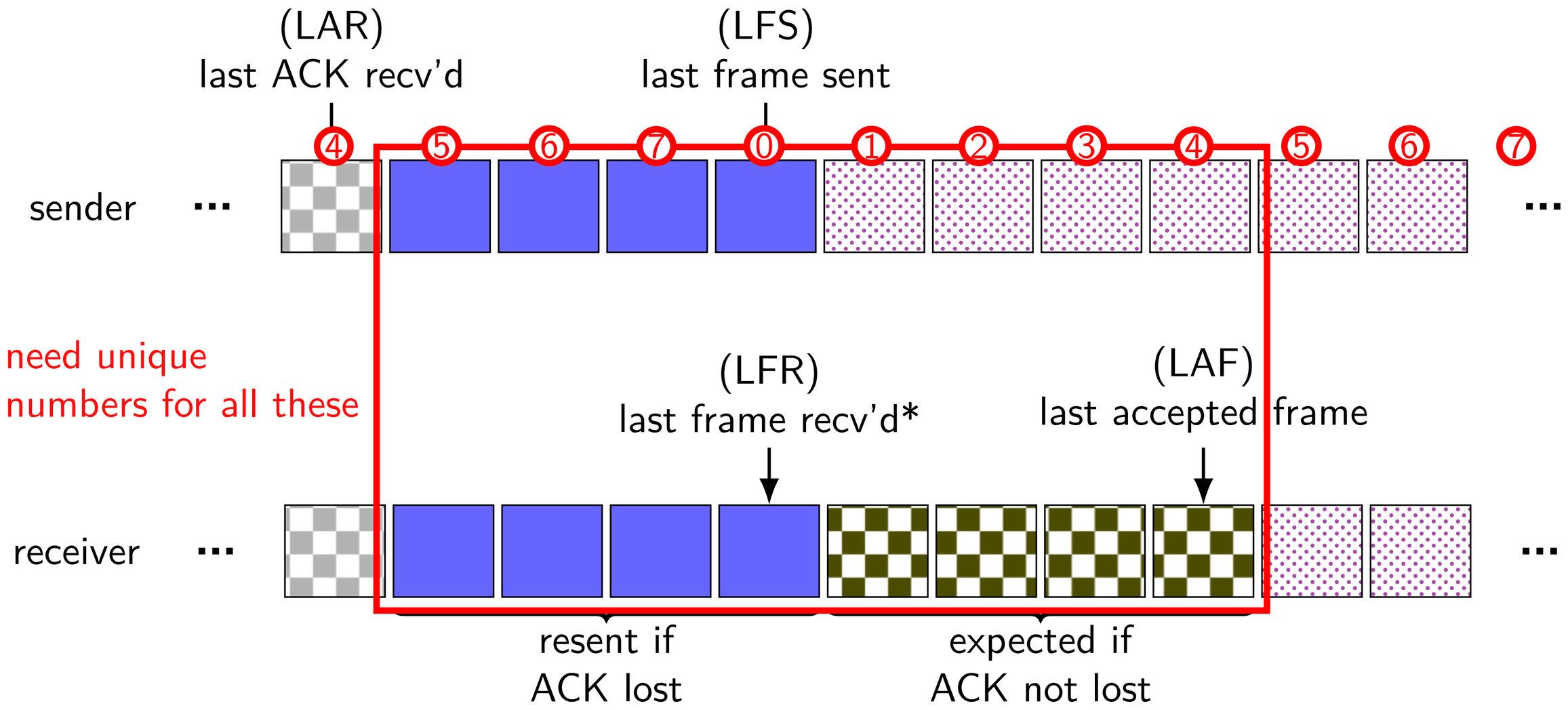
sender/receiver desync: missing ACKs



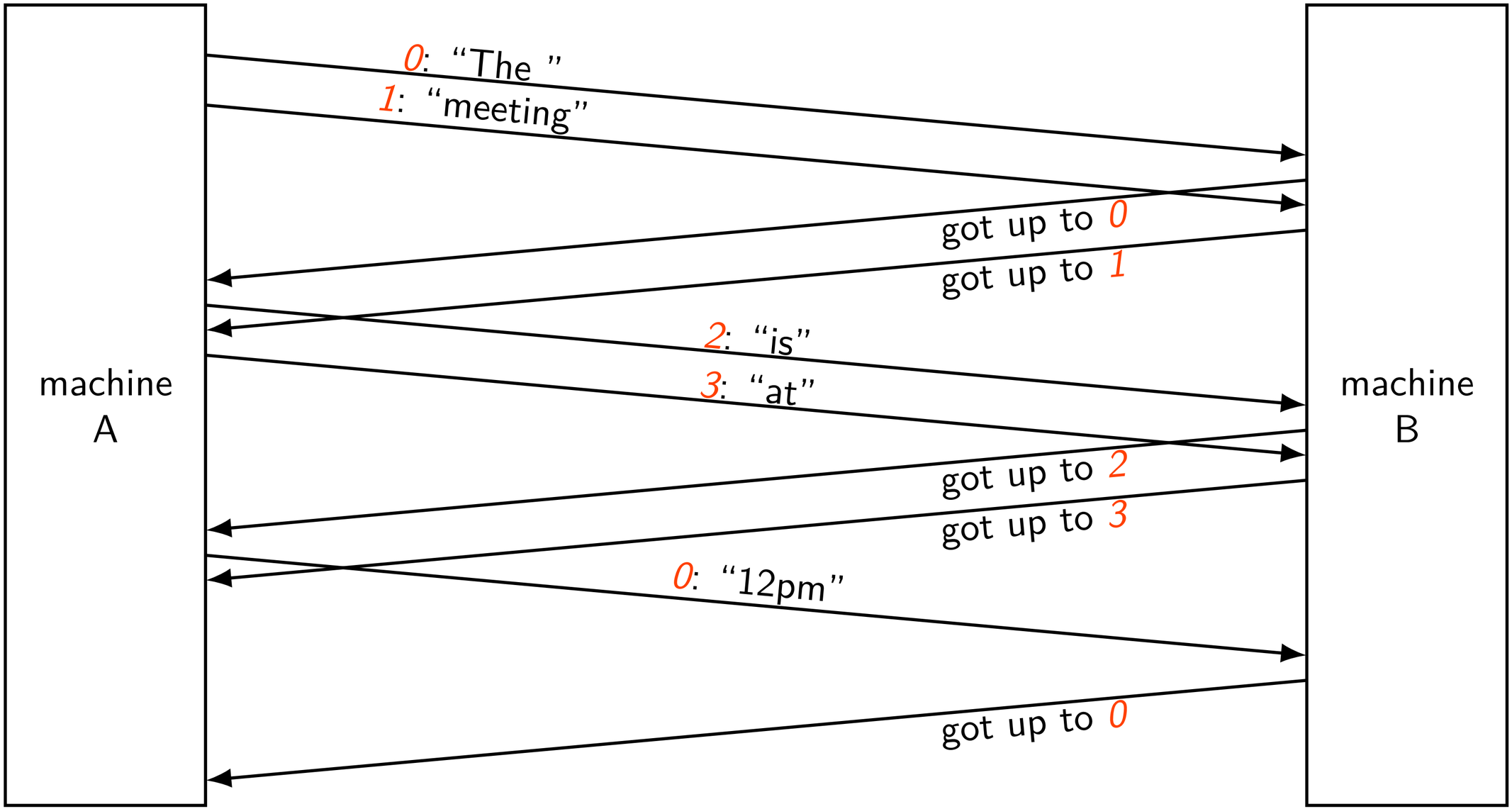
sender/receiver desync: missing ACKs



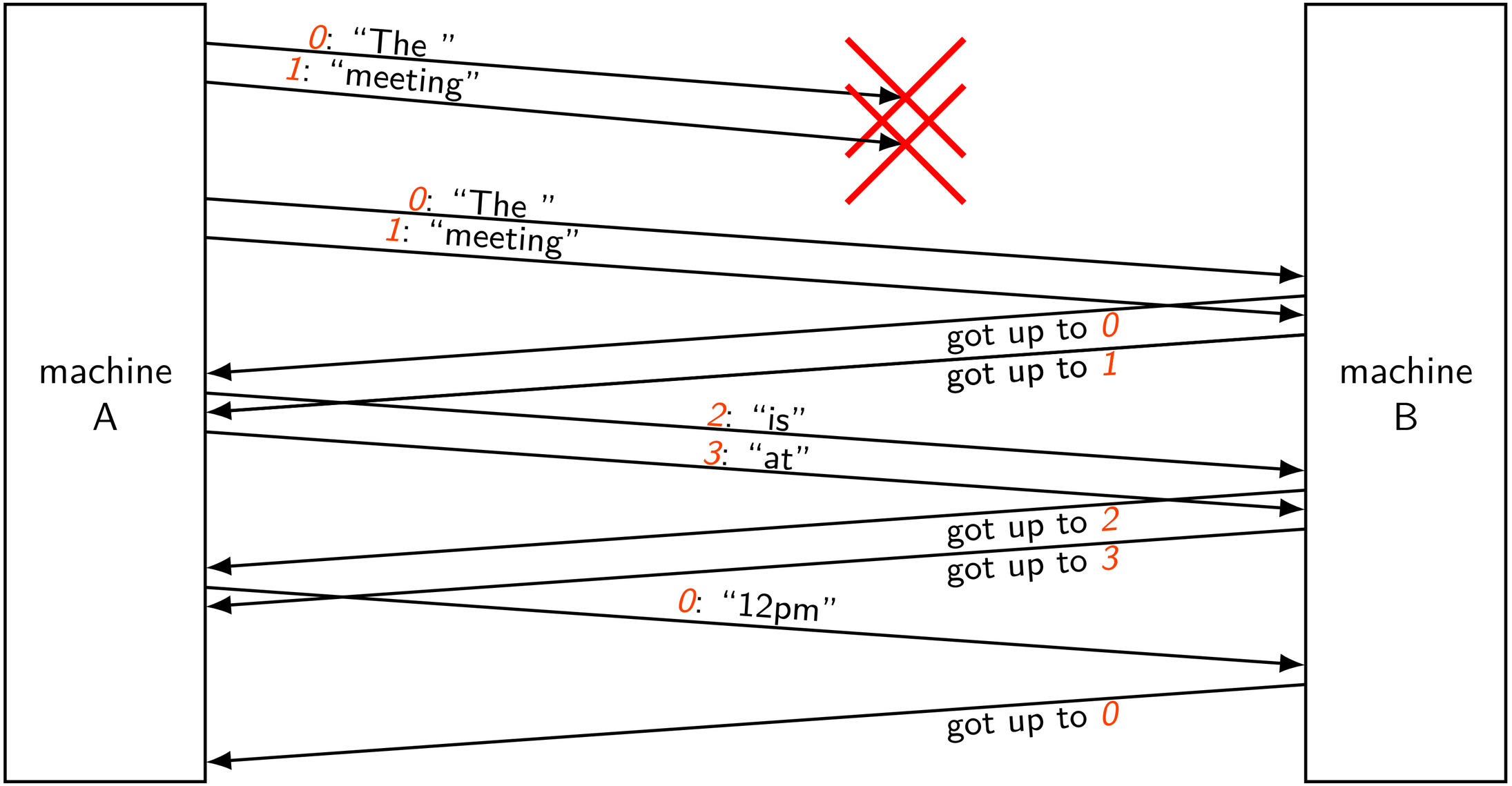
sender/receiver desync: missing ACKs



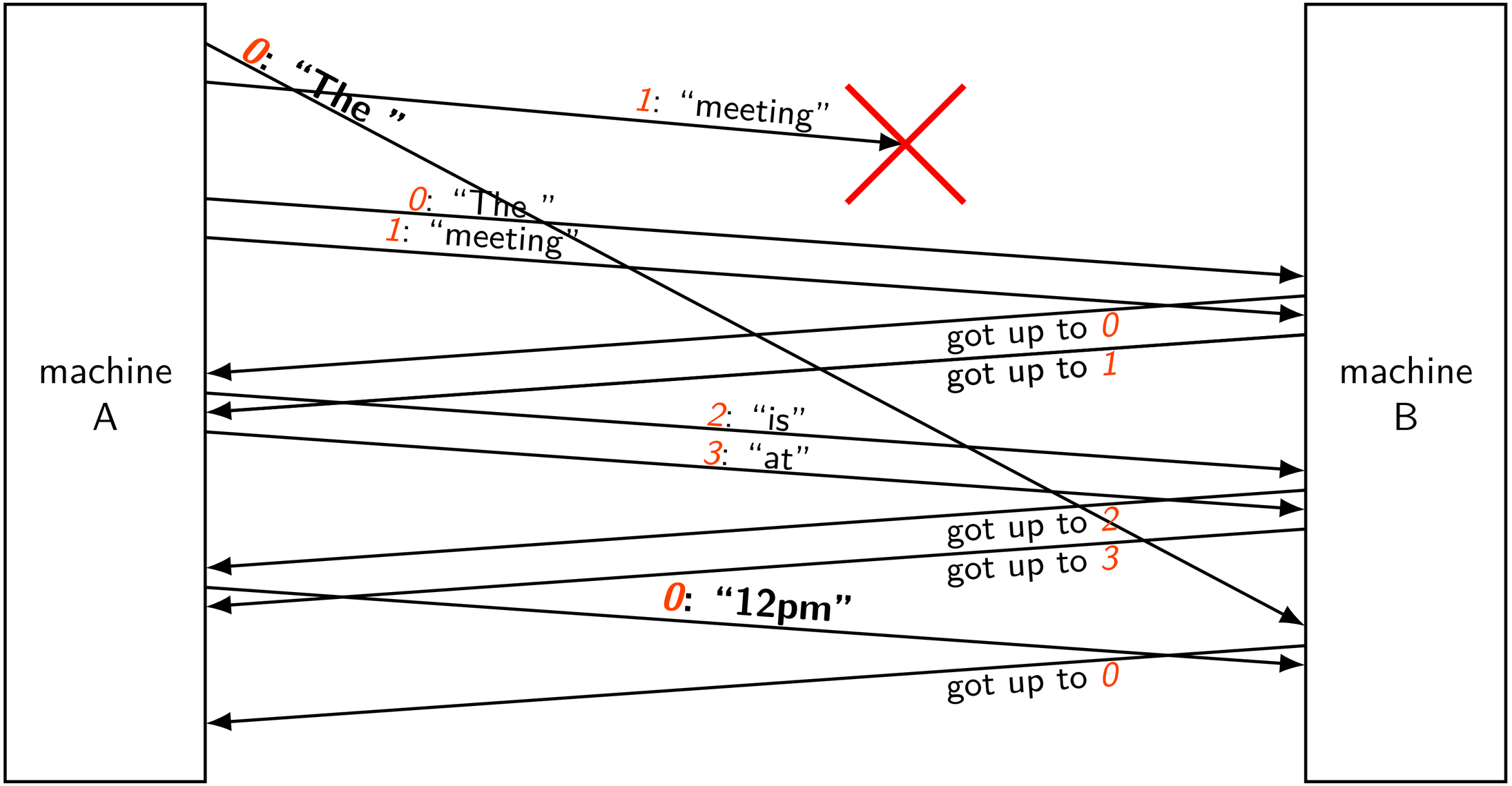
wraparound


machine A
machine B
0: “The ”
1: “meeting”
got up to 0
got up to 1
2: “is”
3: “at”
got up to 2
got up to 3
0: “12pm”
got up to 0

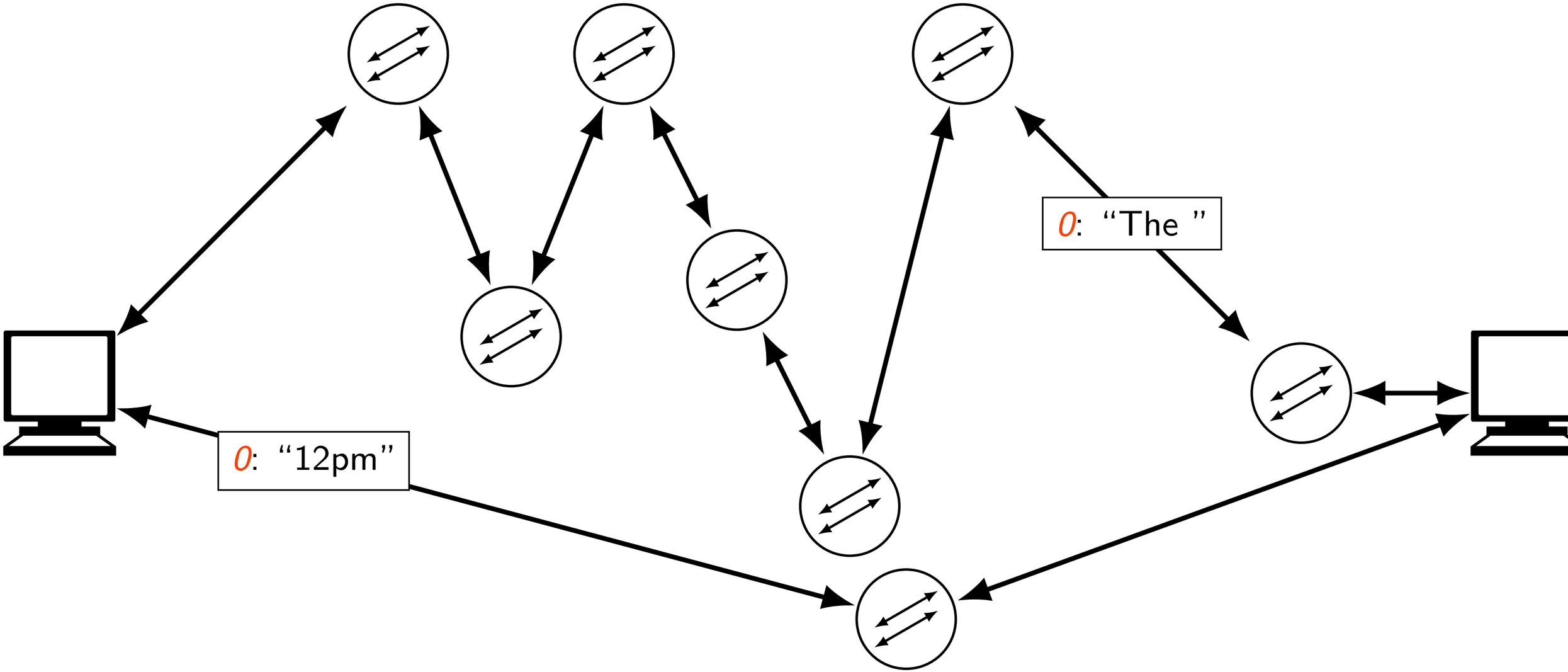
loss and resend?



very bad reordering



possible reason



sequence numbers in practice

TCP tries to assume 120 second "maximum segment lifetime"

- segment = TCP's name for a packet

original TCP used 32-bit sequence number identifying *byte* number (not segment number)

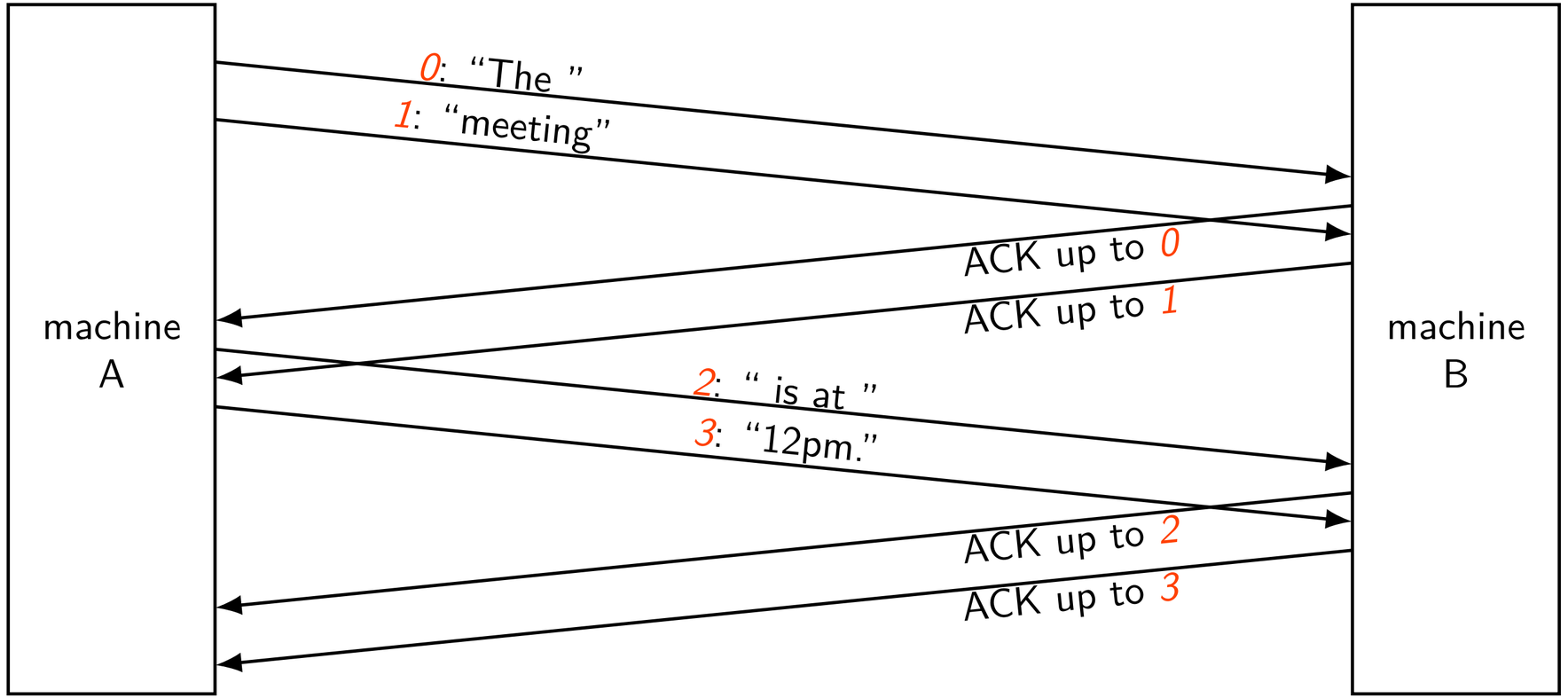
problem: means wraparound happens on modern (Gigibit+) links in seconds!

workaround: add *additional* 32-bit timestamp field

- used to detect/discard duplicates
- can also be used to set timeouts and/or window sizes

backup slides

sending two at a time



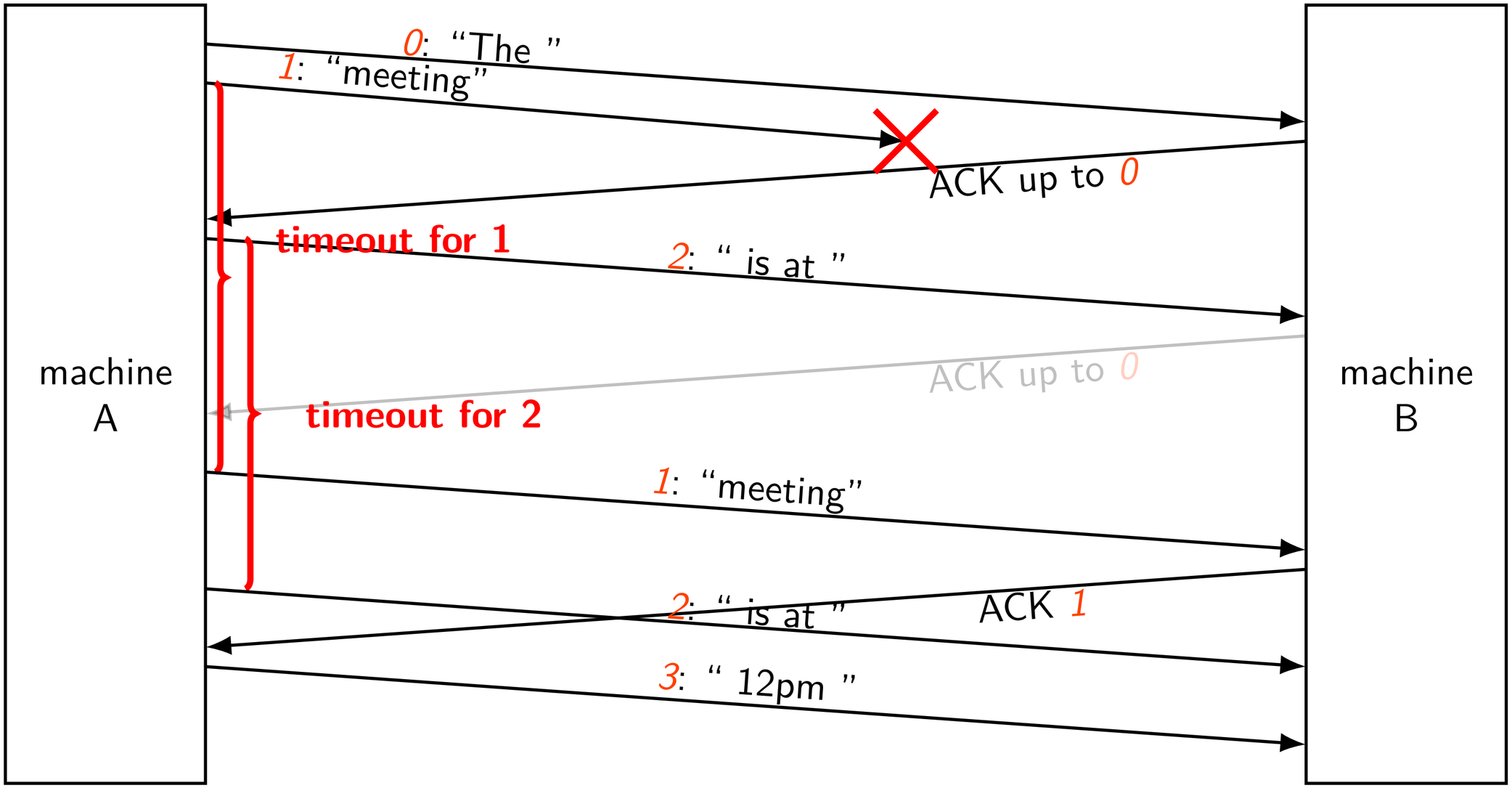
(ACK up to X = ACK X and everything before it)

key idea: always have two in flight

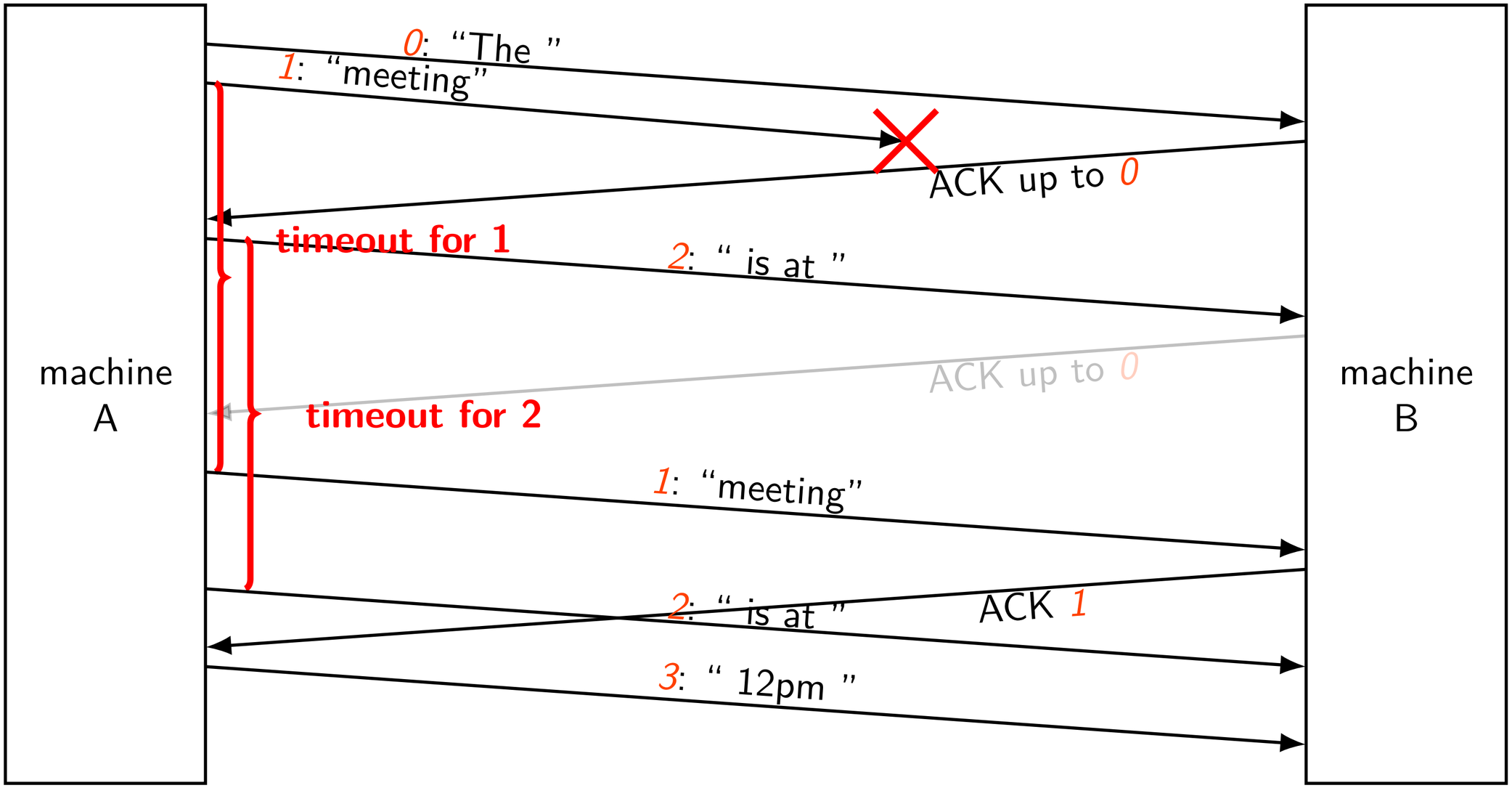
send next when previous ack'd

timeouts per message

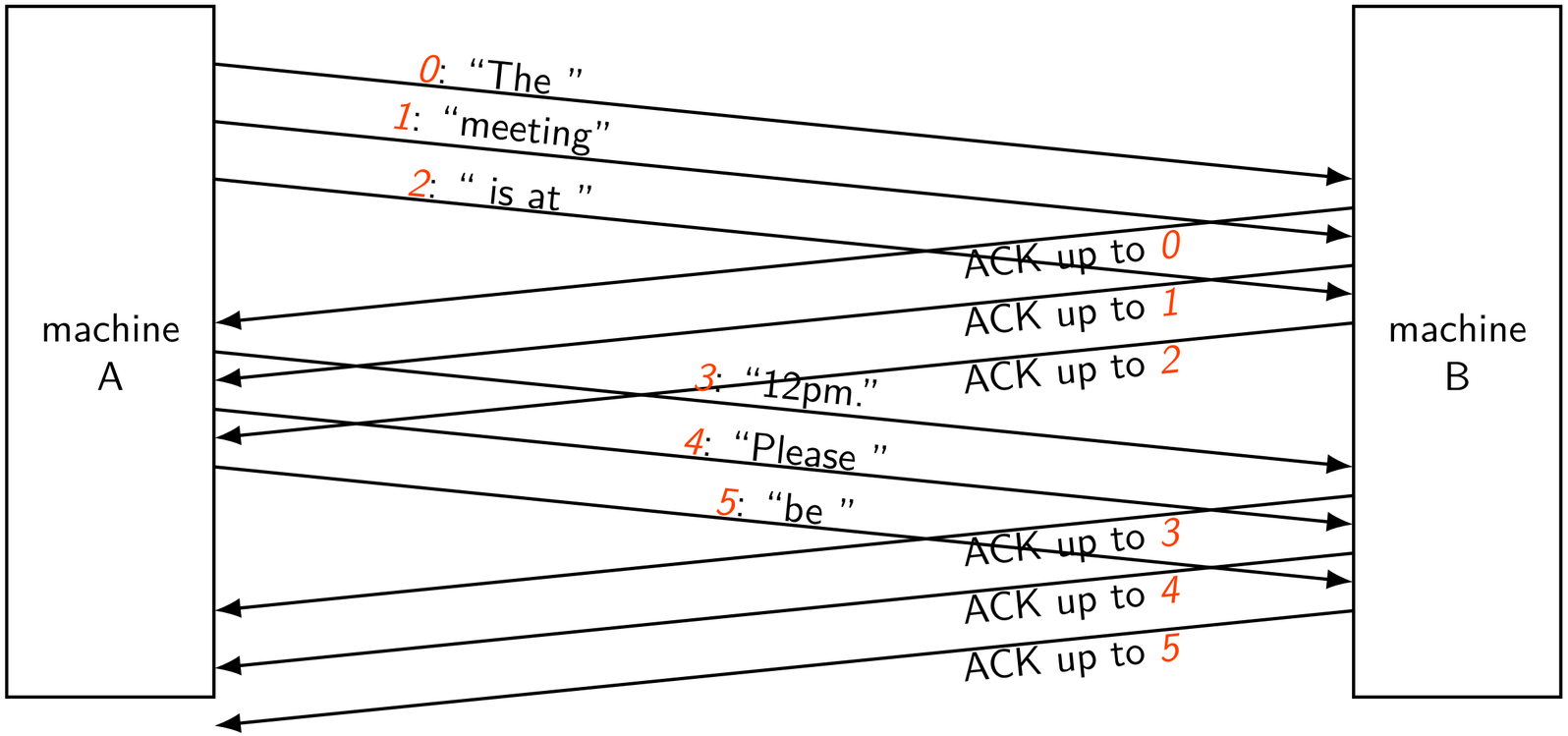
timeouts per message



timeouts per message



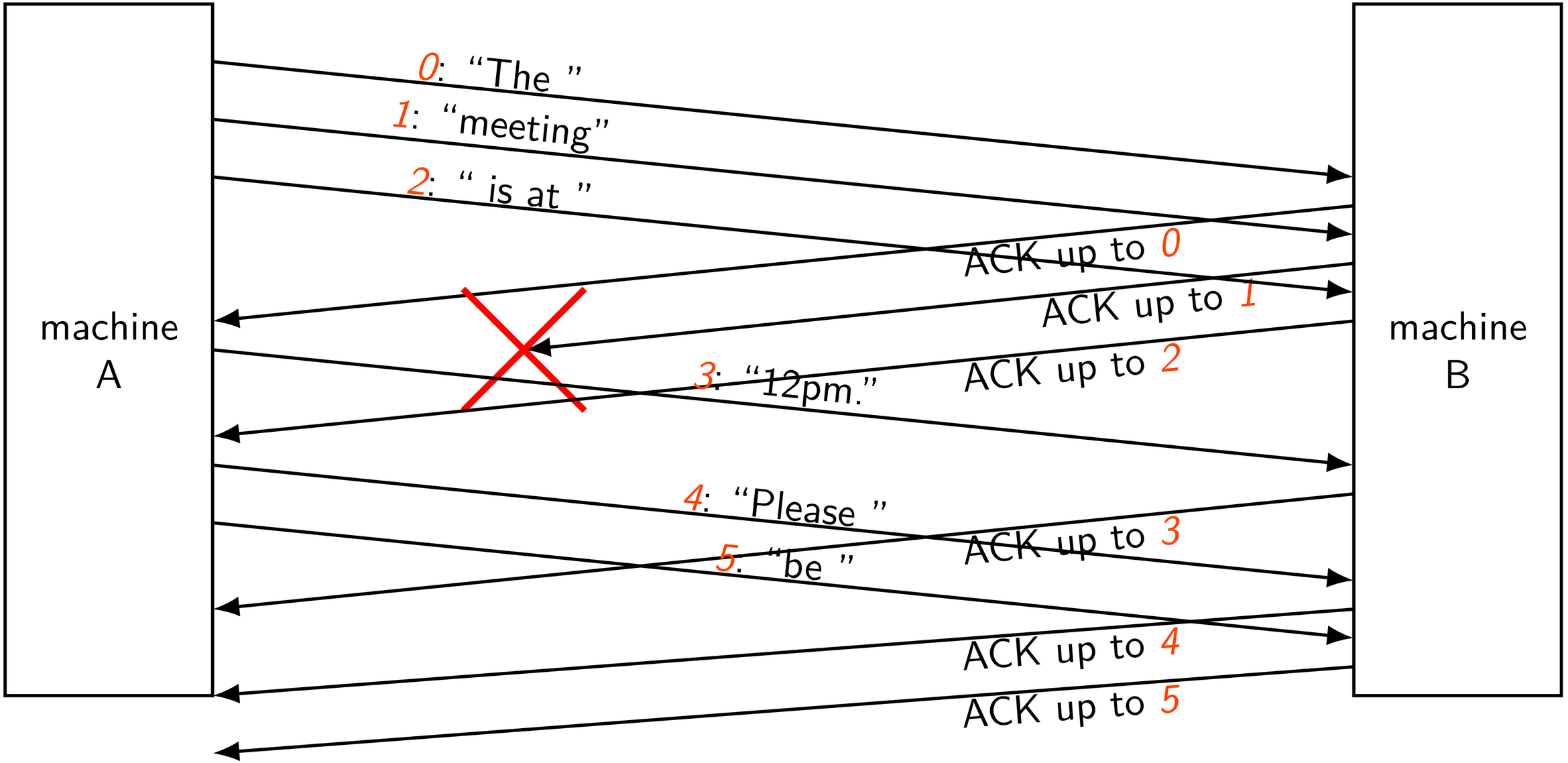
sending three at a time



choose "window size" to have in flight

send when previous acknowledged

lost ACKs?


machine A
machine B
0: "The "
1: "meeting"
2: " is at "
ACK up to 0
ACK up to 1
3: "12pm."
ACK up to 2
4: "Please "
5: "be "
ACK up to 3
ACK up to 4
ACK up to 5

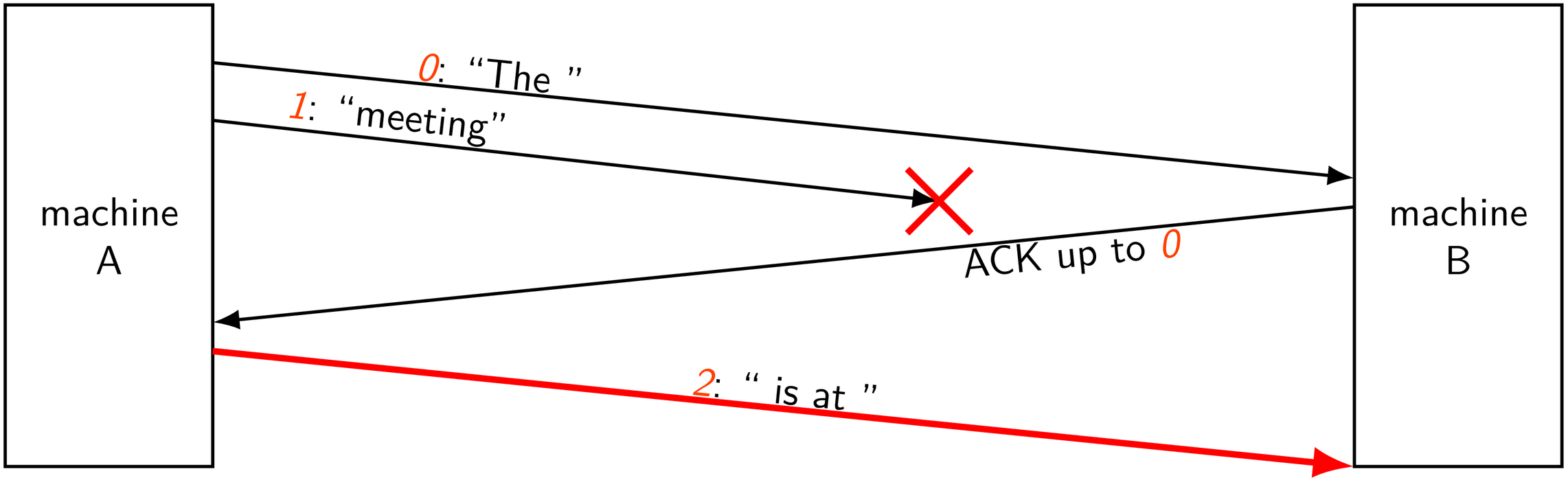
missing messages?

question: what should receiver do with sequence number 2?

one idea: ignore it?

better idea: send something back to sender

missing messages?

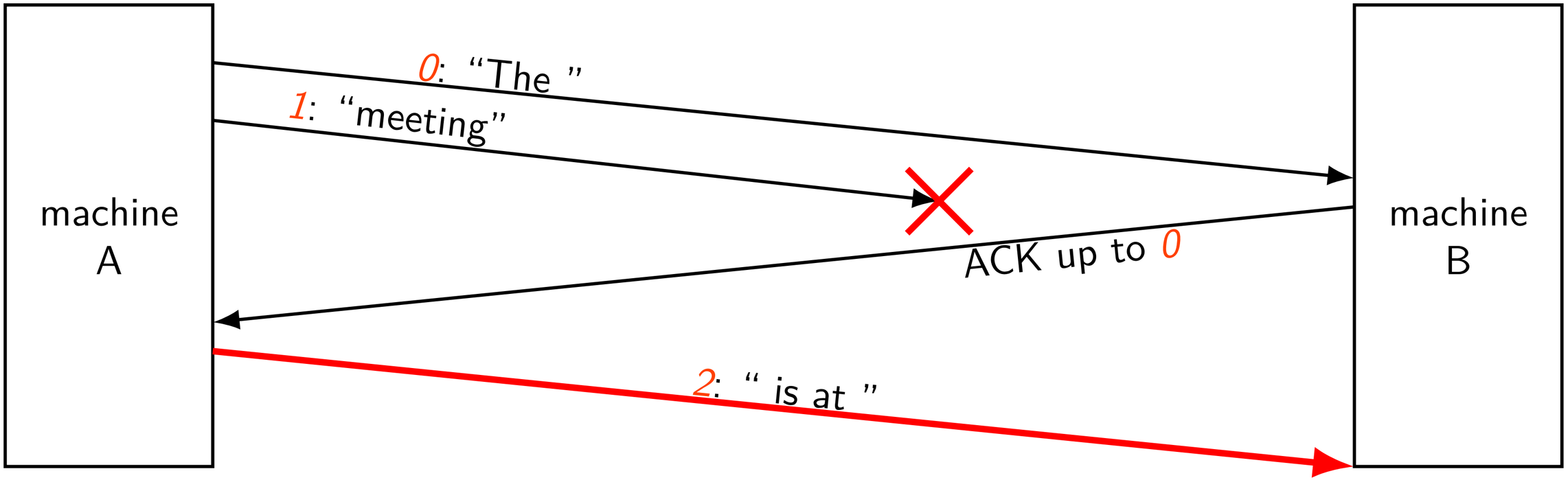


question: what should receiver do with sequence number 2?

one idea: ignore it?

better idea: send something back to sender

missing messages?



question: what should receiver do with sequence number 2?

one idea: ignore it?

better idea: send something back to sender

missing messages?

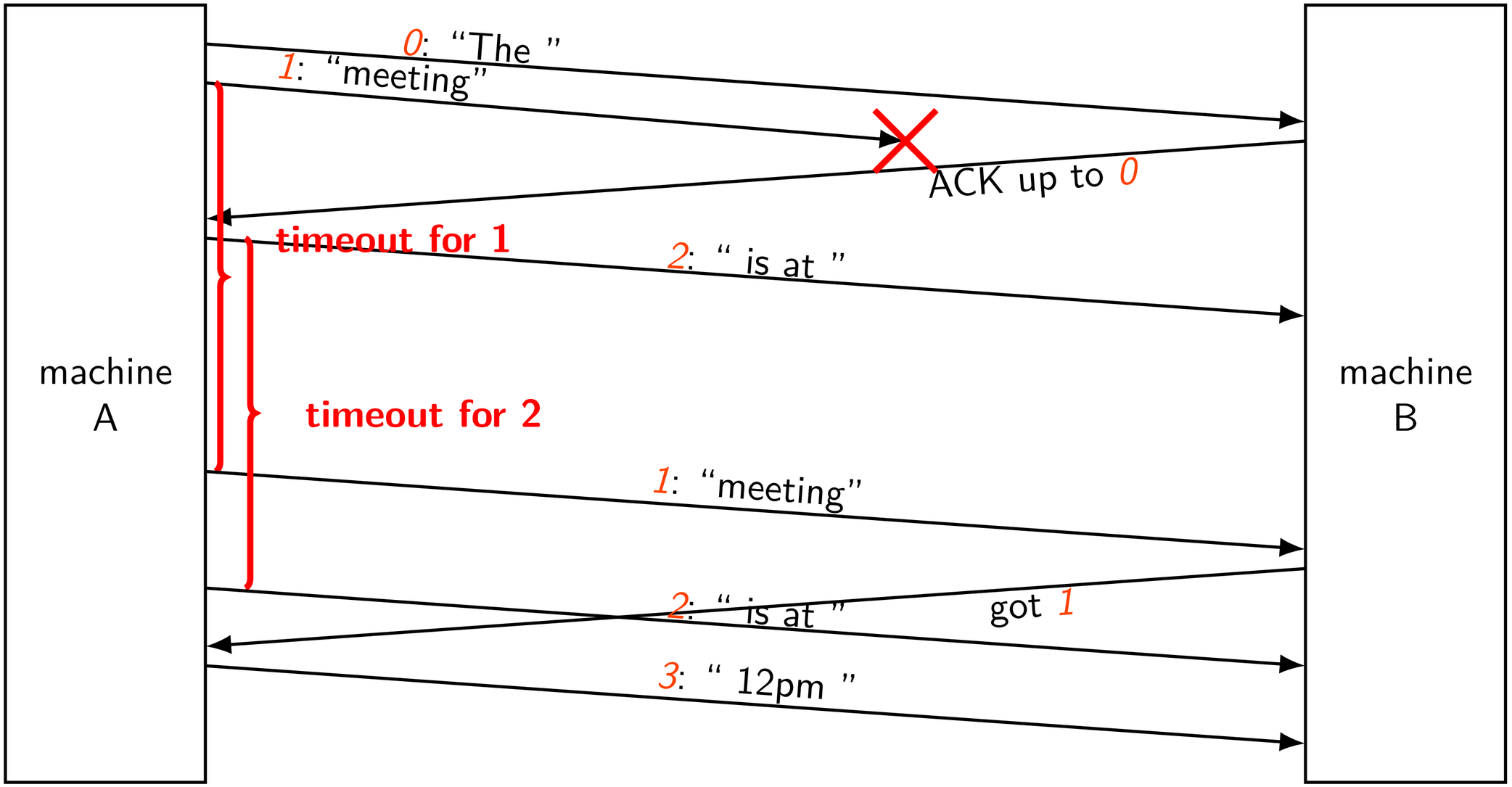
question: what should receiver do with sequence number 2?

one idea: ignore it?

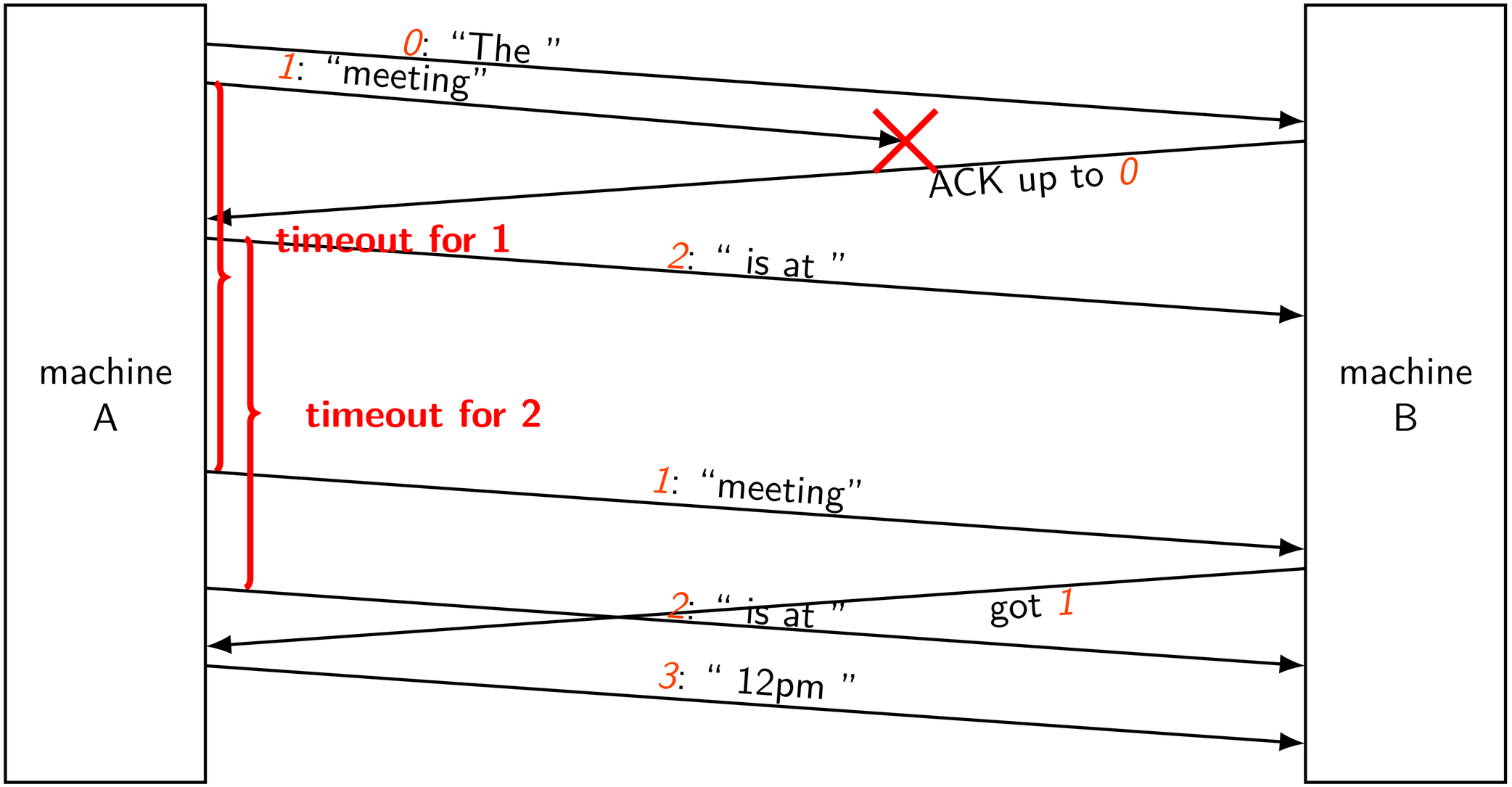
better idea: send something back to sender

not great: ignore it (1)

not great: ignore it (1)



not great: ignore it (1)



not great: only ACK if next (2)

works because we still have timeouts

but we'd like to give more feedback to receiver

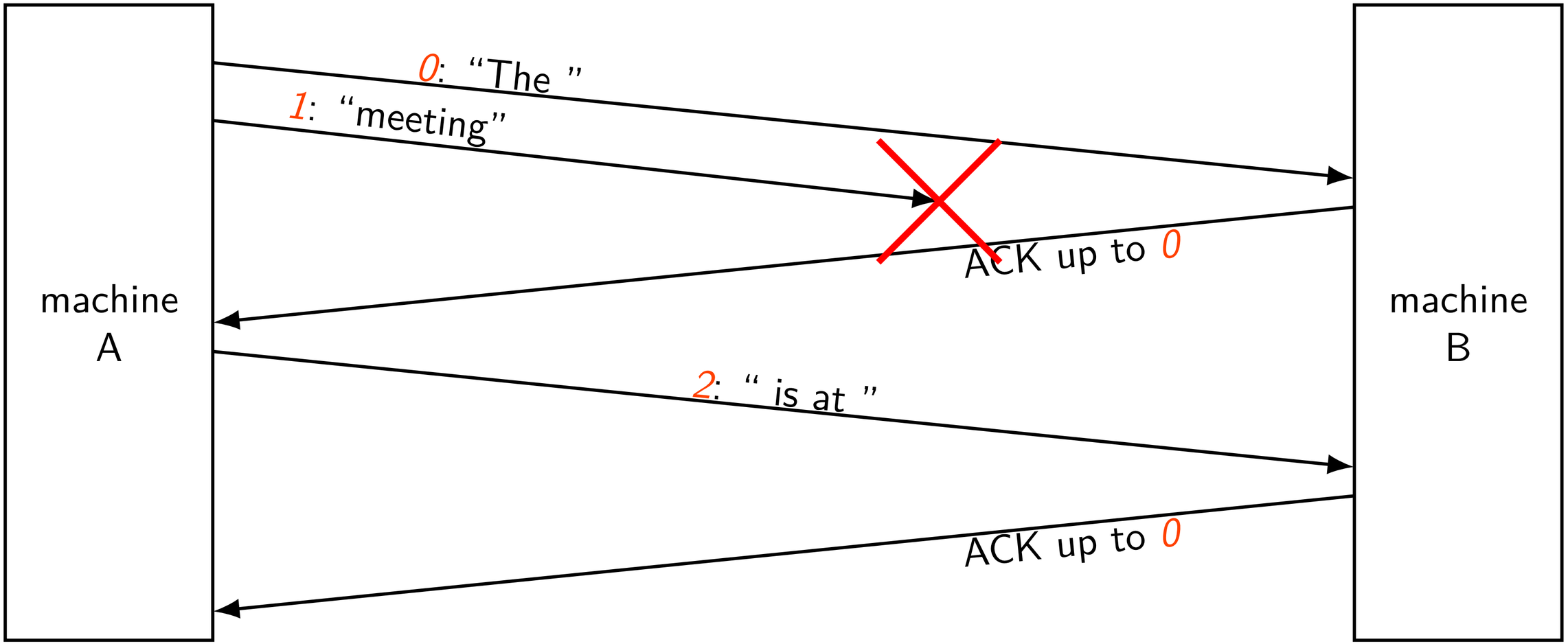
againframe(twoAndTimeouts)

better idea: always ACK

only ACK x if *everything up to and including x* received

intuition: ACK tells sender *where to start sending more*

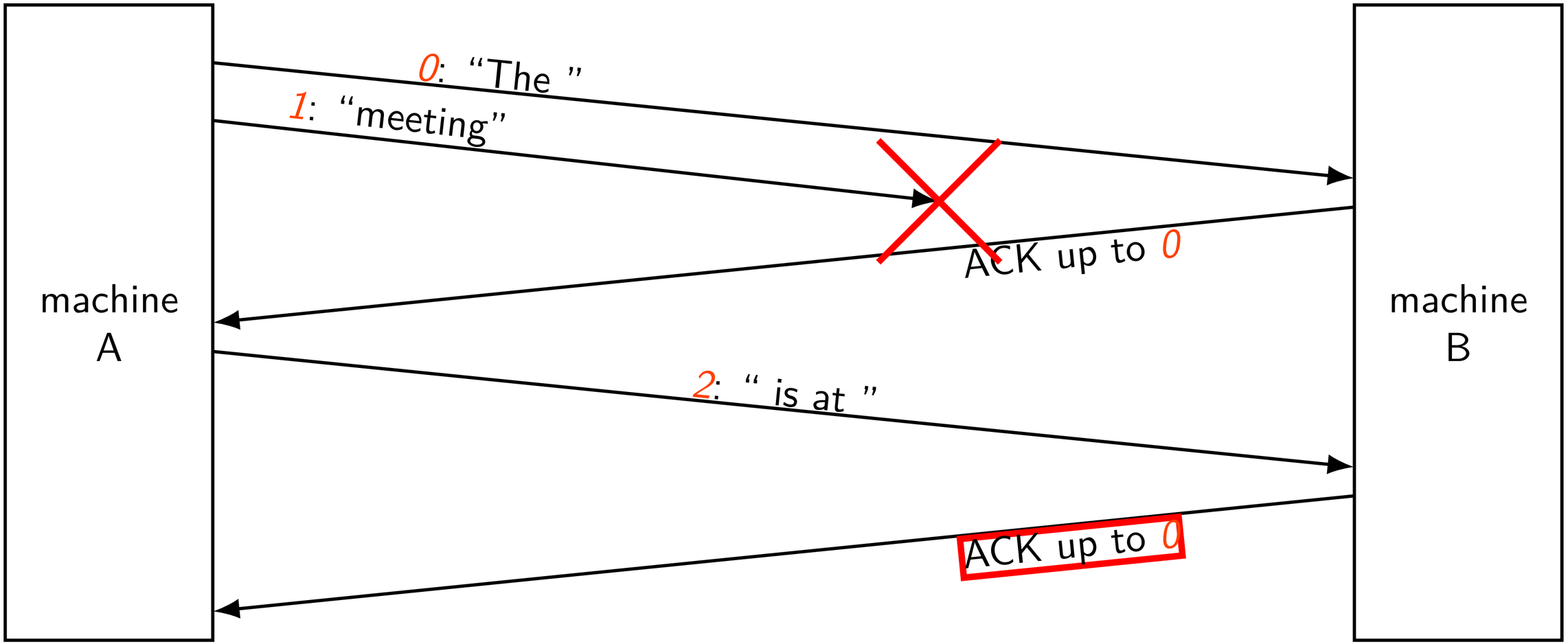
better idea: always ACK



only ACK x if *everything up to and including* x received

intuition: ACK tells sender *where to start sending more*

better idea: always ACK



only ACK x if *everything up to and including* x received

intuition: ACK tells sender *where to start sending more*

fast retransmit

if large window + data packet 2 is lost, then sender will see

ACK 0, *ACK 1, ACK 1, ACK 1, ACK 1, ACK 1*

duplicate ACKs indicate missing packet 2

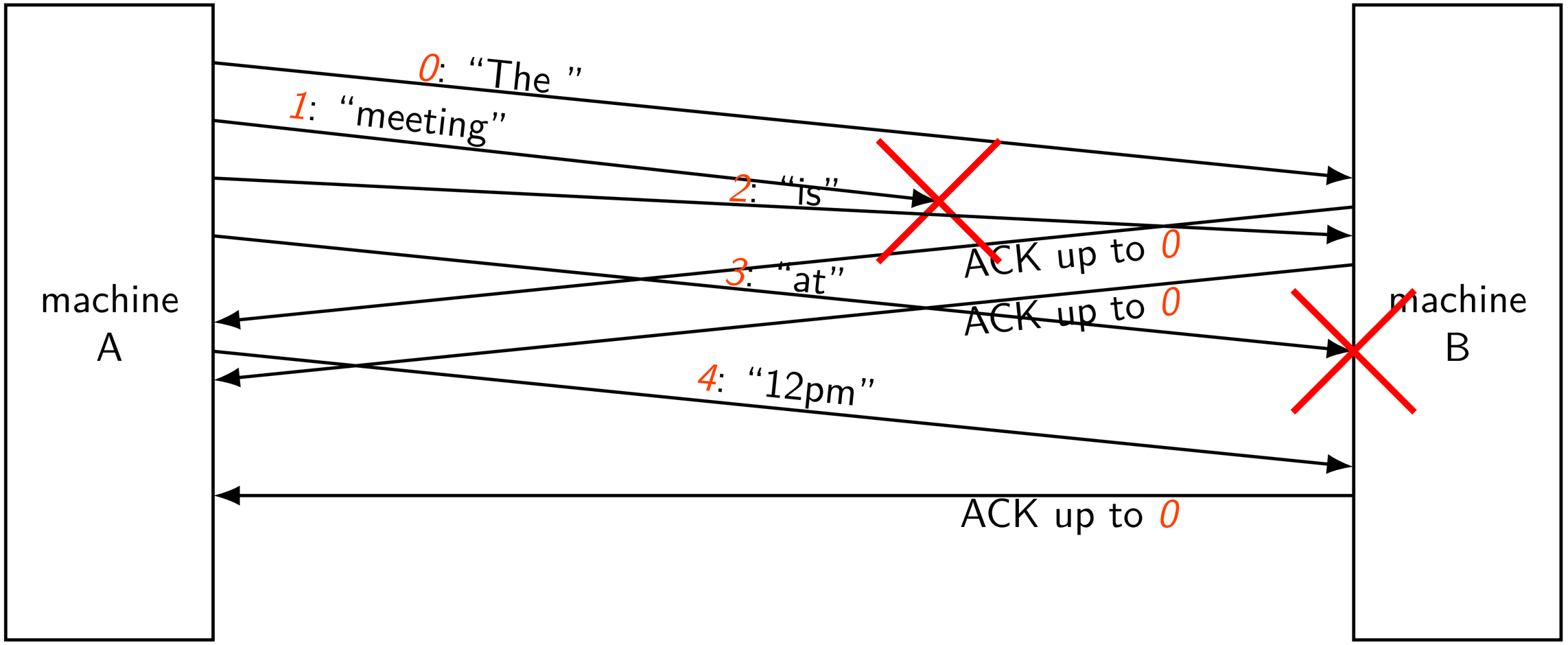
shouldn't wait for timeout

→ TCP heuristic: retransmit immediately after $$3 duplicate ACKs

not 1 duplicate ACK to tolerate some reordering

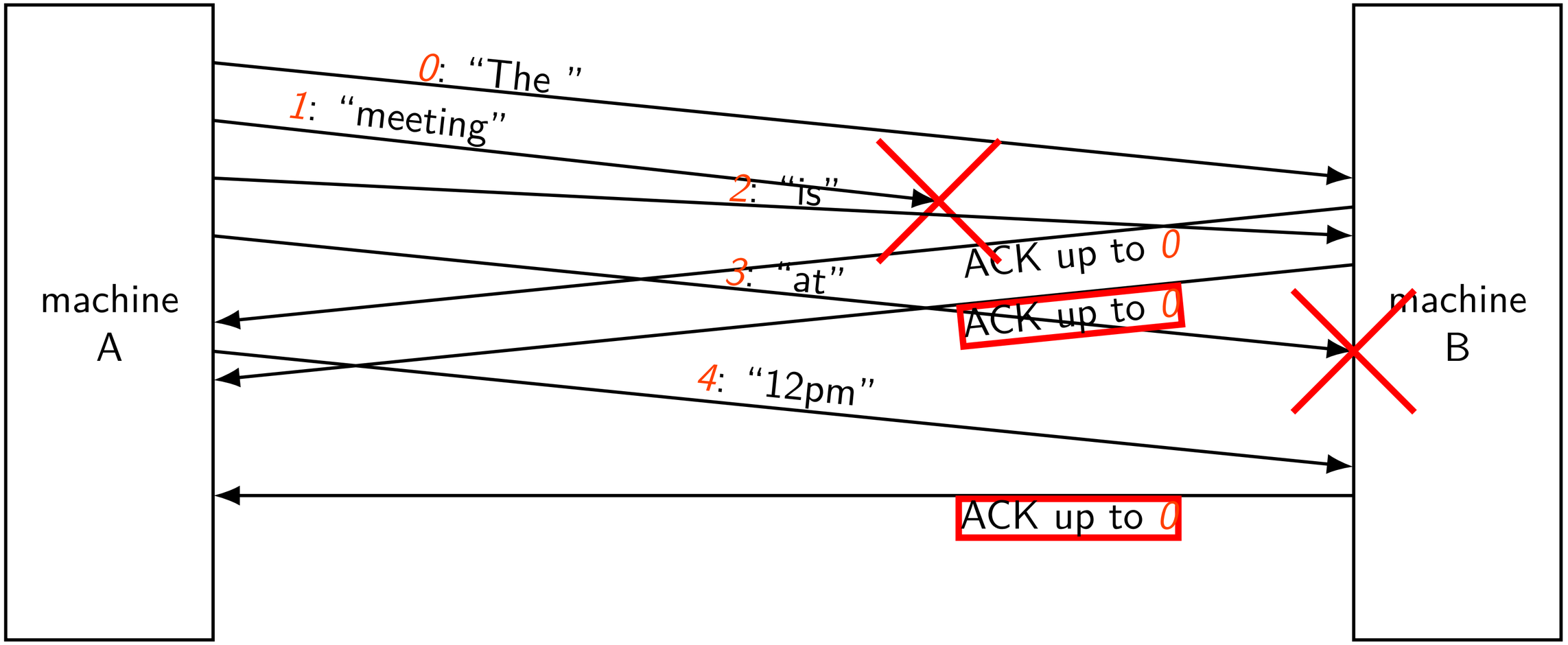
also some other details (we'll talk later)

multiple missing



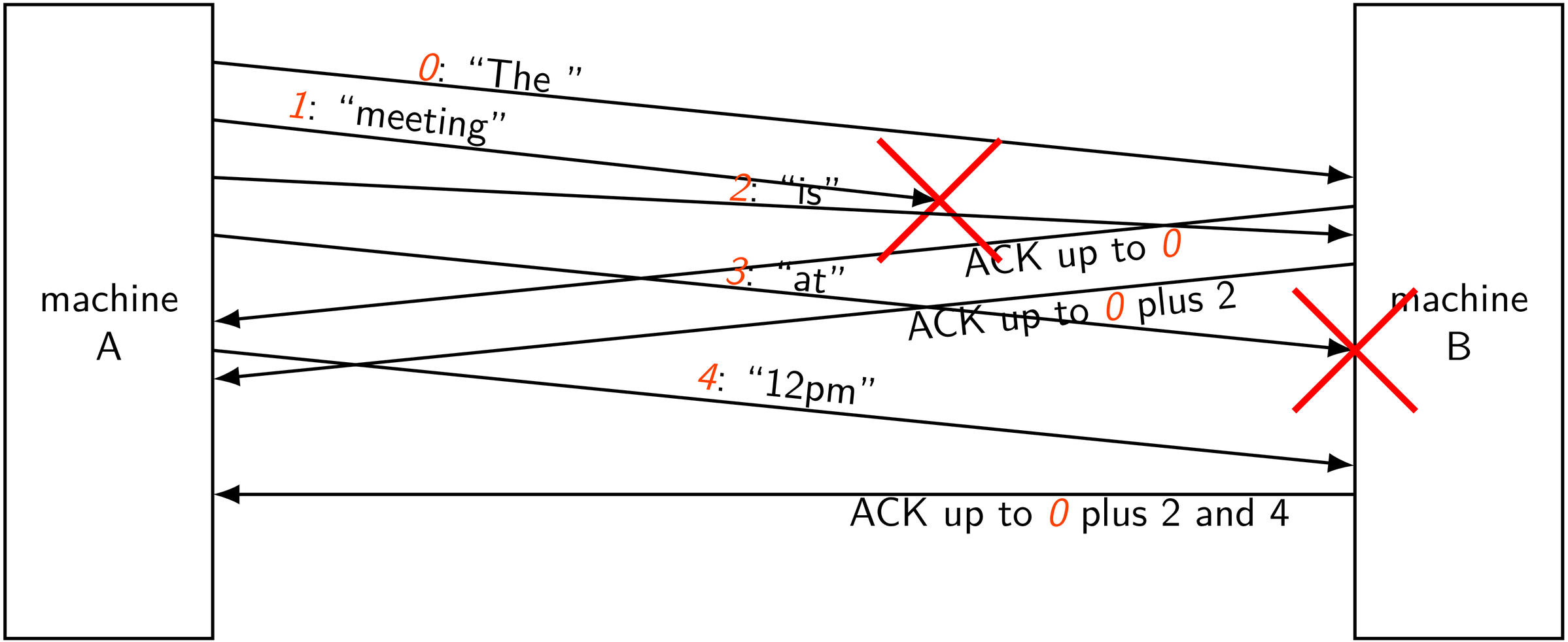
duplicate ACK heuristic will quickly resend 1, but not 3

multiple missing

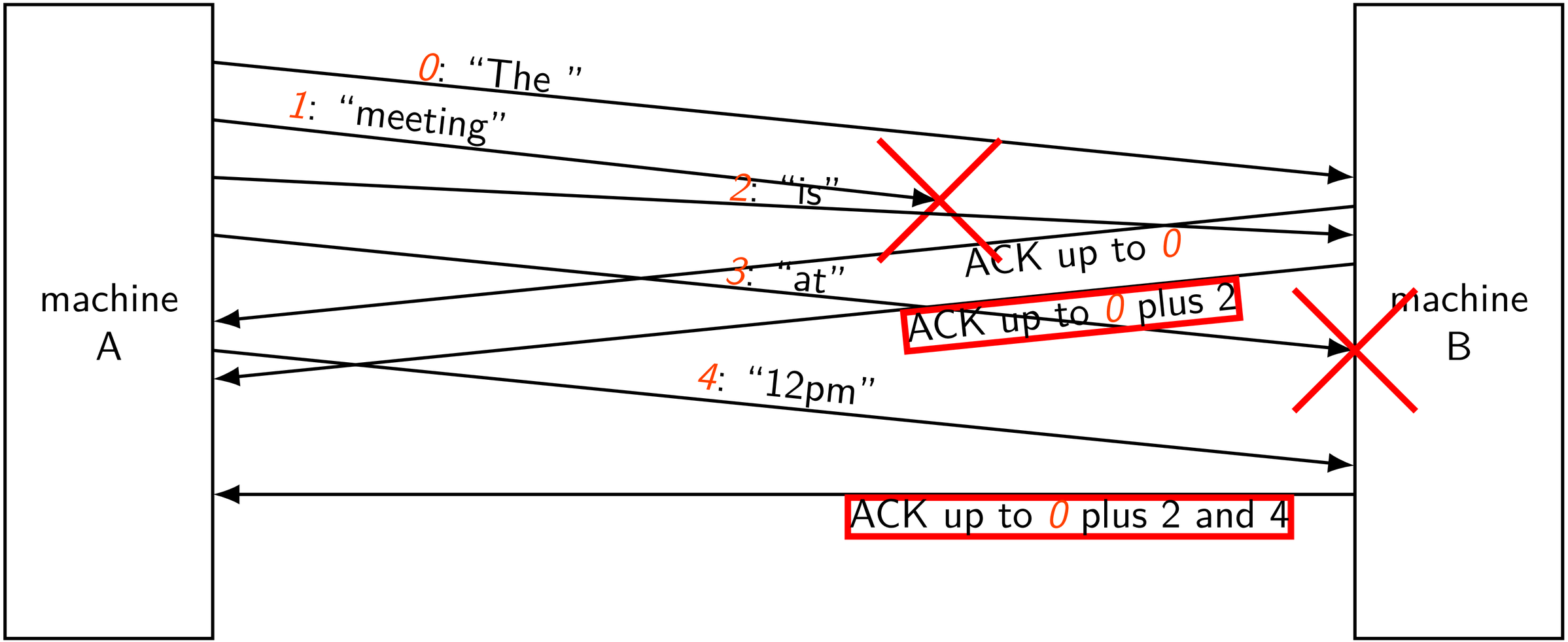


duplicate ACK heuristic will quickly resend 1, but not 3

selective acknowledgments



selective acknowledgments



selective acknowledgments in TCP

optional feature (“extension”) described in RFC 2018

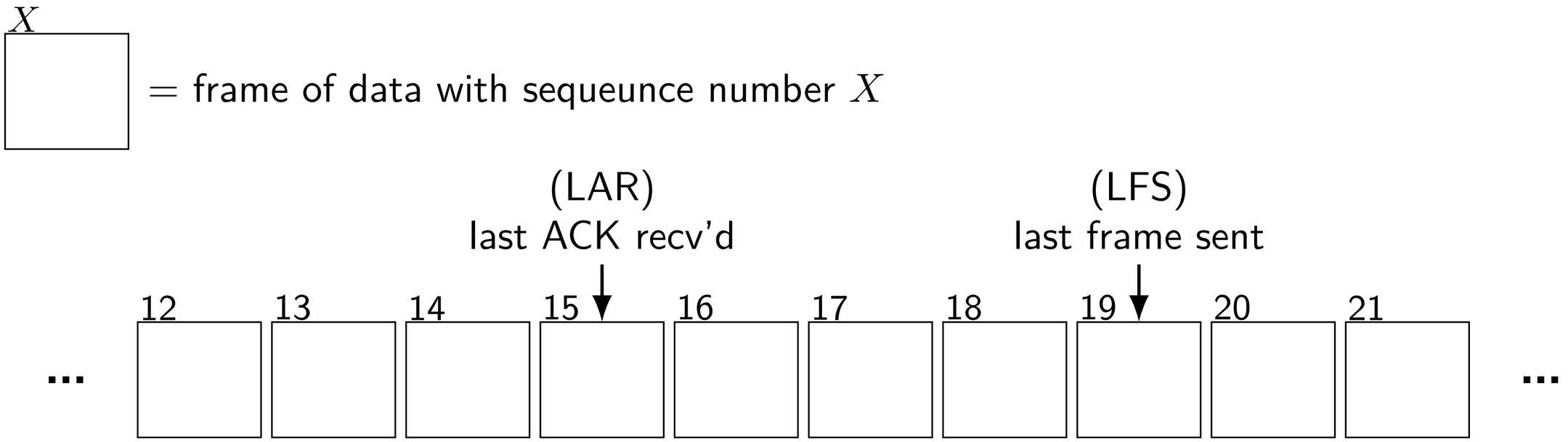
send list of ranges received

typically room for 3 ranges

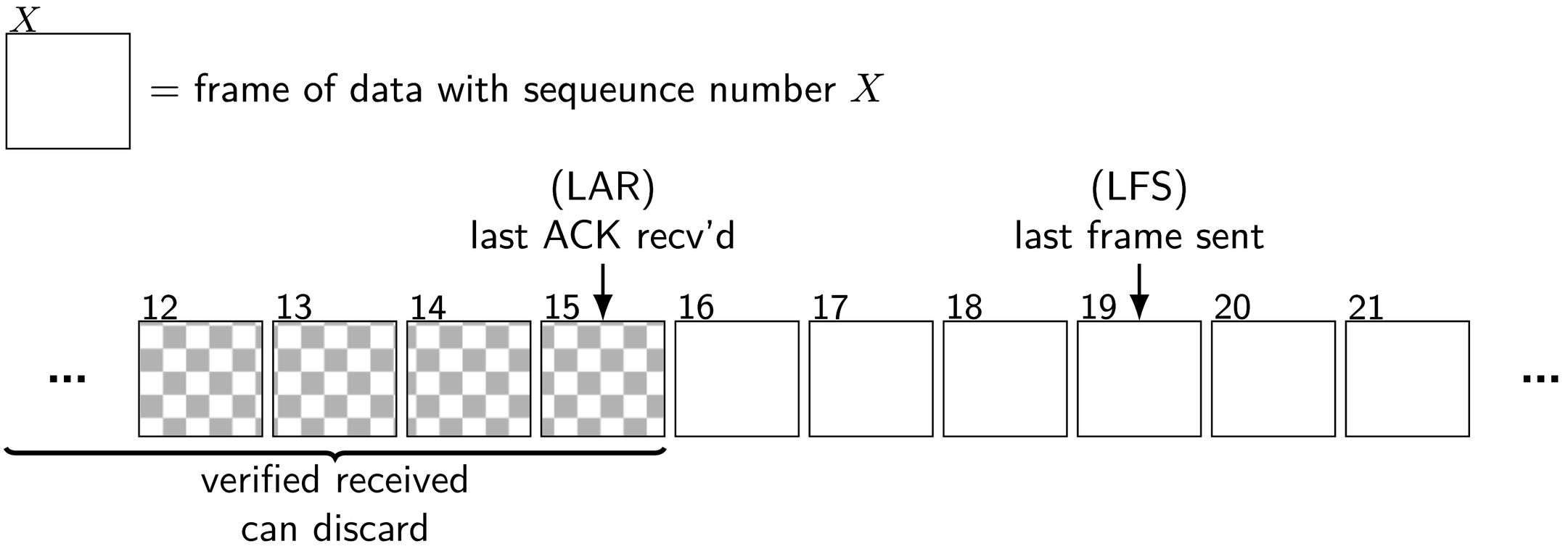
if more than 3 ranges to report, then:

- include range with most recently received frame
- include other ranges until sent three times

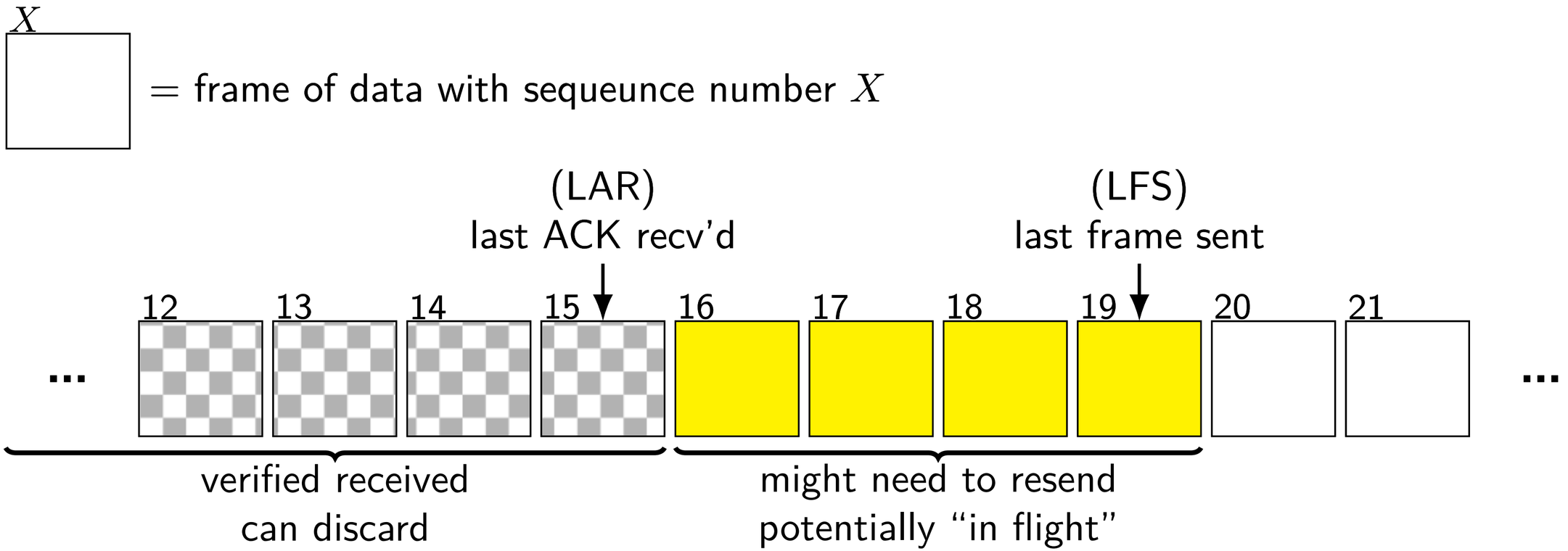
sender window tracking



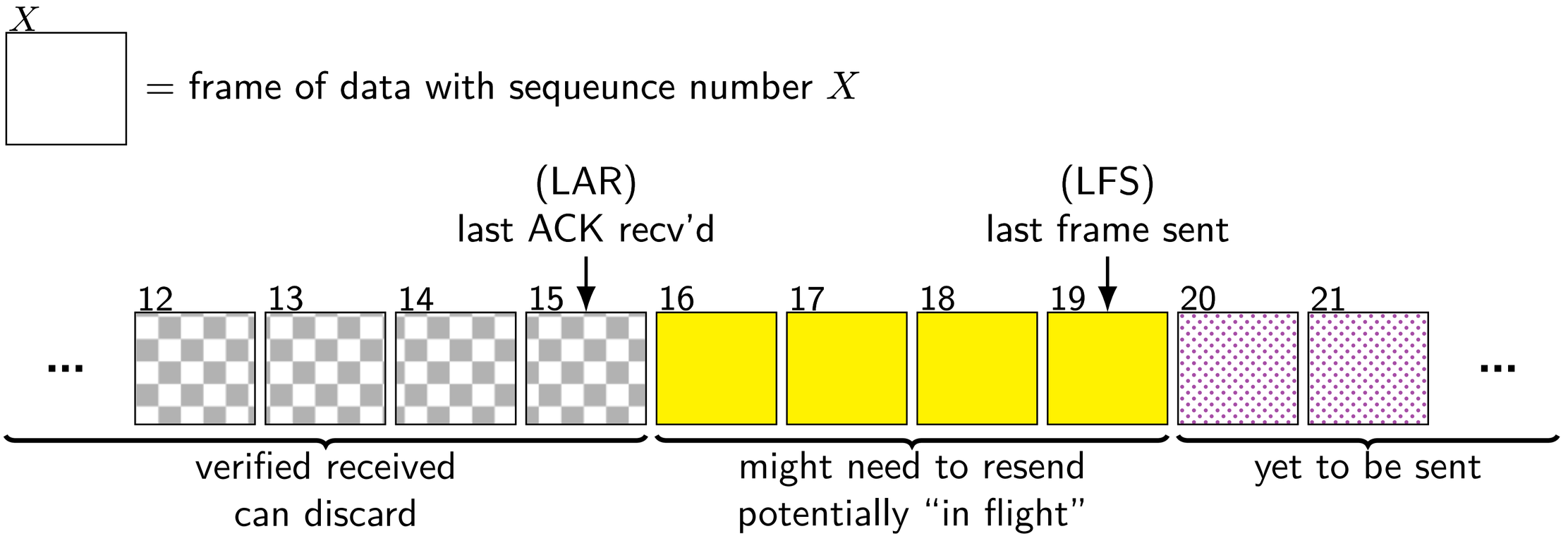
sender window tracking



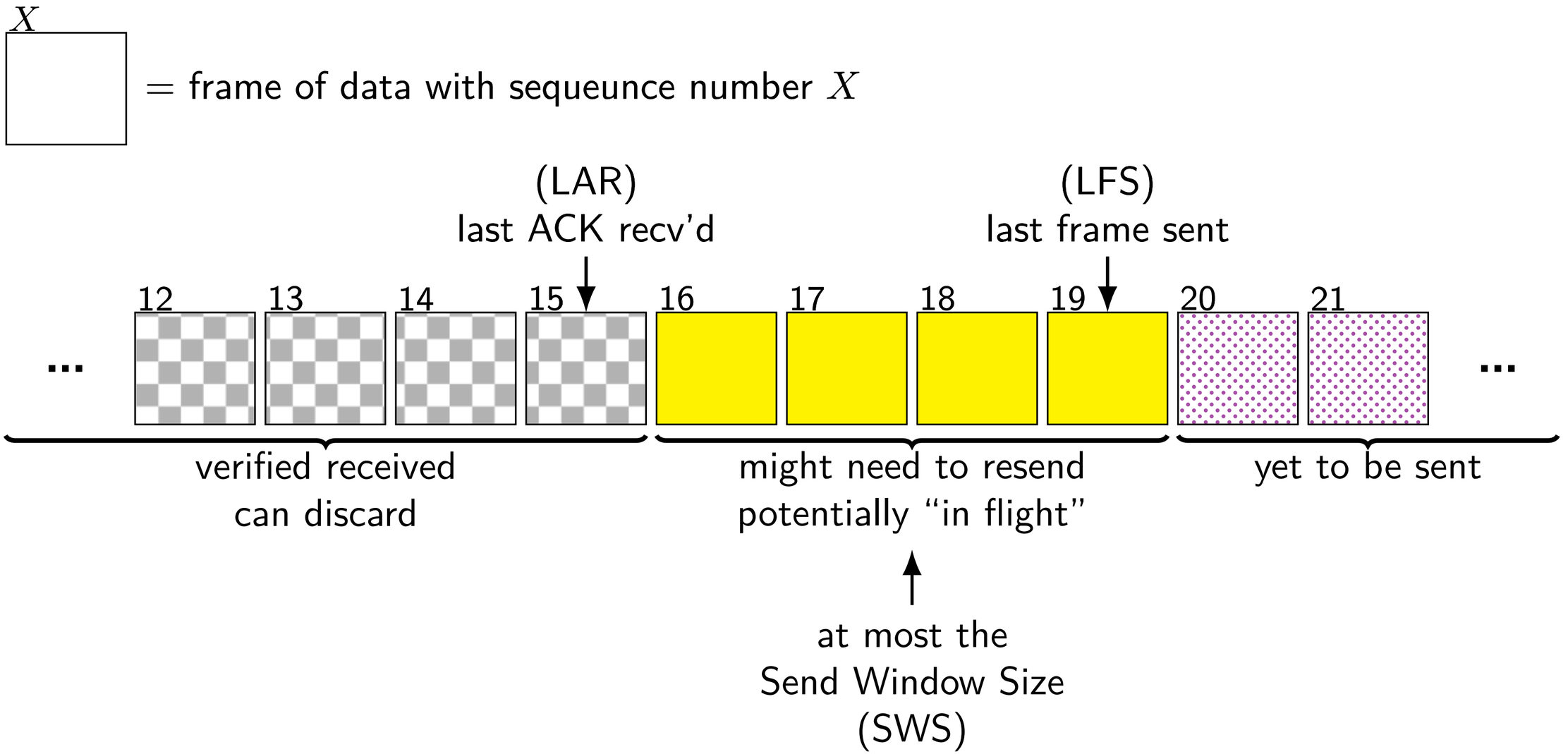
sender window tracking



sender window tracking



sender window tracking



sender window tracking

exercise 1: out-of-bounds ACK

last ACK recv'd (LAR)	10
last frame sent (LFS)	15
send window size (SWS)	5

what probably happened if we receive an ACK for…

9? 10? 13? 16?

A. if network reorders frames
B. lost ACK for frame ≤ 10
C. lost ACK for frame > 10
D. lost frame 11
E. resent frame from timeout

exercise 2: sender logic

last ACK recv'd (LAR)	10
last frame sent (LFS)	15
send window size (SWS)	5

In this case, there's a timeout that will trigger frame 13 to be resent. If still active, this timeout should be cancelled upon …

A. receiving ACK 12 B. receiving ACK 13

C. receiving ACK 14 D. sending frame 16

exercise 3a: new data

last ACK recv'd (LAR)	4
last frame sent (LFS)	6
send window size (SWS)	5

if we compute a new frame of data with sequence number 7 to eventually send, we should

A. send it now, advancing LFS
B. wait until we get an ACK for 5 or 6 to send it
C. wait until we get an ACK for 6 to send it
D. wait until the frame with sequence number 6 is resent to send it
D. something else

exercise 3b: new data

last ACK recv'd (LAR)	4
last frame sent (LFS)	8
send window size (SWS)	4

if we compute a new frame of data with sequence number 9 to eventually send, we should

A. send it now, advancing LFS
B. wait until we get an ACK for 5 or 6 or 7 or 8 to send it
C. wait until we get an ACK for 6 or 7 or 8 to send it
D. decline to accept the data because we will never be able to send it
E. something else

sender logic summarized

track variables:

- LFS (last frame sent)
- LAR (last ACK recv'd)
- SWS (send window size)

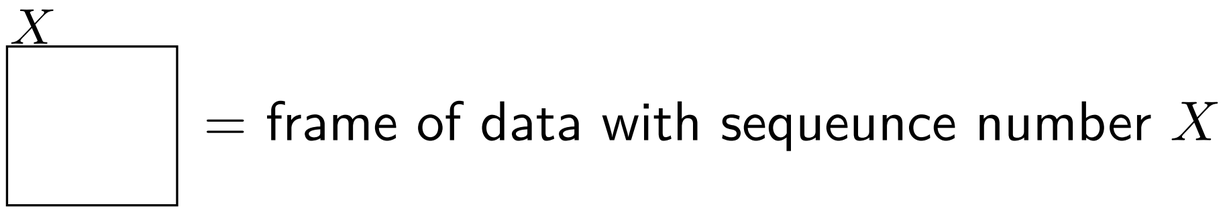
when receiving ACK $LAR < X \leq LFS$:

- LAR $\leftarrow X$
- clear any timers to resend frames $\leq X$

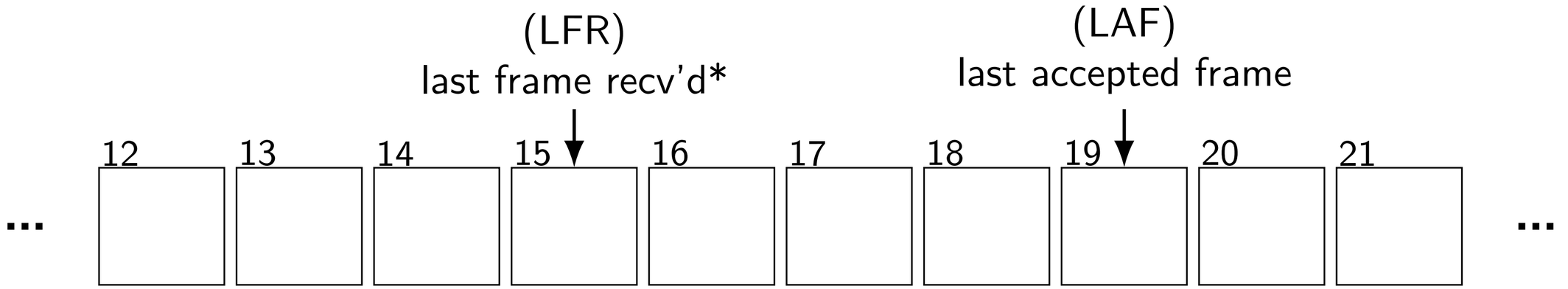
whenever SWS [send window size] > LFS - LAR *and*
data for frame LFS + 1 is available:

- send frame LFS + 1
- set timer to resend frame LFS + 1
- LFS $\leftarrow$ LFS + 1

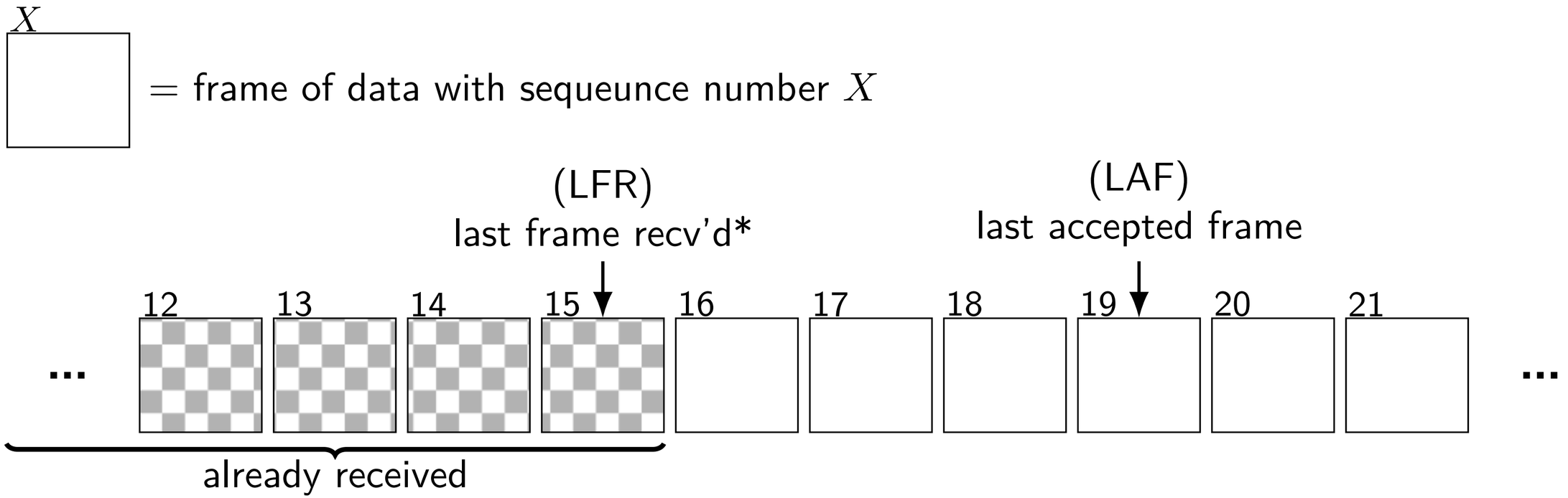
receiver window tracking



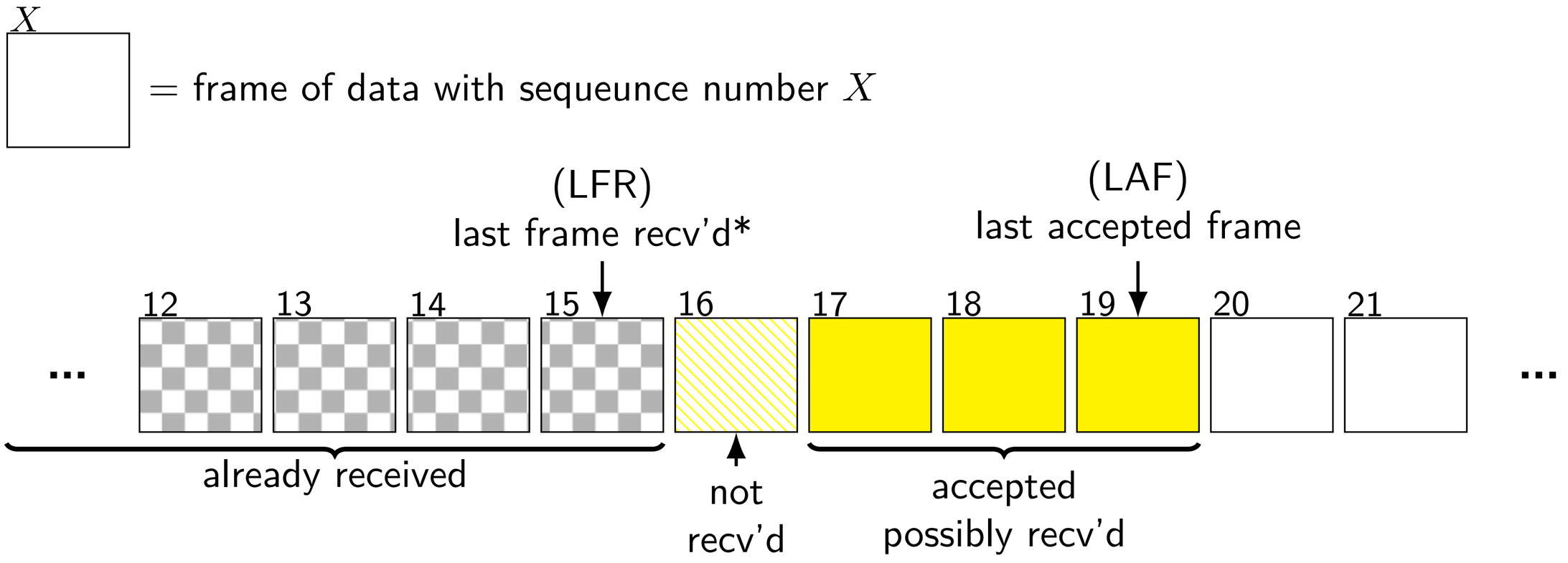
$=$ frame of data with sequeunce number X



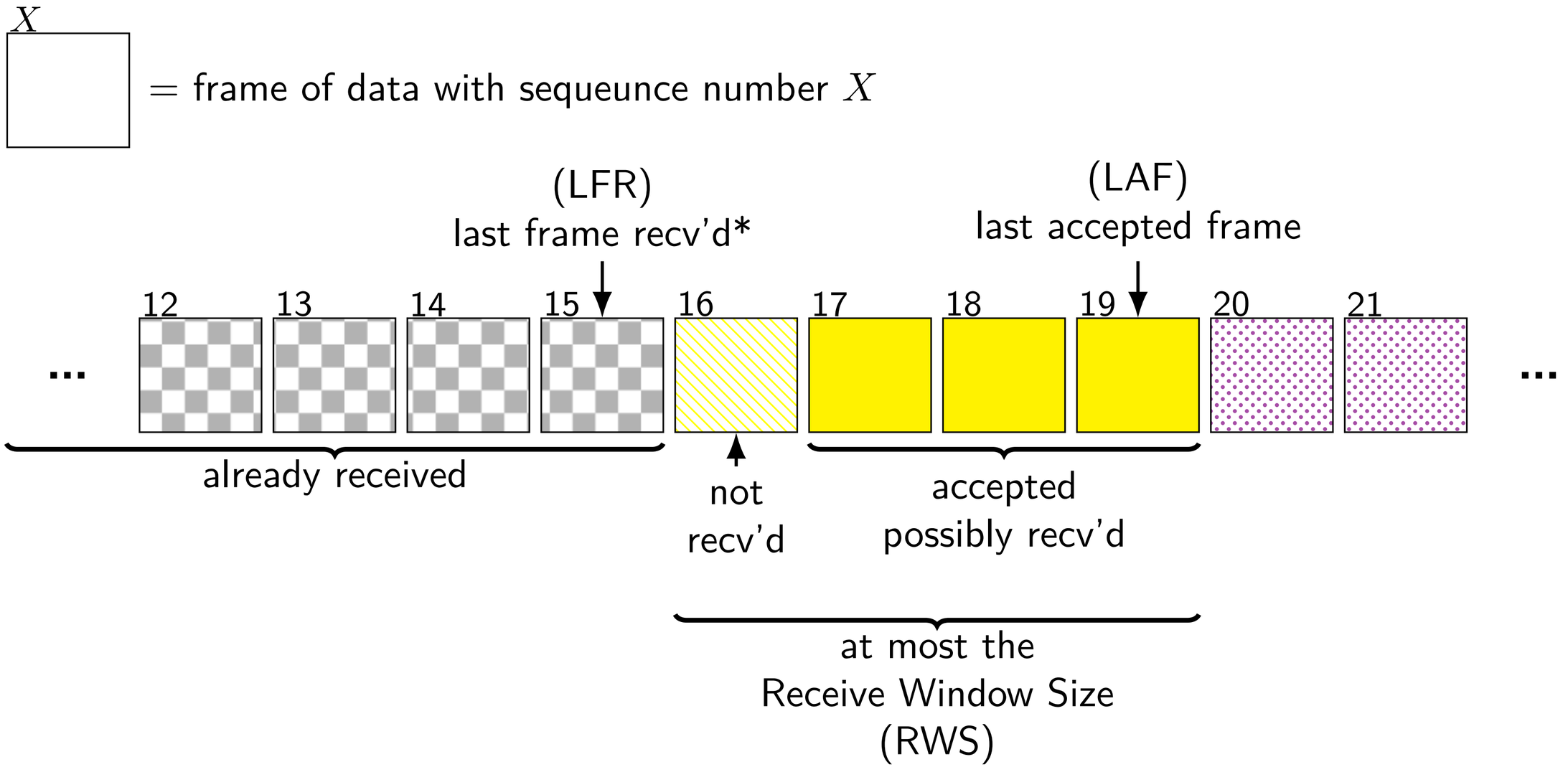
receiver window tracking



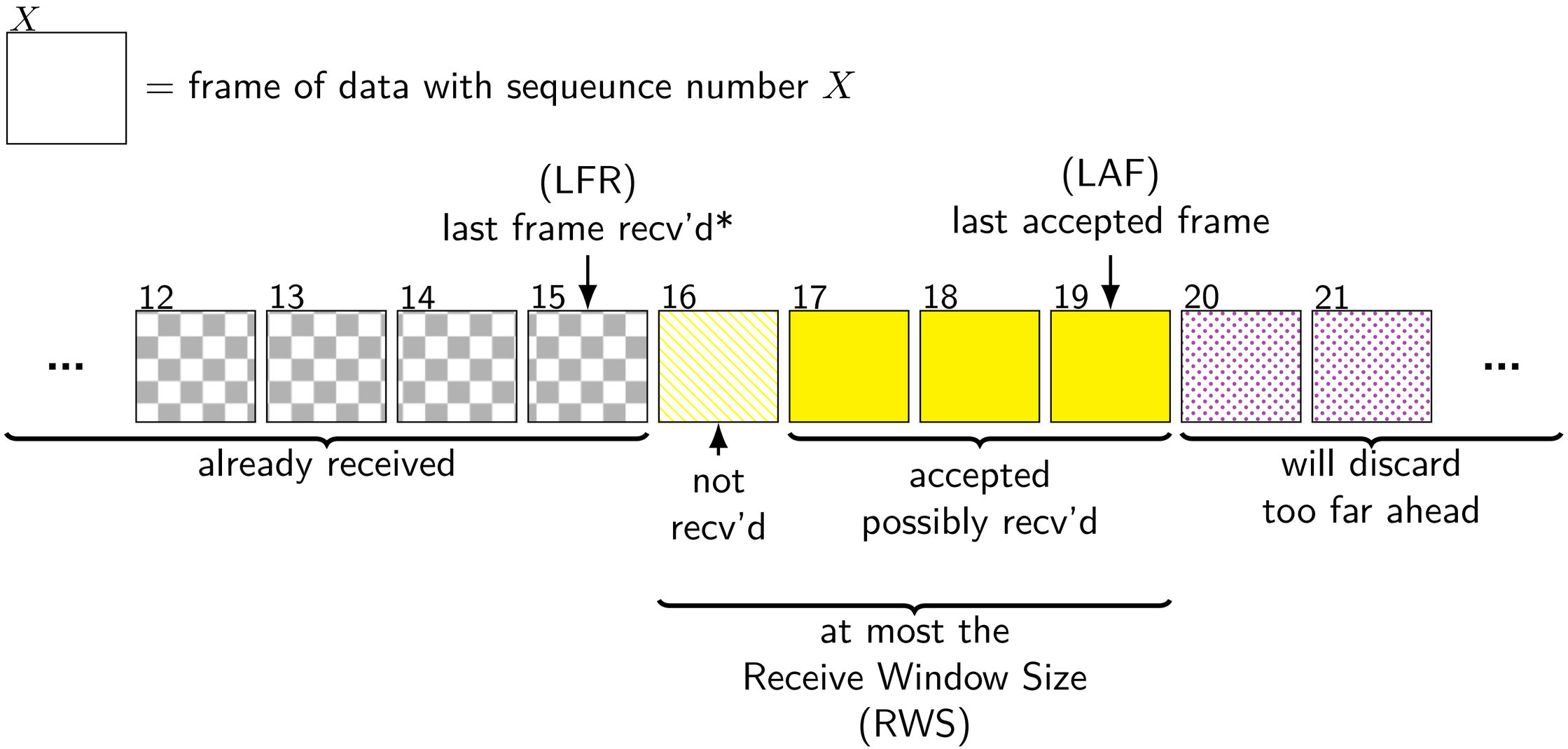
receiver window tracking



receiver window tracking



receiver window tracking



receiver logic summarized

track variables:

- LFR (last frame recv'd) — excludes frames after a missing frame
- LAF (last accepted frame)
- RWS (receive window size)

when receiving frame $LFR < X \leq LAF$:

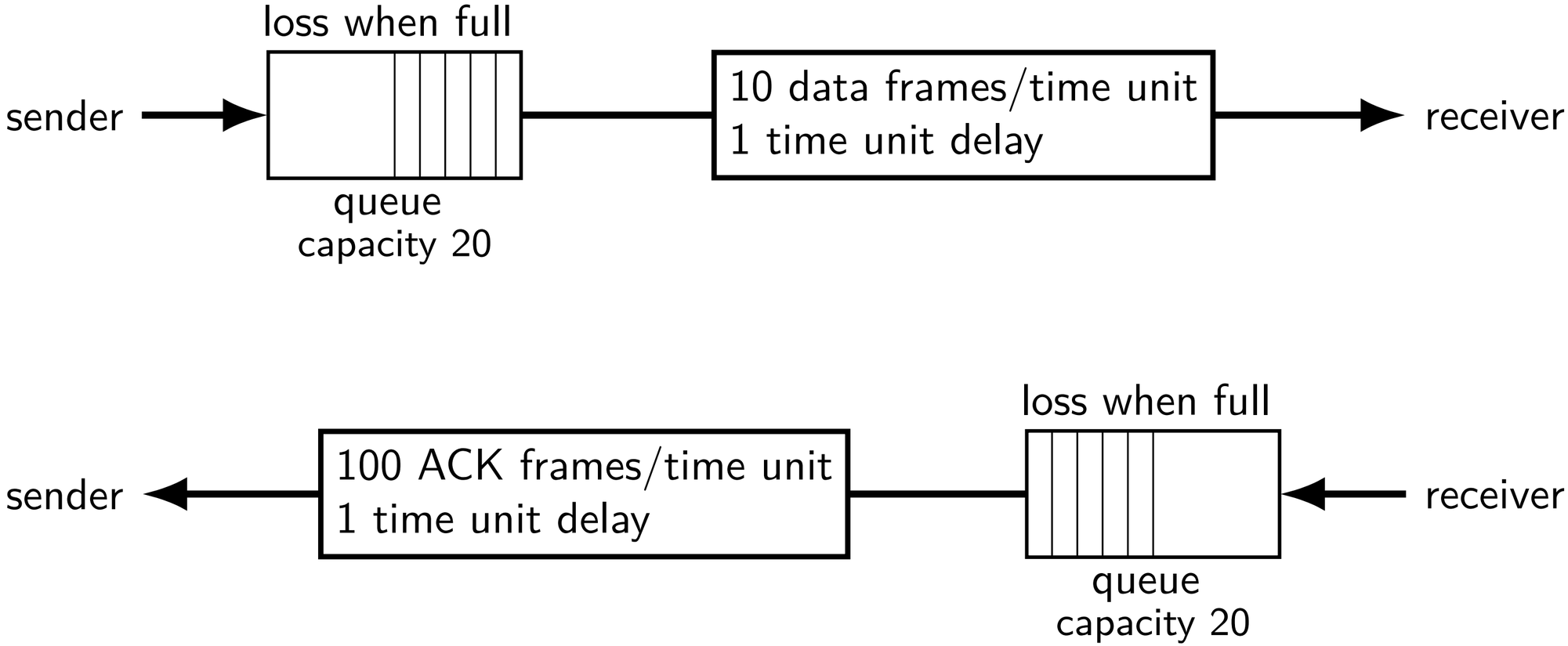
LFR $\leftarrow$ (first missing frame after LFR) - 1

- only advances if X $=$ LFR
- could advance by more than one if frames previously out of order

LAS $\leftarrow$ LFR + RWS

- only advances if X $=$ LFR

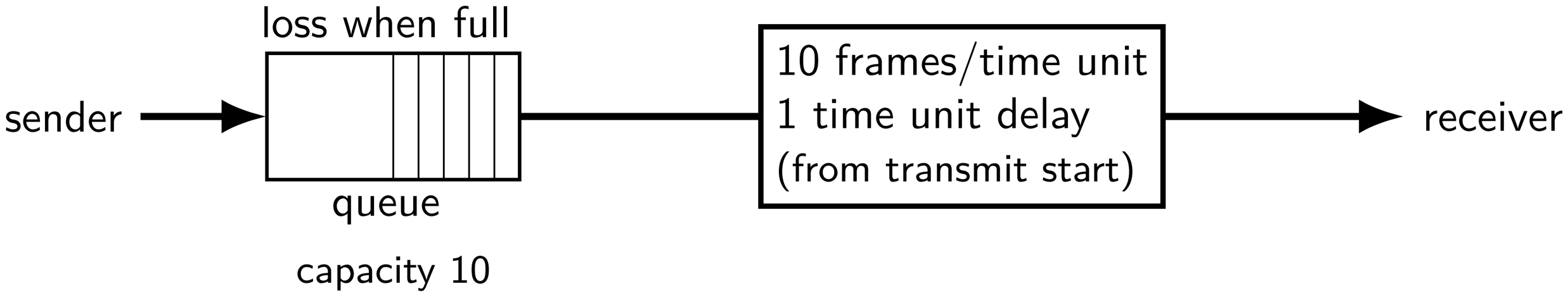
simple network model



simulator from upcoming assignment

command line `-delay 1 -bandwidth-forward 10 -bandwidth-backward 100 -buffer 30`

exercise: forward latency

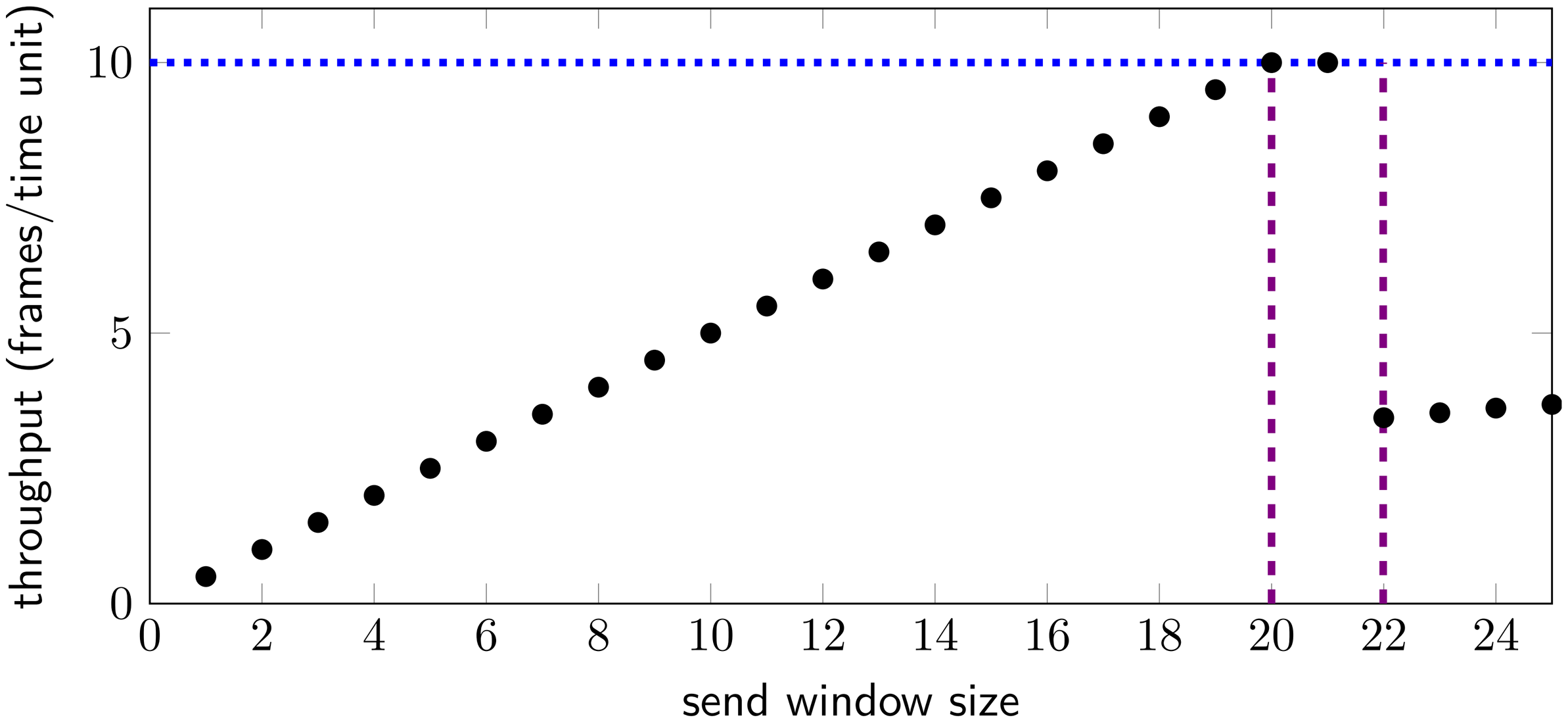


minimum latency = 1 time unit

exercise: maximum latency?

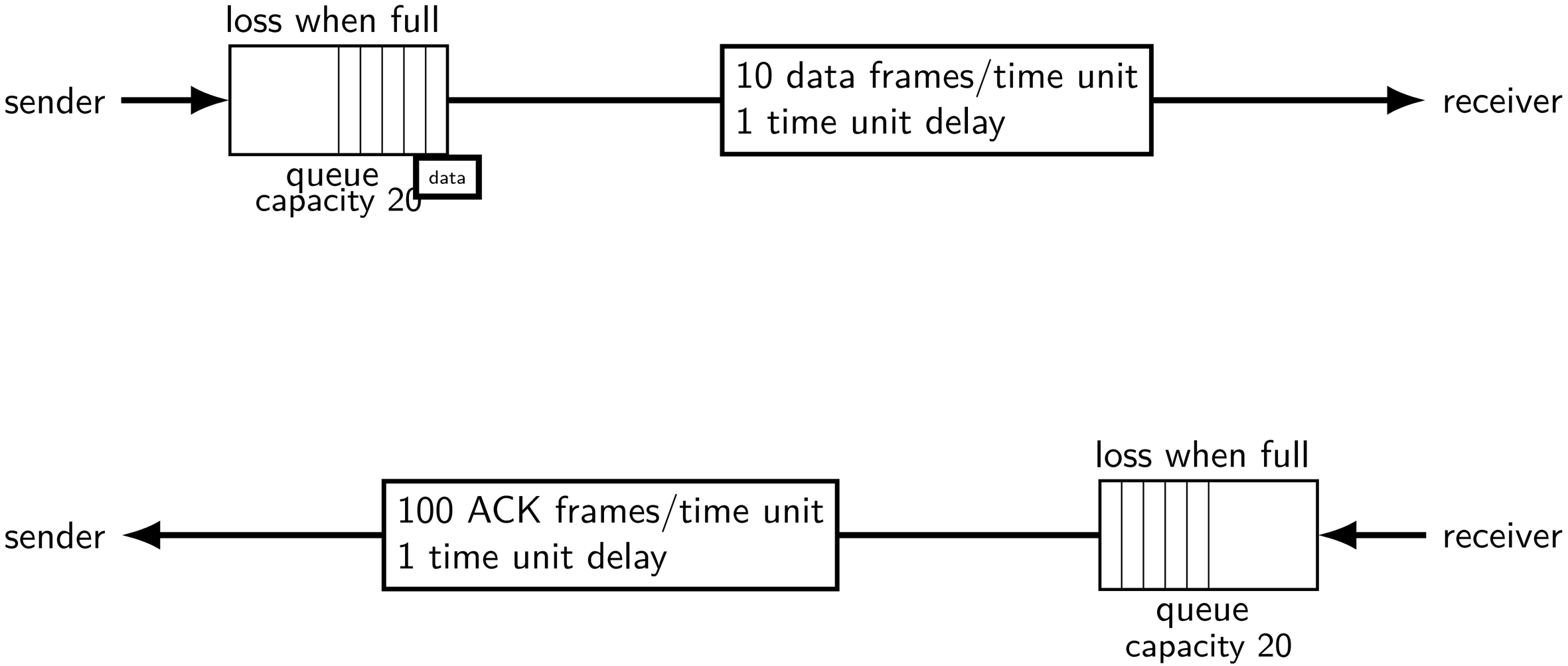
A. 1 time unit
B. 1.1 time unit
C. 1.2 time unit
C. 1.4 time unit
D. 1.9 time unit
E. 2.0 time unit
F. 2.1 time unit
G. something else

throughput and window size

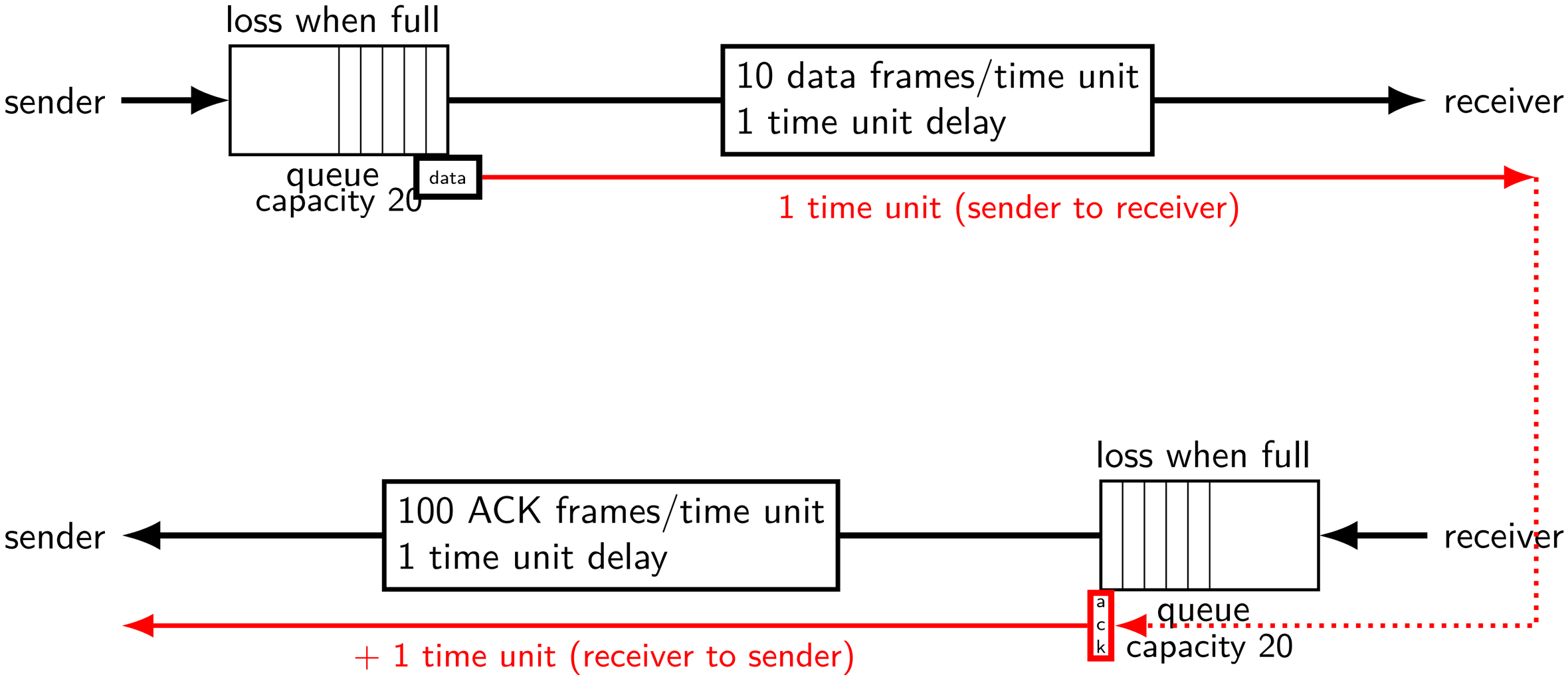


packet transit time

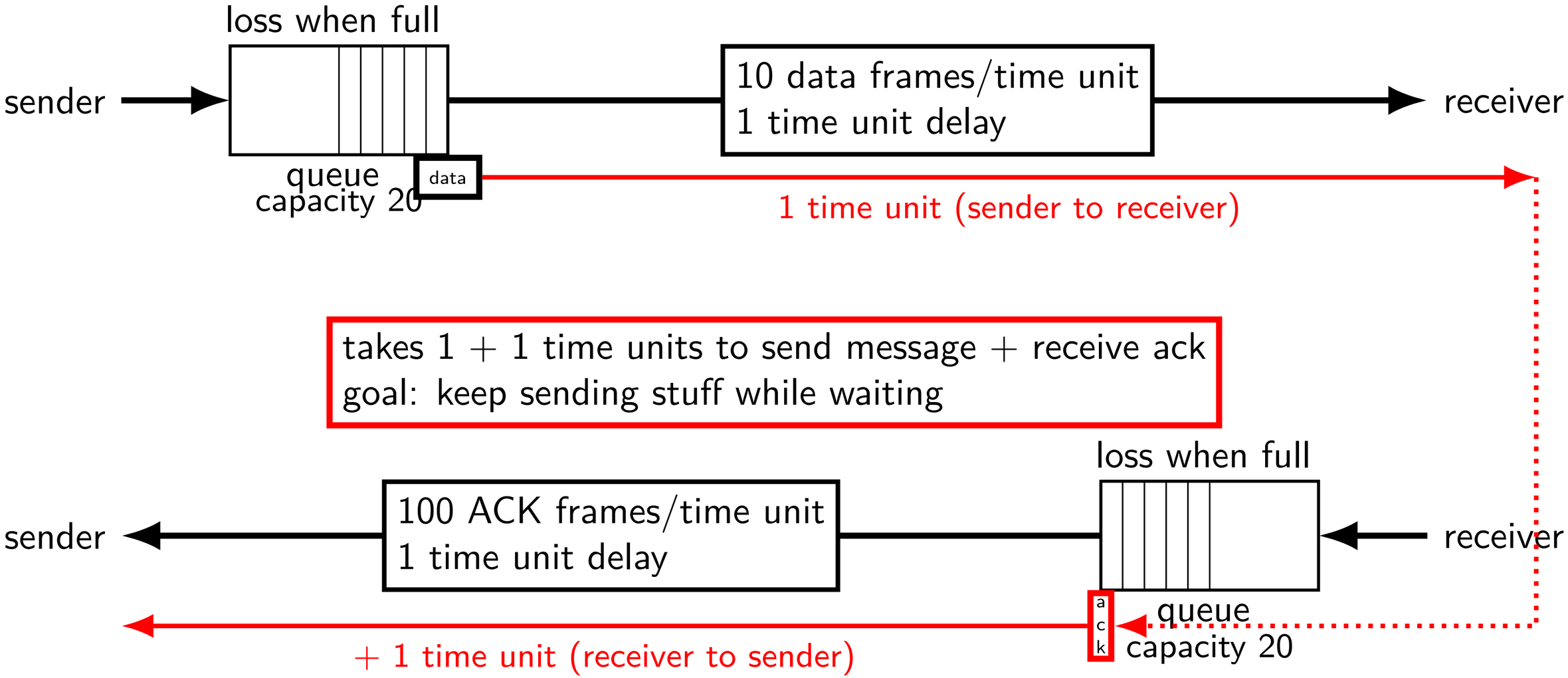
packet transit time



packet transit time



packet transit time



filling the pipe

round-trip time of 2 time units

from send data to receive ACK (assuming no queuing delay)

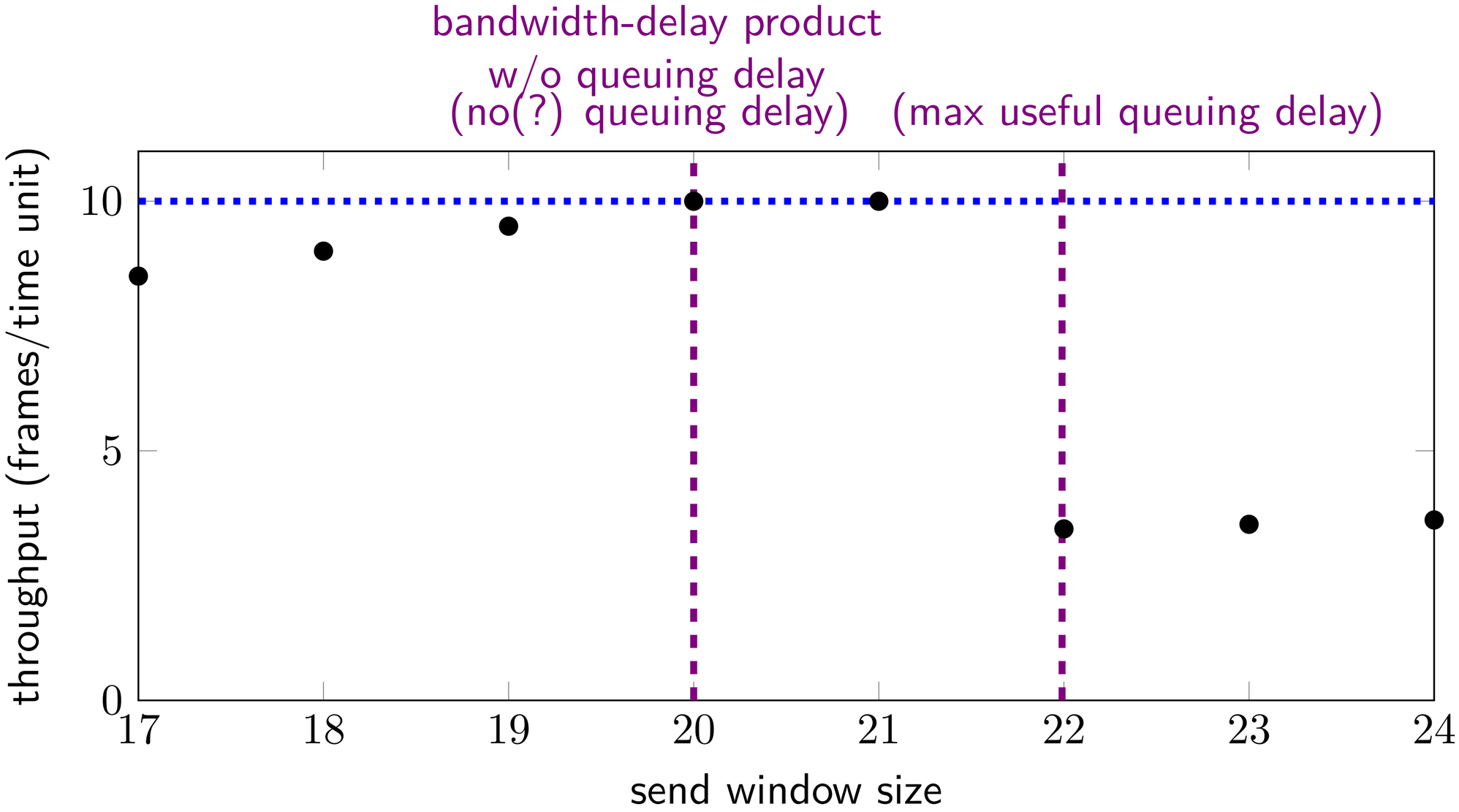
can send 10 data frames per time unit

= can send 20 data frames while waiting for ACK

“bandwidth-delay product”

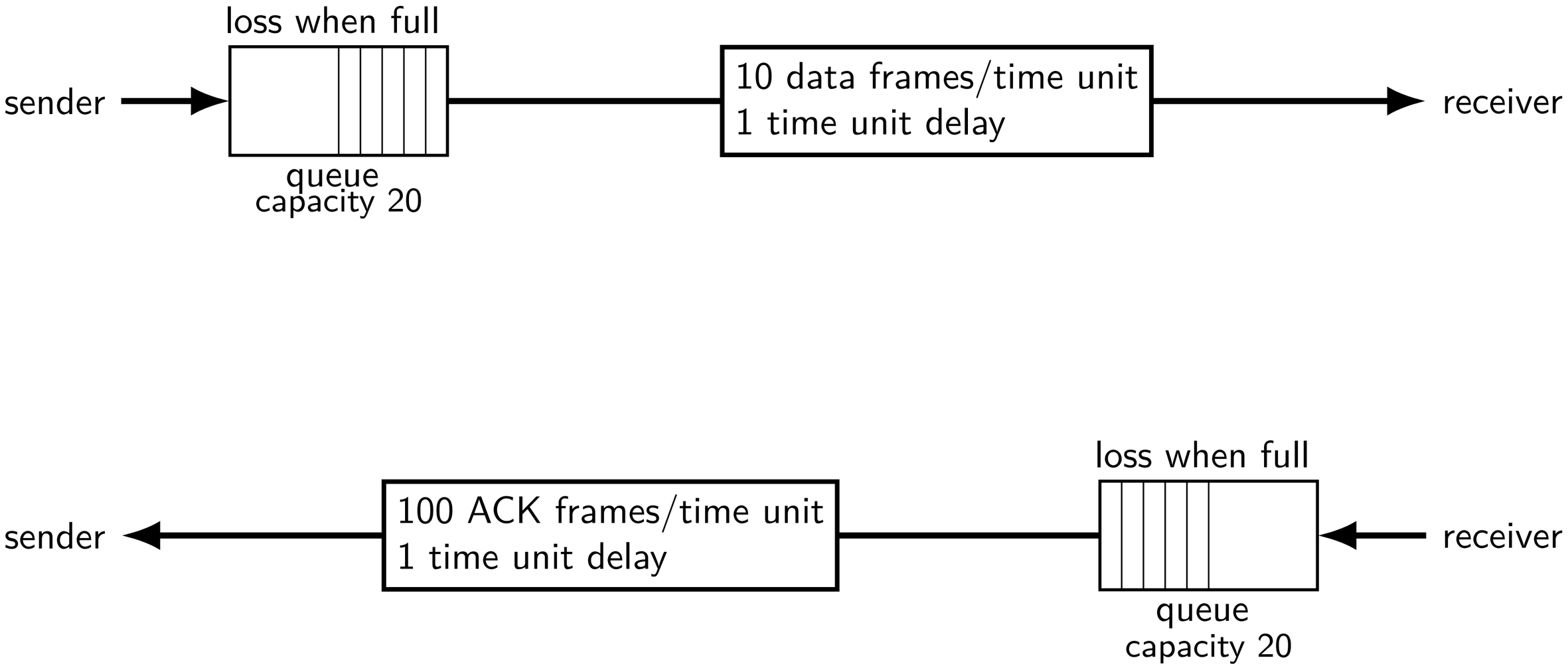
10/time unit (banwidth) times 2 time unit (RTT = delay)

throughput and window size (detail)

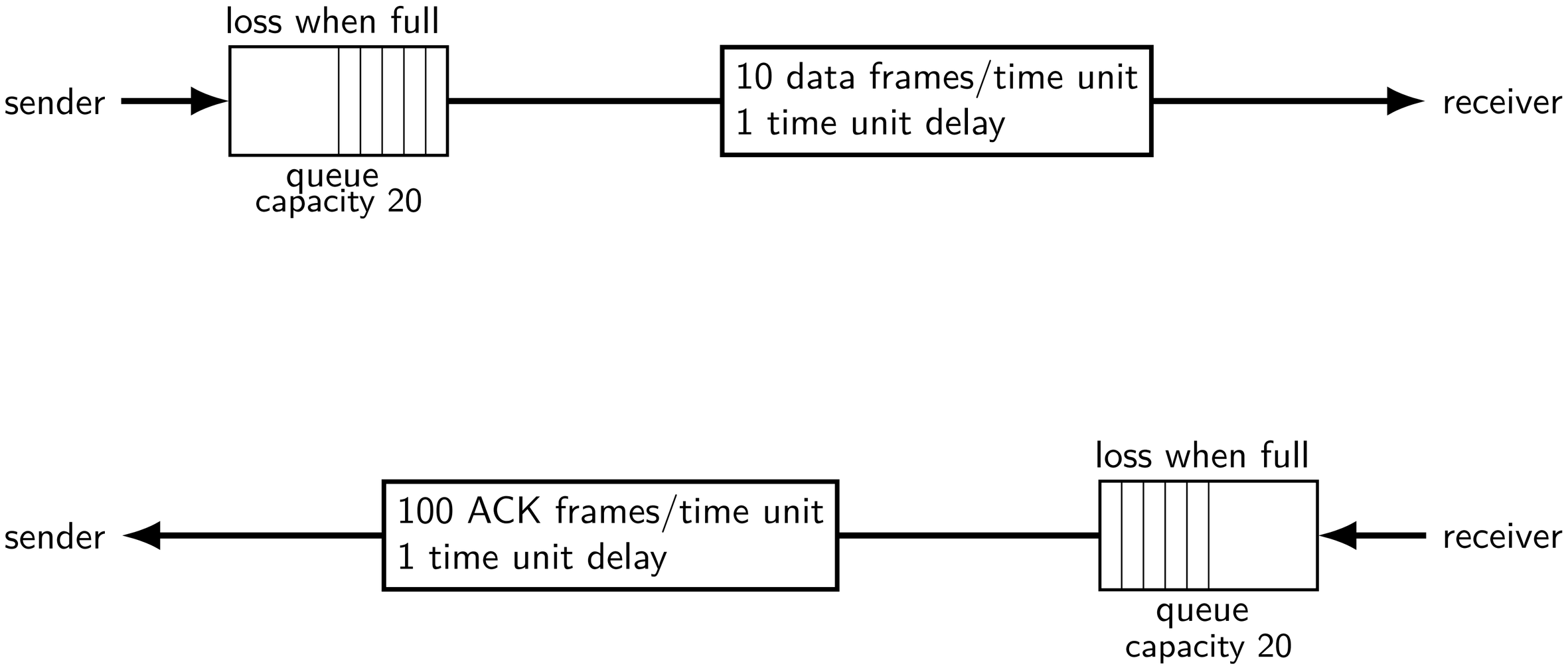


filling the pipe

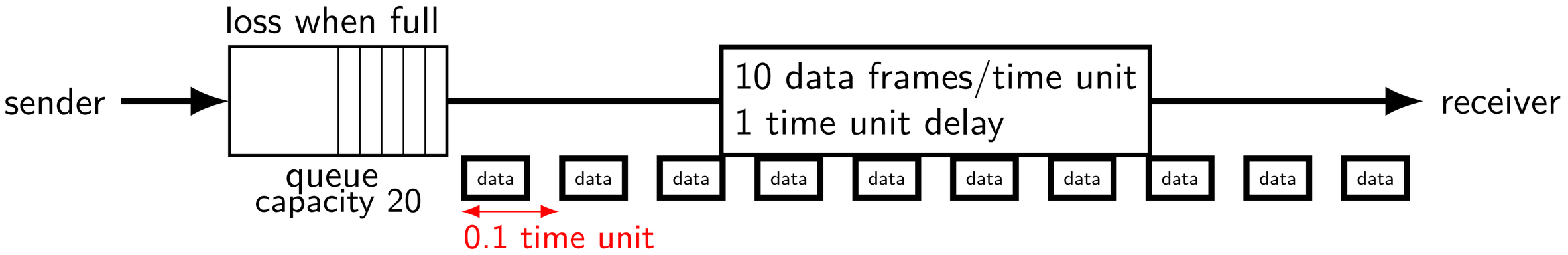
filling the pipe

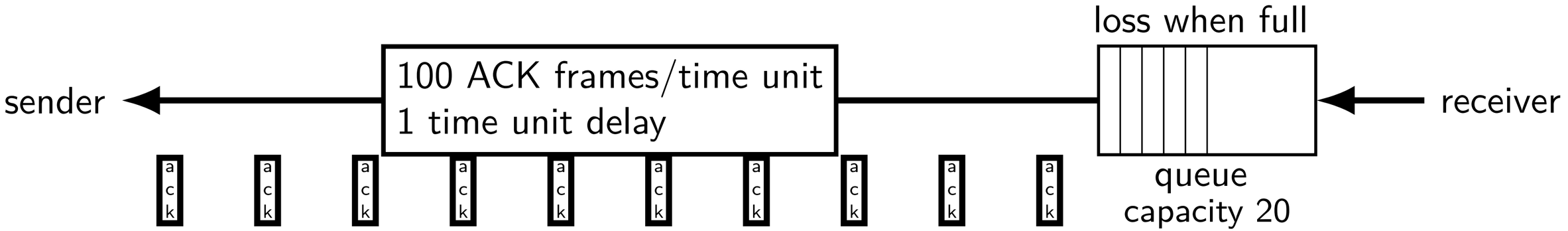


filling the pipe



filling the pipe





on bursts

max possible queuing delay suggests window size of 30

- approx. 3 time units times 10

problem: “bursts” temporarily exceed queue size

achievable average queue size not that high

sender could moderate by “pacing” packets

sliding windows used to solve

flow control

keep sender from getting too far ahead of receiver

… by having *window sizes set correctly*

how? receiver tells sender what window size is okay

congestion control

keep network from being overloaded

(while making good use of available bandwidth)

… by having *window sizes set correctly*

how? it's complicated — big topic later

sequence number wraparound

protocol so far requires arbitrarily large sequence numbers

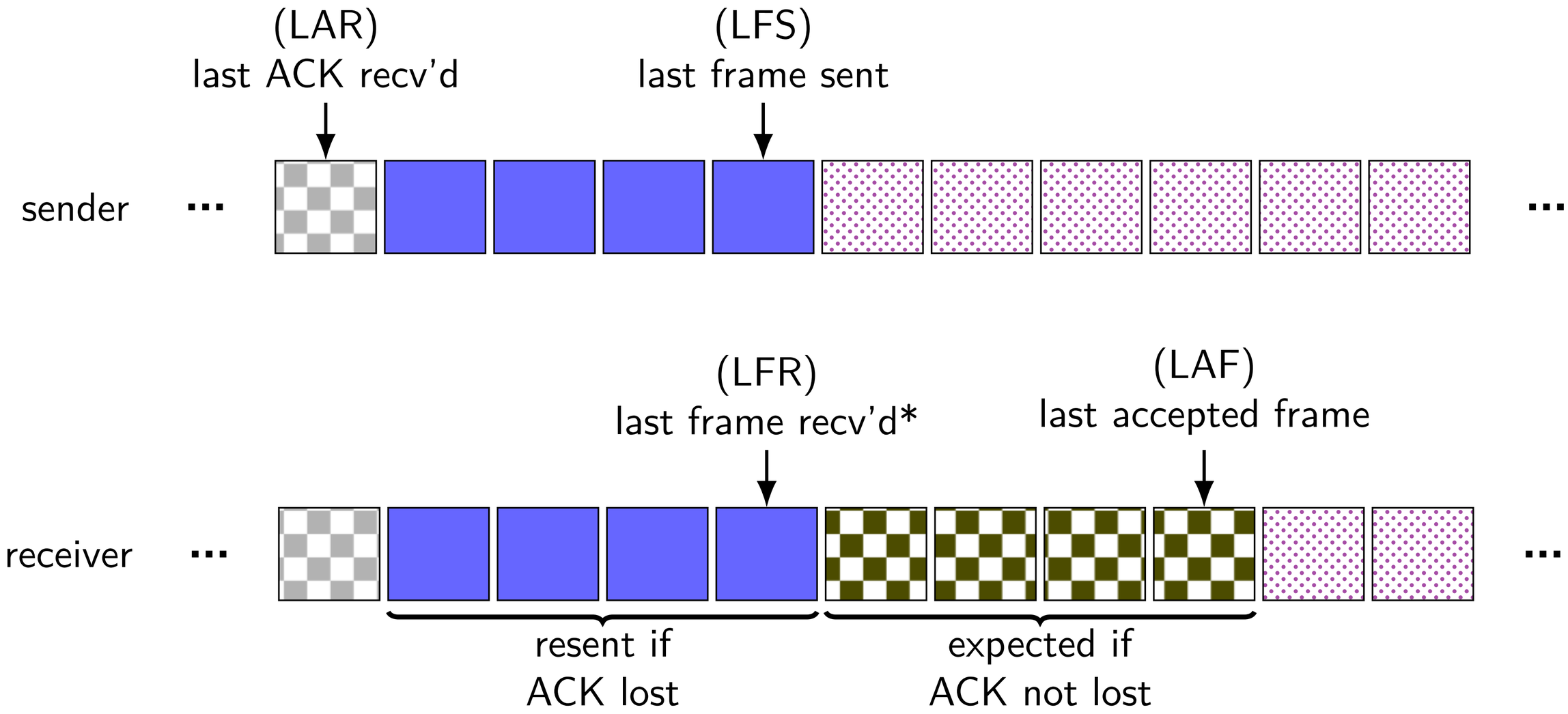
- doing $<$ and $>$ checks on sequence number, so they need to increase

would like to use smaller sequence numbers

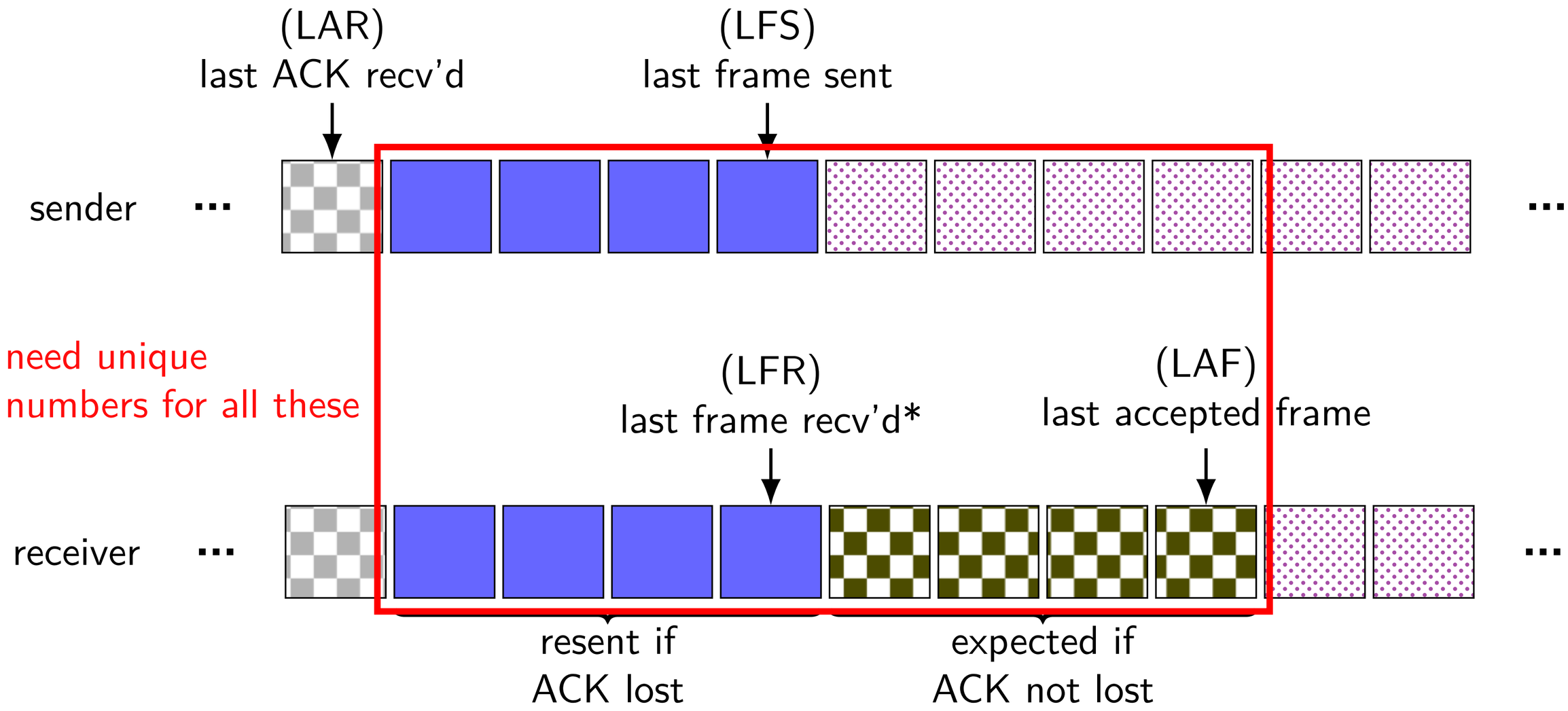
- think: transferring multi-gigabyte file

question: what goes wrong when we reuse sequeunce numbers?

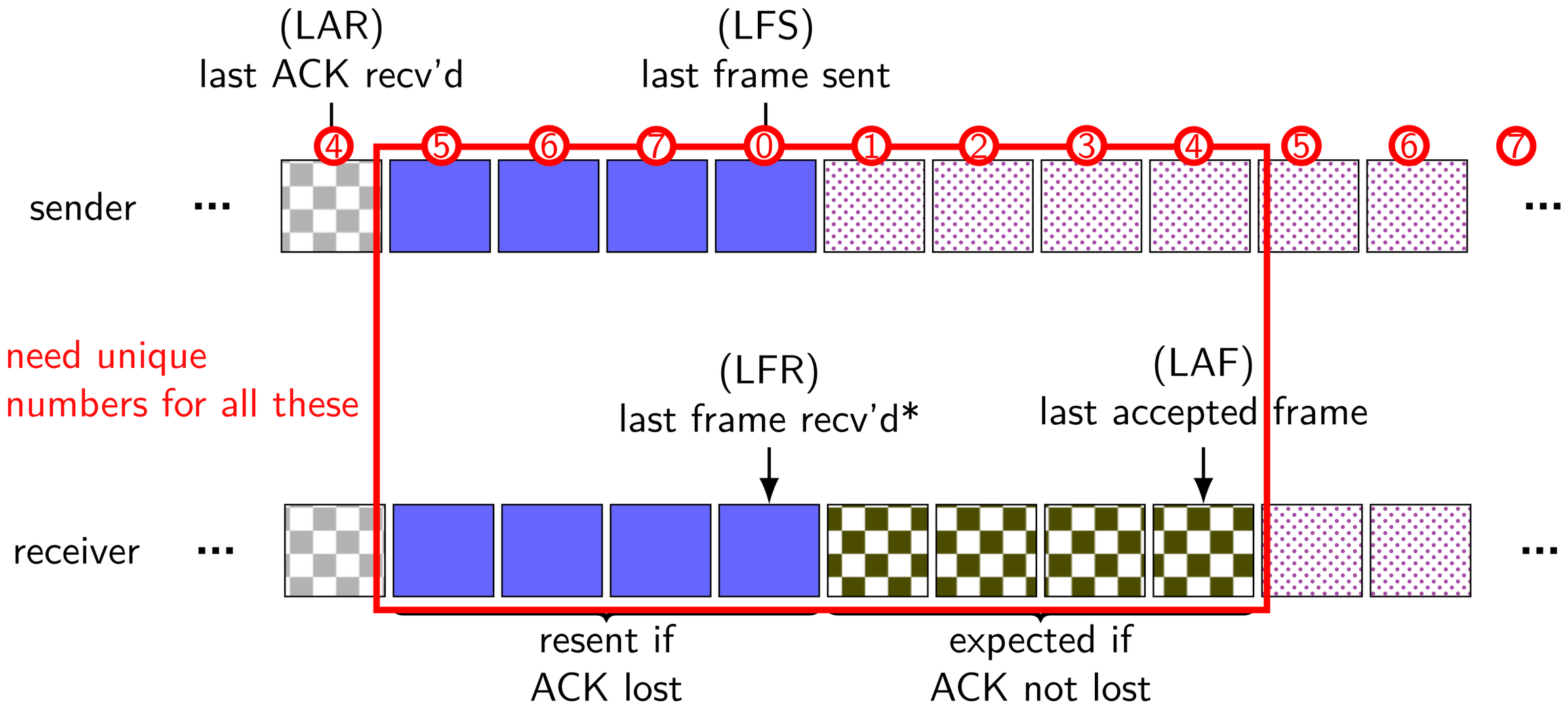
sender/receiver desync: missing ACKs



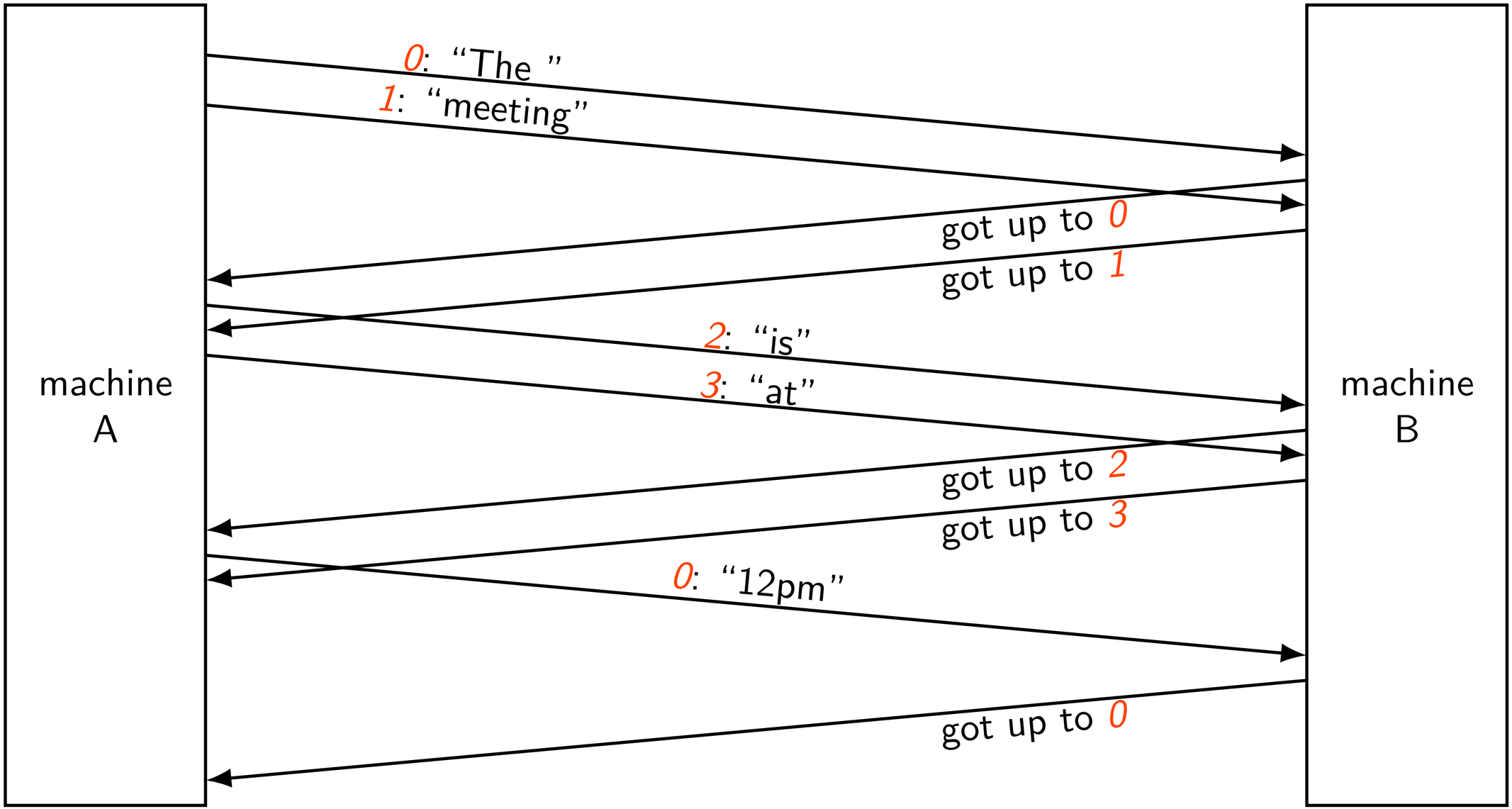
sender/receiver desync: missing ACKs



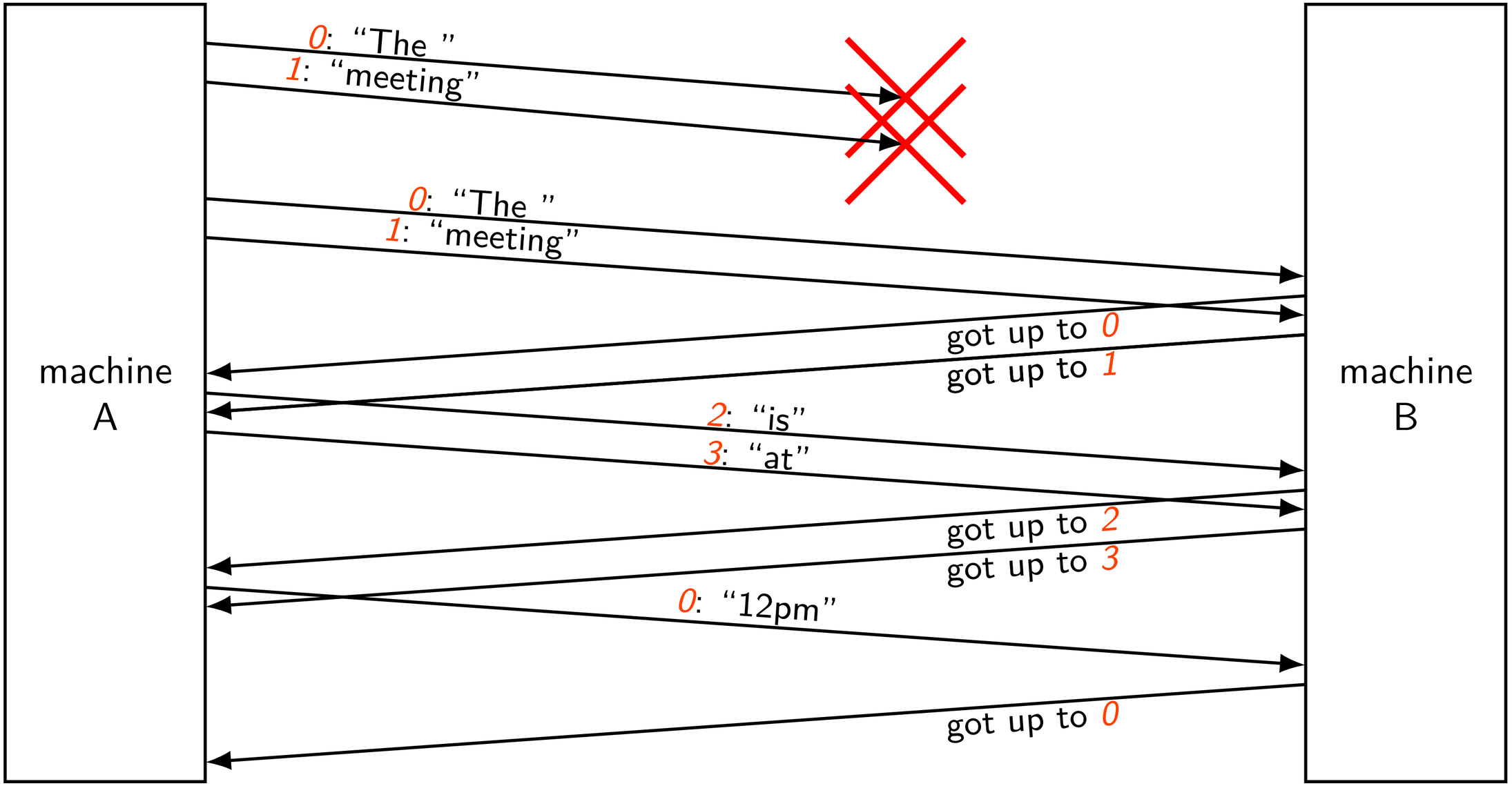
sender/receiver desync: missing ACKs



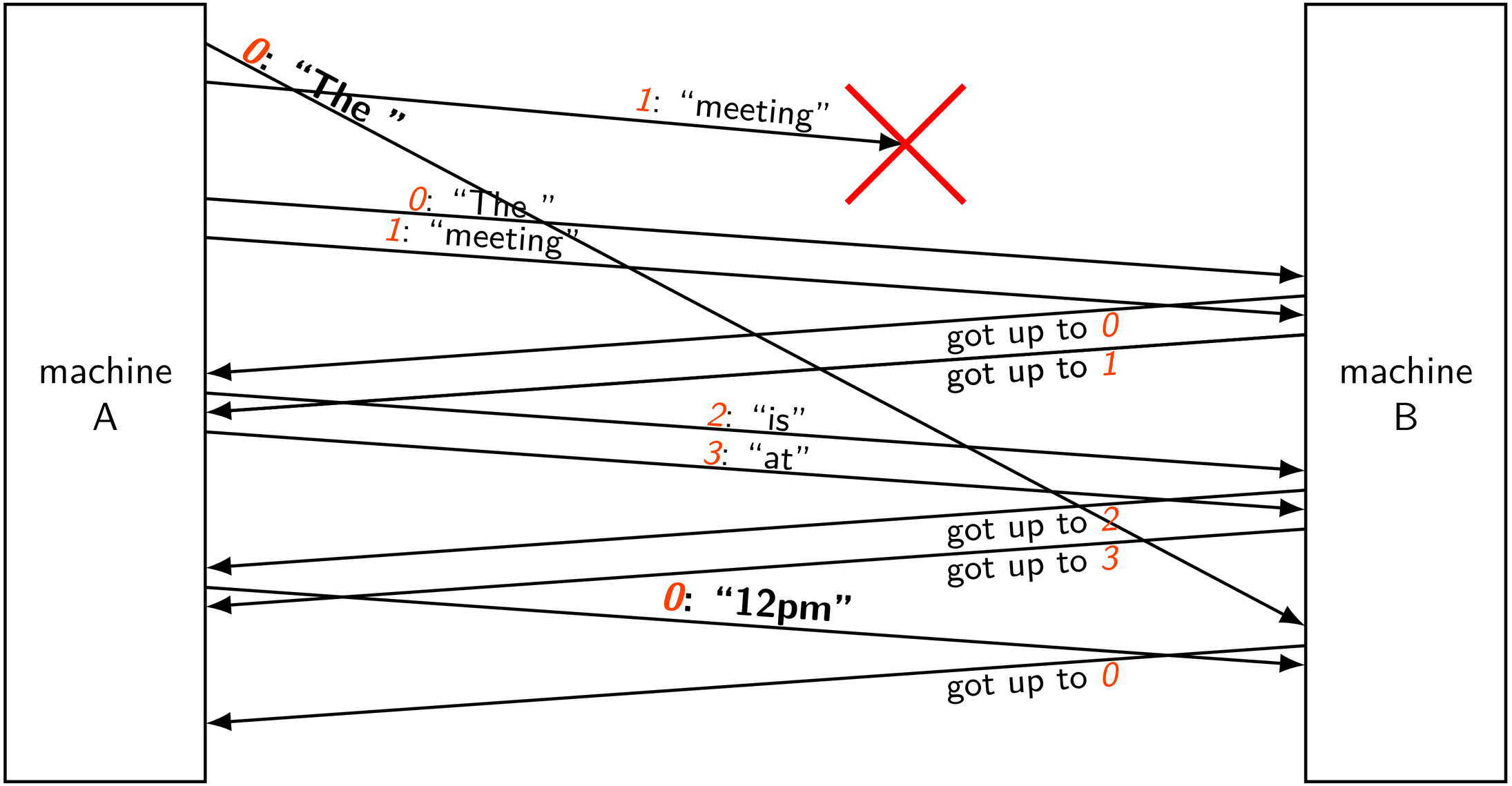
wraparound


machine A
machine B
0: “The ”
1: “meeting”
got up to 0
got up to 1
2: “is”
3: “at”
got up to 2
got up to 3
0: “12pm”
got up to 0

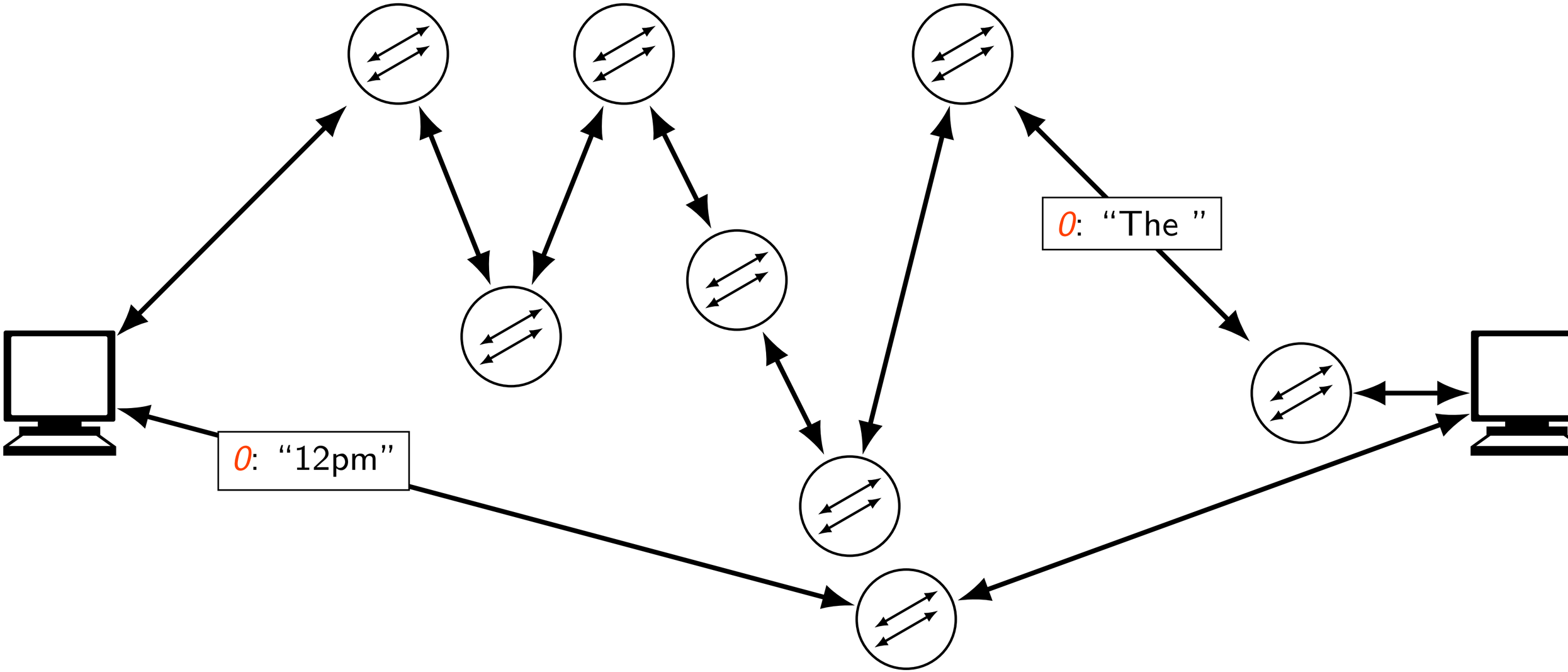
loss and resend?



very bad reordering



possible reason



sequence numbers in practice

TCP tries to assume 120 second "maximum segment lifetime"

- segment = TCP's name for a packet

original TCP used 32-bit sequence number identifying *byte* number (not segment number)

problem: means wraparound happens on modern (Gigibit+) links in seconds!

workaround: add *additional* 32-bit timestamp field

- used to detect/discard duplicates
- can also be used to set timeouts and/or window sizes